

面试题 (Java建设者出品)



面试题 (Java建设者出品)

Java 基础篇

Java 有哪些特点

Java 的特性

描述一下值传递和引用传递的区别

== 和 equals 区别是什么

String 中的 equals 是如何重写的

为什么重写 equals 方法必须重写 hashCode 方法

String s1 = new String("abc") 在内存中创建了几个对象

String 为什么是不可变的、jdk 源码中的 String 如何定义的、为什么这么设计。

static 关键字是干什么用的? 谈谈你的理解

final 关键字是干什么用的? 谈谈你的理解

抽象类和接口的区别是什么

重写和重载的区别

byte的取值范围是多少, 怎么计算出来的

HashMap 和 Hashtable 的区别

HashMap 和 HashSet 的区别

HashMap 的底层结构

HashMap 的长度为什么是 2 的幂次方

HashMap 多线程操作导致死循环问题

HashMap 线程安全的实现有哪些

讲一下 HashMap put 的过程

ConcurrentHashMap 底层实现

Integer 缓存池

UTF-8 和 Unicode 的关系

项目为 UTF-8 环境, char c = '中', 是否合法

Arrays.asList 获得的 List 应该注意什么

Collection 和 Collections 的区别

你知道 fail-fast 和 fail-safe 吗

ArrayList、LinkedList 和 Vector 的区别

Exception 和 Error 有什么区别

String、StringBuilder 和 StringBuffer 有什么区别

动态代理是基于什么原理

int 和 Integer 的区别

Java 提供了哪些 I/O 方式

谈谈你知道的设计模式

Comparator 和 Comparable 有什么不同

Object 类中一般都有哪些方法

Java 泛型和类型擦除

反射的基本原理, 反射创建类实例的三种方式是什么

强引用、弱引用、虚引用和幻象引用的区别

final、finally 和 finalize() 的区别

内部类有哪些分类，分别解释一下

说出几种常用的异常

静态绑定和动态绑定的区别

绑定

静态绑定

动态绑定

动态绑定和静态绑定的特点

HTTP 篇

HTTP 和 HTTPS 的区别

HTTP Get 和 Post 区别

什么是无状态协议，HTTP 是无状态协议吗，怎么解决

UDP 和 TCP 的区别

UDP 是什么

TCP 是什么

TCP 和 UDP 的不同

TCP 三次握手和四次挥手

TCP 三次握手

TCP 四次挥手

简述 HTTP1.0/1.1/2.0 的区别

HTTP 1.0

HTTP 1.1

HTTP 2.0

请你说一下 HTTP 常见的请求头

通用标头

实体标头

请求标头

响应标头

地址栏输入 URL 发生了什么

HTTPS 的工作原理

MySQL 篇

非关系型数据库和关系型数据库区别，优势比较

MySQL 事务四大特性

MySQL 常见存储引擎的区别

MyISAM 存储引擎的特点

InnoDB 存储引擎的特点

MyISAM 和 InnoDB 存储引擎的对比

MySQL 基础架构

连接器

查询缓存

分析器

优化器

执行器

SQL 的执行顺序

FROM 连接

ON 过滤

JOIN 连接

WHERE 过滤

GROUP BY

HAVING

SELECT

DISTINCT

ORDER BY

什么是临时表，何时删除临时表

MySQL 常见索引类型

varchar 和 char 的区别和使用场景

什么是 内连接、外连接、交叉连接、笛卡尔积

谈谈 SQL 优化的经验

操作系统篇

解释一下什么是操作系统

解释一下操作系统的主要目的是什么

操作系统的种类有哪些

操作系统结构

单体系统

分层系统

微内核

客户-服务器模式

什么是按需分页

多处理系统的优势

什么是内核

什么是实时系统

什么是虚拟内存

什么是进程和进程表

什么是线程，线程和进程的区别

使用多线程的好处是什么

什么是 RR 调度算法

导致系统出现死锁的情况

RAID 的不同级别

什么是 DMA

多线程编程的好处是什么

什么是设备驱动程序

进程间的通信方式

通信概念

解决方案

进程间状态模型

调度算法都有哪些

批处理中的调度

先来先服务

最短作业优先

最短剩余时间优先

交互式系统中的调度

轮询调度

优先级调度

最短进程优先

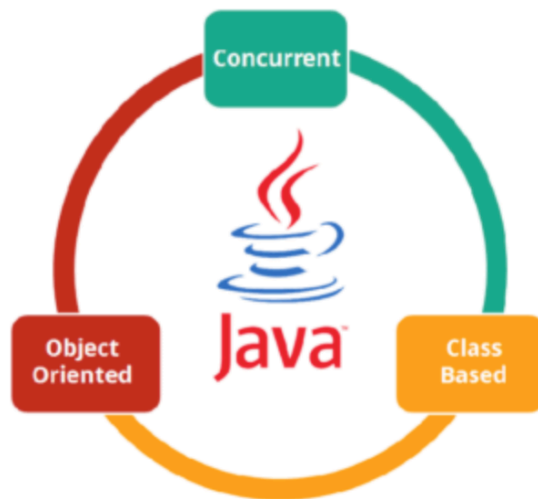
彩票调度

公平分享调度
页面置换算法都有哪些
影响调度程序的指标是什么
什么是僵尸进程

Java 基础篇

Java 有哪些特点

- **并发性的**：你可以在其中执行许多语句，而不必一次执行它
- **面向对象的**：基于类和面向对象的编程语言。
- **独立性的**：支持一次编写，到处运行的独立编程语言，即编译后的代码可以在支持 Java 的所有平台上运行。



Java 的特性

Java 的特性有如下这几点

- **简单**，Java 会让你的工作变得更加轻松，使你把关注点放在主要业务逻辑上，而不必关心指针、运算符重载、内存回收等与主要业务无关的功能。
- **便携性**，Java 是平台无关性的，这意味着在一个平台上编写的任何应用程序都可以轻松移植到另一个平台上。
- **安全性**，编译后会将所有的代码转换为字节码，人类无法读取。它使开发无病毒，无篡改的系统/应用成为可能。
- **动态性**，它具有适应不断变化的环境的能力，它能够支持动态内存分配，从而减少了内存浪费，提高了应用程序的性能。
- **分布式**，Java 提供的功能有助于创建分布式应用。使用 **远程方法调用 (RMI)**，程序可以通过网络调用另一个程序的方法并获取输出。您可以通过从互联网上的任何计算机上调用来访问文件。这是革命性的一个特点，对于当今的互联网来说太重要了。
- **健壮性**，Java 有强大的内存管理功能，在编译和运行时检查代码，它有助于消除错误。
- **高性能**，Java 最黑的科技就是字节码编程，Java 代码编译成的字节码可以轻松转换为本地机器代码。通过 JIT 即时编译器来实现高性能。

- **解释性**，Java 被编译成字节码，由 Java 运行时环境解释。
- **多线程性**，Java 支持多个执行线程（也称为轻量级进程），包括一组同步原语。这使得使用线程编程更加容易，Java 通过管程模型来实现线程安全性。

描述一下值传递和引用传递的区别

要想真正理解的话，可以参考这篇文章：<https://www.zhihu.com/question/31203609>

简单理解的话就是

值传递 是指在调用函数时将实际参数复制一份到函数中，这样的话如果函数对其传递过来的形式参数进行修改，将不会影响到实际参数

引用传递 是指在调用函数时将对象的地址直接传递到函数中，如果在对形式参数进行修改，将影响到实际参数的值。

== 和 equals 区别是什么

== 是 Java 中一种操作符，它有两种比较方式

- 对于 **基本数据类型** 来说，**==** 判断的是两边的 **值** 是否相等

```
1 public class DoubleCompareAndEquals {
2
3     Person person1 = new Person(24,"boy");
4     Person person2 = new Person(24,"girl");
5     int c = 10;
6
7     private void doubleCompare(){
8
9         int a = 10;
10        int b = 10;
11
12        System.out.println(a == b);
13        System.out.println(a == c);
14        System.out.println(person1.getId() == person2.getId());
15
16    }
17 }
```

- 对于 **引用类型** 来说，**==** 判断的是两边的 **引用** 是否相等，也就是判断两个对象是否指向了同一块内存区域。

```
1 private void equals(){
2
3     System.out.println(person1.getName().equals(person2.getName()));
4 }
```

equals 是 Java 中所有对象的父类，即 **Object** 类定义的一个方法。它只能比较对象，它表示的是引用双方的值是否相等。所以记住，并不是说 **==** 比较的就是引用是否相等，**equals** 比较的就是值，这需要区分来说的。

equals 用作对象之间的比较具有如下特性

- **自反性**：对于任何非空引用 x 来说，x.equals(x) 应该返回 true。
- **对称性**：对于任何非空引用 x 和 y 来说，若x.equals (y) 为 true，则y.equals (x) 也为 true。
- **传递性**：对于任何非空引用的值来说，有三个值，x、y 和 z，如果x.equals(y) 返回true，y.equals(z) 返回true，那么x.equals(z) 也应该返回true。
- **一致性**：对于任何非空引用 x 和 y 来说，如果 x.equals(y) 相等的话，那么它们必须始终相等。
- **非空性**：对于任何非空引用的值 x 来说，x.equals(null) 必须返回 false。

String 中的 equals 是如何重写的

String 代表的是 Java 中的 **字符串**，String 类比较特殊，它整个类都是被 **final** 修饰的，也就是说，String 不能被任何类继承，任何 **修改** String 字符串的方法都是创建了一个新的字符串。

equals 方法是 Object 类定义的方法，Object 是所有类的父类，当然也包括 String，String 重写了 **equals** 方法，下面我们来看看是怎么重写的

```
public boolean equals(Object anObject) {
    if (this == anObject) {
        return true;
    }
    if (anObject instanceof String) {
        String anotherString = (String)anObject;
        int n = value.length;
        if (n == anotherString.value.length) {
            char v1[] = value;
            char v2[] = anotherString.value;
            int i = 0;
            while (n-- != 0) {
                if (v1[i] != v2[i])
                    return false;
                i++;
            }
            return true;
        }
    }
    return false;
}
```

- 首先会判断要比较的两个字符串它们的 **引用** 是否相等。如果引用相等的话，直接返回 true，不相等的话继续下面的判断
- 然后再判断被比较的对象是否是 String 的实例，如果不是的话直接返回 false，如果是的话，再比较两个字符串的长度是否相等，如果长度不想等的话也就没有比较的必要了；长度如果相同，会比较字符串中的每个 **字符** 是否相等，一旦有一个字符不相等，就会直接返回 false。

下面是它的流程图

在 JDK1.7 及以后调用 `intern` 方法是判断运行时常量池中是否有指定的字符串，如果没有的话，就把字符串添加到常量池中，并返回常量池中的对象。

验证过程如下

```
1 private void StringOverrideEquals(){
2
3     String s1 = "aaa";
4     String s2 = "aa" + new String("a");
5     String s3 = new String("aaa");
6
7     System.out.println(s1.intern().equals(s1));
8     System.out.println(s1.intern().equals(s2));
9     System.out.println(s3.intern().equals(s1));
10
11 }
```

- 首先 s1.intern().equals(s1) 这个无论如何都返回 true，因为 s1 字符串创建出来就已经在常量池中存在了。
- 然后第二条语句返回 false，因为 s1 返回的是常量池中的对象，而 s2 返回的是堆中的对象
- 第三条语句 s3.intern().equals(s1)，返回 true，因为 s3 对象虽然在堆中创建了一个对象，但是 s3 中的 "aaa" 返回的是常量池中的对象。

```
public boolean equals(Object anObject) { anObject: "aaa"
    if (this == anObject) { anObject: "aaa"
        return true;
    }
    if (anObject instanceof String) {
        String anotherString = (String)anObject;
        int n = value.length;
        if (n == anotherString.value.length) {
            char v1[] = value;
            char v2[] = anotherString.value;
            int i = 0;
            while (n-- != 0) {
                if (v1[i] != v2[i])
                    return false;
                i++;
            }
            return true;
        }
    }
    return false;
}
```

为什么重写 equals 方法必须重写 hashCode 方法

equals 方法和 hashCode 都是 Object 中定义的方法，它们经常被一起重写。

equals 方法是用来比较对象大小是否相等的方法，hashCode 方法是用来判断每个对象 hash 值的一种方法。如果只重写 equals 方法而不重写 hashCode 方法，很可能造成两个不同的对象，它们的 hashCode 也相等，造成冲突。比如

```
1 String str1 = "通话";
2 String str2 = "重地";
```

它们两个的 hashCode 相等，但是 equals 可不相等。

我们来看一下 hashCode 官方的定义

```
* {@link java.util.HashMap}.
* 返回一个对象的 hash code 值，支持此方法是为了 HashMap 处理使用
*
* <p>
* The general contract of {@code hashCode} is:
* 遵守 hashCode 的一致约定
* <ul>
* <li>Whenever it is invoked on the same object more than once during
*     an execution of a Java application, the {@code hashCode} method
*     must consistently return the same integer, provided no information
*     used in {@code equals} comparisons on the object is modified.
*     This integer need not remain consistent from one execution of an
*     application to another execution of the same application.
*     无论每次、何时在运行 Java 程序执行期间调用同一个对象，这个 hashCode 方法都必须
*     一致性的返回同一个 integer 值，前提是在 equals 的比较中没有被修改。
*     在一个应用程序与另一个程序的执行过程中，执行 hashCode 方法所返回的值可以不一致。
*
*
* <li>If two objects are equal according to the {@code equals(Object)}
*     method, then calling the {@code hashCode} method on each of
*     the two objects must produce the same integer result.
*     如果两个对象根据 equals(Object) 比较相等，然后调用两个对象上各自的 hashCode 方法
*     必须得到相同 integer 结果。
*
* <li>It is not required that if two objects are unequal
*     according to the {@link java.lang.Object#equals(java.lang.Object)}
*     method, then calling the {@code hashCode} method on each of the
*     two objects must produce distinct integer results. However, the
*     programmer should be aware that producing distinct integer results
*     for unequal objects may improve the performance of hash tables.
* </ul>
* 必须意识到一个问题，那就是如果两个对象不相等，那么调用 hashCode 方法必须返回
* 不同的结果，为不同对象生成不同的 hashCode 会提高哈希表的性能。
*
* <p>
* As much as is reasonably practical, the hashCode method defined by
* class {@code Object} does return distinct integers for distinct
*     * objects. (This is typically implemented by converting the internal
*     * address of the object into an integer, but this implementation
*     * technique is not required by the
*     * Java&trade; programming language.)
* 在合理可行的范围内，由 Object 类定义的 hashCode 方法确实能为不同的对象返回
* 不同的整数。hashCode 是将对象的地址转换为整数来实现的。
```

总结起来就是

- 如果在 Java 运行期间对同一个对象调用 hashCode 方法后，无论调用多少次，都应该返回相同的 hashCode，但是在不同的 Java 程序中，执行 hashCode 方法返回的值可能不一致。
- 如果两个对象的 equals 相等，那么 hashCode 必须相同
- 如果两个对象 equals 不相等，那么 hashCode 也有可能相同，所以需要重写 hashCode 方法，因为你不知道 hashCode 的底层构造（反正我是不知道，有大牛可以传授传授），所以你需要重写 hashCode 方法，来为不同的对象生成不同的 hashCode 值，这样能够提高不同对象的访问速度。
- hashCode 通常是将地址转换为整数来实现的。

String s1 = new String("abc") 在内存中创建了几个对象

一个或者两个，String s1 是声明了一个 String 类型的 s1 变量，它不是对象。使用 new 关键字会在堆中创建一个对象，另外一个对象是 abc，它会在常量池中创建，所以一共创建了两个对象；如果 abc 在常量池中已经存在的话，那么就会创建一个对象。

详细请翻阅作者的另外一篇文章 [一篇与众不同的 String、StringBuffer、StringBuilder 详解](#)

String 为什么是不可变的、jdk 源码中的 String 如何定义的、为什么这么设计。

首先了解一下什么是 **不可变对象**，不可变对象就是一经创建后，其对象的内部状态不能被修改，啥意思呢？也就是说不可变对象需要遵守下面几条原则

- 不可变对象的内部属性都是 final 的
- 不可变对象的内部属性都是 private 的
- 不可变对象不能提供任何可以修改内部状态的方法、setter 方法也不行
- 不可变对象不能被继承和扩展

与其说问 String 为什么是不可变的，不如说如何把 String 设计成不可变的。

String 类是一种对象，它是独立于 Java 基本数据类型而存在的，String 你可以把它理解为字符串的集合，String 被设计为 final 的，表示 String 对象一经创建后，它的值就不能再被修改，任何对 String 值进行修改的方法就是重新创建一个字符串。String 对象创建后会存在于运行时常量池中，运行时常量池是属于方法区的一部分，JDK1.7 后把它移到了堆中。

不可变对象不是真的不可变，可以通过 **反射** 来对其内部的属性和值进行修改，不过一般我们不这样做。

static 关键字是干什么用的？谈谈你的理解

static 是 Java 中非常重要的关键字，static 表示的概念是 **静态的**，在 Java 中，static 主要用来

- 修饰变量，static 修饰的变量称为 **静态变量**、也称为 **类变量**，类变量属于类所有，对于不同的类来说，static 变量只有一份，static 修饰的变量位于方法区中；static 修饰的变量能够直接通过 **类名.变量名** 来进行访问，不用通过实例化类再进行使用。
- 修饰方法，static 修饰的方法被称为 **静态方法**，静态方法能够直接通过 **类名.方法名** 来使用，在静态方法内部不能使用非静态属性和方法
- static 可以修饰代码块，主要分为两种，一种直接定义在类中，使用 **static{}**，这种被称为 **静态代码块**，一种是在类中定义 **静态内部类**，使用 **static class xxx** 来进行定义。
- static 可以用于静态导包，通过使用 **import static xxx** 来实现，这种方式一般不推荐使用
- static 可以和单例模式一起使用，通过双重检查锁来实现线程安全的单例模式。

详情请参考这篇文章 [一篇 static 还能难得住我？](#)

final 关键字是干什么用的？谈谈你的理解

final 是 Java 中的关键字，它表示的意思是 **不可变的**，在 Java 中，final 主要用来

- 修饰类，final 修饰的类不能被继承，不能被继承的意思就是不能使用 **extends** 来继承被 final 修饰的类。
- 修饰变量，final 修饰的变量不能被改写，不能被改写的意思有两种，对于基本数据类型来说，final 修饰的变量，其值不能被改变，final 修饰的对象，对象的引用不能被改变，但是对象内部的属性可以被修改。final 修饰的变量在某种程度上起到了 **不可变** 的效果，所以，可以用来保护只读数据，尤其是在并发编程中，因为明确的不能再为 final 变量进行赋值，有利于减少额外的同步开销。
- 修饰方法，final 修饰的方法不能被重写。
- final 修饰符和 Java 程序性能优化没有必然联系

抽象类和接口的区别是什么

抽象类和接口都是 Java 中的关键字，抽象类和接口中都允许进行方法的定义，而不用具体的方法实现。抽象类和接口都允许被继承，它们广泛的应用于 JDK 和框架的源码中，来实现多态和不同的设计模式。

不同点在于

- **抽象级别不同**：类、抽象类、接口其实是三种不同的抽象级别，抽象程度依次是 接口 > 抽象类 > 类。在接口中，只允许进行方法的定义，不允许有方法的实现，抽象类中可以进行方法的定义和实现；而类中只允许进行方法的实现，我说的方法的定义是不允许在方法后面出现 **{ }**
- **使用的关键字不同**：类使用 **class** 来表示；抽象类使用 **abstract class** 来表示；接口使用 **interface** 来表示
- **变量**：接口中定义的变量只能是公共的静态常量，抽象类中的变量是普通变量。

重写和重载的区别

在 Java 中，重写和重载都是对同一方法的不同表现形式，下面我们针对重写和重载做一下简单的区分

- **子父级关系不同**，重写是针对子级和父级的不同表现形式，而重载是在同一类中的不同表现形式；
- **概念不同**，子类重写父类的方法一般使用 **@override** 来表示；重写后的方法其方法的声明和参数类型、顺序必须要与父类完全一致；重载是针对同一类中概念，它要求重载的方法必须满足下面任何一个要求：方法参数的顺序，参数的个数，参数的类型任意一个保持不同即可。

byte的取值范围是多少，怎么计算出来的

byte 的取值范围是 -128 -> 127 之间，一共是 256 个。一个 byte 类型在计算机中占据一个字节，那么就是 8 bit，所以最大就是 $2^7 = 1111\ 1111$ 。

Java 中用 **补码** 来表示二进制数，补码的最高位是符号位，最高位用 0 表示正数，最高位 1 表示负数，正数的补码就是其 **本身**，由于最高位是符号位，所以正数表示的就是 0111 1111，也就是 127。最大负数就是 1111 1111，这其中会涉及到两个 0，一个 +0，一个 -0，+0 归为正数，也就是 0，-0 归为负数，也就是 -128，所以 byte 的范围就是 -128 - 127。

HashMap 和 Hashtable 的区别

相同点

HashMap 和 Hashtable 都是基于哈希表实现的，其内部每个元素都是 `key-value` 键值对，HashMap 和 Hashtable 都实现了 Map、Cloneable、Serializable 接口。

不同点

- 父类不同：HashMap 继承了 `AbstractMap` 类，而 Hashtable 继承了 `Dictionary` 类

```
public class Hashtable<K,V>
    extends Dictionary<K,V>
    implements Map<K,V>, Cloneable, java.io.Serializable {
```

```
public class HashMap<K,V> extends AbstractMap<K,V>
    implements Map<K,V>, Cloneable, Serializable {
```

- 空值不同：HashMap 允许空的 key 和 value 值，Hashtable 不允许空的 key 和 value 值。HashMap 会把 Null key 当做普通的 key 对待。不允许 null key 重复。

```
/**
 * @param value the value
 * @return the previous value of the specified key in this hashtable,
 *         or null if it did not have one
 * @exception NullPointerException if the key or value is
 *         null
 * @see Object#equals(Object)
 * @see #get(Object)
 */
```

如果 key 或者 value 是 null，就抛出空指针异常

```
public synchronized V put(K key, V value) {
    // Make sure the value is not null
    if (value == null) {
        throw new NullPointerException();
    }
}
```

- 线程安全性：HashMap 不是线程安全的，如果多个外部操作同时修改 HashMap 的数据结构比如 add 或者是 delete，必须进行同步操作，仅仅对 key 或者 value 的修改不是改变数据结构的操作。可以选择构造线程安全的 Map 比如 `Collections.synchronizedMap` 或者是 `ConcurrentHashMap`。而 Hashtable 本身就是线程安全的容器。
- 性能方面：虽然 HashMap 和 Hashtable 都是基于 `单链表` 的，但是 HashMap 进行 put 或者 get 操作，可以达到常数时间的性能；而 Hashtable 的 put 和 get 操作都是加了 `synchronized` 锁的，所以效率很差。

HashMap

```
public V get(Object key) {  
    Node<K,V> e;  
    return (e = getNode(hash(key), key)) == null ? null : e.value;  
}  
  
public V put(K key, V value) {  
    return putVal(hash(key), key, value, onlyIfAbsent: false, evict: true);  
}
```

HashTable

```
public synchronized V get(Object key) {  
    Entry<?,?> tab[] = table;  
    int hash = key.hashCode();  
    int index = (hash & 0x7FFFFFFF) % tab.length;  
    for (Entry<?,?> e = tab[index] ; e != null ; e = e.next) {  
        if ((e.hash == hash) && e.key.equals(key)) {  
            return (V)e.value;  
        }  
    }  
    return null;  
}  
  
public synchronized V put(K key, V value) {  
    // Make sure the value is not null  
    if (value == null) {  
        throw new NullPointerException();  
    }  
  
    // Makes sure the key is not already in the hashtable.  
    Entry<?,?> tab[] = table;  
    int hash = key.hashCode();  
    int index = (hash & 0x7FFFFFFF) % tab.length;  
    /unchecked/  
    Entry<K,V> entry = (Entry<K,V>)tab[index];  
    for (; entry != null ; entry = entry.next) {  
        if ((entry.hash == hash) && entry.key.equals(key)) {  
            V old = entry.value;  
            entry.value = value;  
            return old;  
        }  
    }  
  
    addEntry(hash, key, value, index);  
    return null;  
}
```

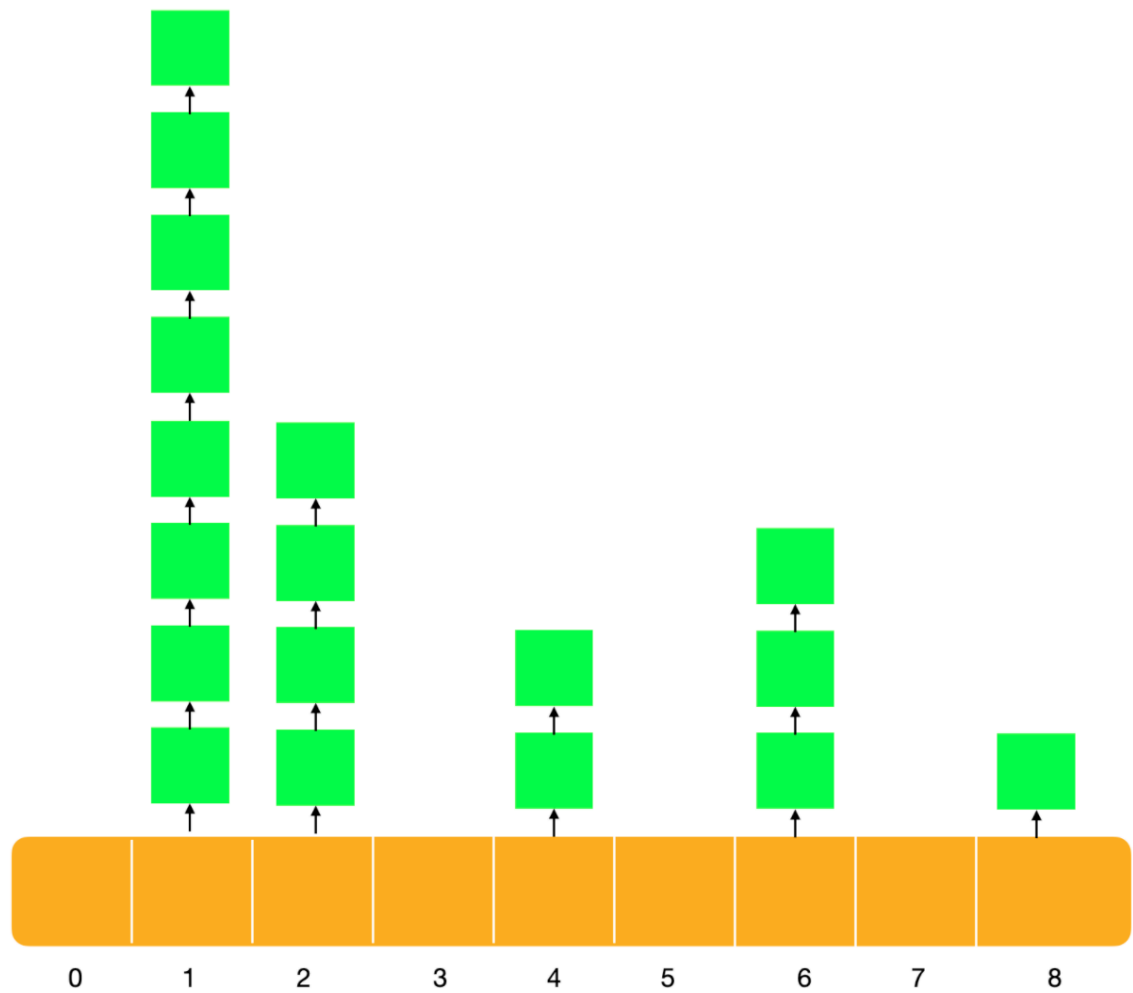
- 初始容量不同：HashTable 的初始长度是11，之后每次扩充容量变为之前的 $2n+1$ （ n 为上一次的长度）而 HashMap 的初始长度为16，之后每次扩充变为原来的两倍。创建时，如果给定了容量初始值，那么HashTable 会直接使用你给定的大小，而 HashMap 会将其扩充为2的幂次方大小。

HashMap 和 HashSet 的区别

HashSet 继承于 AbstractSet 接口，实现了 Set、Cloneable、java.io.Serializable 接口。HashSet 不允许集合中出现重复的值。HashSet 底层其实就是 HashMap，所有对 HashSet 的操作其实就是对 HashMap 的操作。所以 HashSet 也不保证集合的顺序，也不是线程安全的容器。

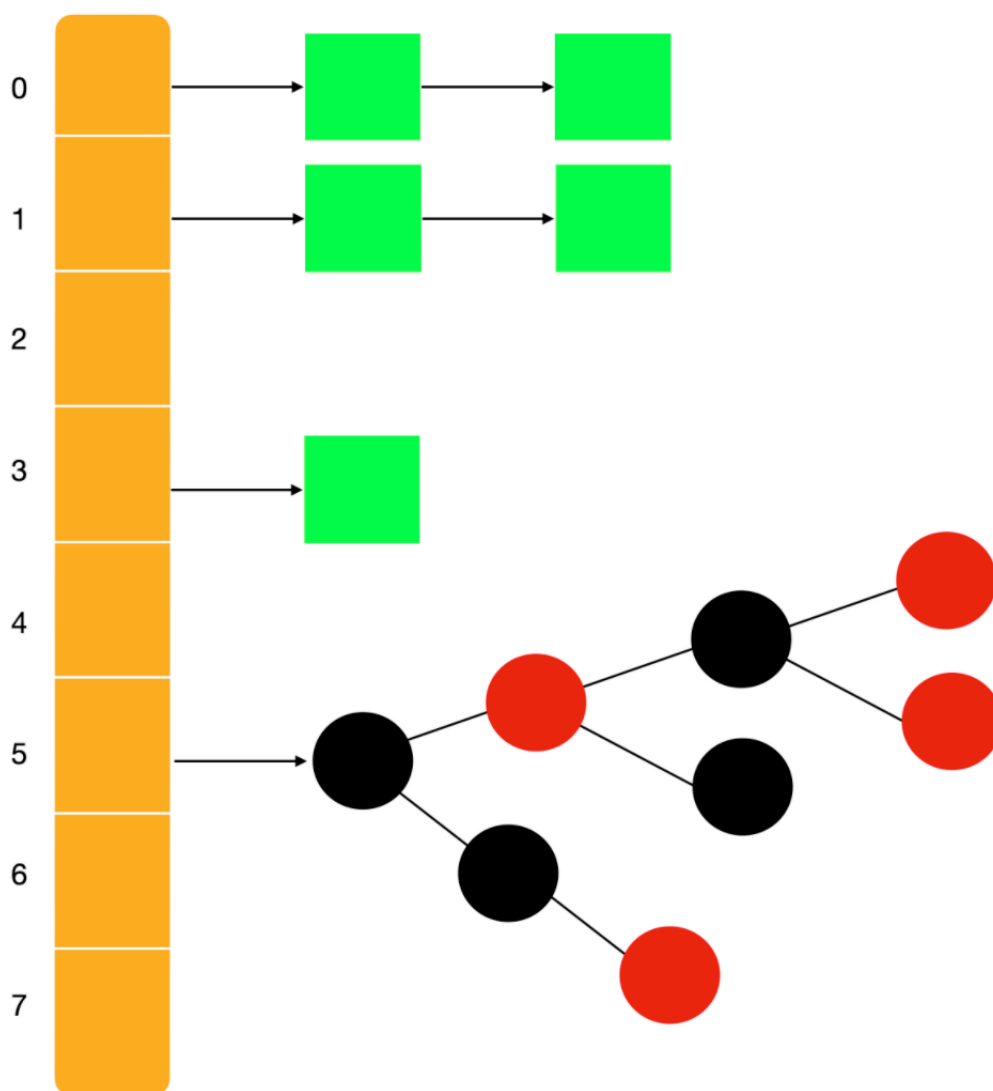
HashMap 的底层结构

JDK1.7 中，HashMap 采用 位桶 + 链表 的实现，即使用 链表 来处理冲突，同一 hash 值的链表都存储在一个数组中。但是当位于一个桶中的元素较多，即 hash 值相等的元素较多时，通过 key 值依次查找的效率较低。



HashMap 大致结构

所以，与 JDK 1.7 相比，JDK 1.8 在底层结构方面做了一些改变，当每个桶中元素大于 8 的时候，会转变为红黑树，目的就是优化查询效率。



HashMap 的长度为什么是 2 的幂次方

这道题我想了几天，之前和群里小伙伴们探讨每日一题的时候，问他们为什么 $\text{length} \% \text{hash} == (n - 1) \& \text{hash}$ ，它们说相等的前提是 length 的长度 2 的幂次方，然后我回了一句难道 length 还能不是 2 的幂次方吗？其实是我没有搞懂因果关系，因为 HashMap 的长度是 2 的幂次方，所以使用余数来判断在桶中的下标。如果 length 的长度不是 2 的幂次方，小伙伴们可以举个例子来试试

例如长度为 9 时候， $3 \& (9-1) = 0$ ， $2 \& (9-1) = 0$ ，都在 0 上，碰撞了；

这样会增大 HashMap 碰撞的几率。

HashMap 多线程操作导致死循环问题

HashMap 不是一个线程安全的容器，在高并发场景下，应该使用 `ConcurrentHashMap`，在多线程场景下使用 HashMap 会造成死循环问题（基于 JDK1.7），出现问题的位置在 `rehash` 处，也就是

```

1  do {
2      Entry<K,V> next = e.next; // <---假设线程一执行到这里就被调度挂起了
3      int i = indexFor(e.hash, newCapacity);
4      e.next = newTable[i];
5      newTable[i] = e;
6      e = next;
7  } while (e != null);

```

这是 JDK1.7 的 rehash 代码片段，在并发的场景下会形成环。

JDK1.8 也会造成死循环问题。

HashMap 线程安全的实现有哪些

因为 HashMap 不是一个线程安全的容器，所以并发场景下推荐使用 `ConcurrentHashMap`，或者使用线程安全的 HashMap，使用 `Collections` 包下的线程安全的容器，比如说

```

1  Collections.synchronizedMap(new HashMap());

```

还可以使用 `HashTable`，它也是线程安全的容器，基于 key-value 存储，经常用 `HashMap` 和 `HashTable` 做比较就是因为 `HashTable` 的数据结构和 `HashMap` 相同。

上面效率最高的就是 `ConcurrentHashMap`。

讲一下 HashMap put 的过程

首先会使用 hash 函数来计算 key，然后执行真正的插入方法

```

1  final V putVal(int hash, K key, V value, boolean onlyIfAbsent,
2                boolean evict) {
3      Node<K,V>[] tab; Node<K,V> p; int n, i;
4      // 如果table为null 或者没有为table分配内存, 就resize一次
5      if ((tab = table) == null || (n = tab.length) == 0)
6          n = (tab = resize()).length;
7      // 指定hash值节点为空则直接插入, 这个(n - 1) & hash才是表中真正的哈希
8      if ((p = tab[i = (n - 1) & hash]) == null)
9          tab[i] = newNode(hash, key, value, null);
10     // 如果不为空
11     else {
12         Node<K,V> e; K k;
13         // 计算表中的这个真正的哈希值与要插入的key.hash相比
14         if (p.hash == hash &&
15             ((k = p.key) == key || (key != null && key.equals(k))))
16             e = p;
17         // 若不同的话, 并且当前节点已经在 TreeNode 上了
18         else if (p instanceof TreeNode)
19             // 采用红黑树存储方式
20             e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, value);
21         // key.hash 不同并且也不再 TreeNode 上, 在链表上找到 p.next==null
22         else {
23             for (int binCount = 0; ; ++binCount) {
24                 if ((e = p.next) == null) {

```

```

25         // 在表尾插入
26         p.next = newNode(hash, key, value, null);
27         // 新增节点后如果节点个数到达阈值, 则进入 treeifyBin() 进行再次判断
28         if (binCount >= TREEIFY_THRESHOLD - 1) // -1 for 1st
29             treeifyBin(tab, hash);
30         break;
31     }
32     // 如果找到了同hash、key的节点, 那么直接退出循环
33     if (e.hash == hash &&
34         ((k = e.key) == key || (key != null && key.equals(k))))
35         break;
36     // 更新 p 指向下一节点
37     p = e;
38 }
39 }
40 // map中含有旧值, 返回旧值
41 if (e != null) { // existing mapping for key
42     V oldValue = e.value;
43     if (!onlyIfAbsent || oldValue == null)
44         e.value = value;
45     afterNodeAccess(e);
46     return oldValue;
47 }
48 }
49 // map调整次数 + 1
50 ++modCount;
51 // 键值对的数量达到阈值, 需要扩容
52 if (++size > threshold)
53     resize();
54 afterNodeInsertion(evict);
55 return null;
56 }

```

HashMap put 方法的核心就是在 `putval` 方法, 它的插入过程如下

- 首先会判断 HashMap 中是否是新构建的, 如果是的话会首先进行 `resize`
- 然后判断需要插入的元素在 HashMap 中是否已经存在 (说明出现了碰撞情况), 如果不存在, 直接生成新的k-v 节点存放, 再判断是否需要扩容。
- 如果要插入的元素已经存在的话, 说明发生了冲突, 这就会转换成链表或者红黑树来解决冲突, 首先判断链表中的 hash, key 是否相等, 如果相等的话, 就用新值替换旧值, 如果节点是属于 `TreeNode` 类型, 会直接在红黑树中进行处理, 如果 hash, key 不相等也不属于 `TreeNode` 类型, 会直接转换为链表处理, 进行链表遍历, 如果链表的 next 节点是 null, 判断是否转换为红黑树, 如果不转换的话, 在遍历过程中找到 key 完全相等的节点, 则用新节点替换老节点

ConcurrentHashMap 底层实现

ConcurrentHashMap 是线程安全的 Map, 它也是高并发场景下的首选数据结构, ConcurrentHashMap 底层是使用 `分段锁` 来实现的。

Integer 缓存池

Integer 缓存池也就是 `IntegerCache`，它是 `Integer` 的静态内部类。

```
private static class IntegerCache {
    static final int low = -128;
    static final int high;
    static final Integer cache[];

    static {
        // high value may be configured by property
        // 所以字符缓存的范围是 -128 - 127 之间
        int h = 127;
        String integerCacheHighPropValue =
            sun.misc.VM.getSavedProperty("java.lang.Integer.IntegerCache.high");
        if (integerCacheHighPropValue != null) {
            try {
                int i = parseInt(integerCacheHighPropValue);
                i = Math.max(i, 127);
                // Maximum array size is Integer.MAX_VALUE
                h = Math.min(i, Integer.MAX_VALUE - (-low) - 1);
            } catch (NumberFormatException nfe) {
                // If the property cannot be parsed into an int, ignore it.
            }
        }
        high = h;

        cache = new Integer[(high - low) + 1];
        int j = low;
        for(int k = 0; k < cache.length; k++)
            cache[k] = new Integer(j++);

        // range [-128, 127] must be interned (JLS7 5.1.7)
        assert IntegerCache.high >= 127;
    }

    private IntegerCache() {}
}
```

它的默认值用于缓存 -128 - 127 之间的数字，如果有 -128 - 127 之间的数字的话，使用 `new Integer` 不用创建对象，会直接从缓存池中取，此操作会减少堆中对象的分配，有利于提高程序的运行效率。

例如创建一个 `Integer a = 24`，其实是调用 `Integer` 的 `valueOf`，可以通过反编译得出这个结论

```
void testInteger();
Code:
    0: bipush          10
    2: invokestatic    #2          // Method java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
    5: astore_1
    6: return
```

然后我们看一下 `valueOf` 方法

```
public static Integer valueOf(int i) {
    if (i >= IntegerCache.low && i <= IntegerCache.high)
        return IntegerCache.cache[i + (-IntegerCache.low)];
    return new Integer(i);
}
```

如果在指定缓存池范围内的话，会直接返回缓存的值而不用创建新的 Integer 对象。

缓存的大小可以使用 `XX:AutoBoxCacheMax` 来指定，在 VM 初始化

时，`java.lang.Integer.IntegerCache.high` 属性会设置和保存在 `sun.misc.VM` 的私有系统属性中。

UTF-8 和 Unicode 的关系

由于每个国家都有自己独有的字符编码，所以Unicode 的发展旨在创建一个新的标准，用来映射当今使用的大多数语言中的字符，这些字符有一些不是必要的，但是对于创建文本来说却是不可或缺的。

Unicode 统一了所有字符的编码，是一个 Character Set，也就是字符集，字符集只是给所有的字符一个唯一编号，但是却没有规定如何存储，不同的字符其存储空间不一样，有的需要一个字节就能存储，有的则需要2、3、4个字节。

UTF-8 只是众多能够对文本字符进行 解码 的一种方式，它是一种变长的方式。UTF-8 代表 8 位一组表示 Unicode 字符的格式，使用 1 - 4 个字节来表示字符。

```
1  U+ 0000 ~ U+ 007F: 0XXXXXXX
2  U+ 0080 ~ U+ 07FF: 110XXXXX 10XXXXXX
3  U+ 0800 ~ U+ FFFF: 1110XXXX 10XXXXXX 10XXXXXX
4  U+10000 ~ U+1FFFF: 11110XXX 10XXXXXX 10XXXXXX 10XXXXXX
```

可以看到，UTF-8 通过开头的标志位位数实现了变长。对于单字节字符，只占用一个字节，实现了向下兼容 ASCII，并且能和 UTF-32 一样，包含 Unicode 中的所有字符，又能有效减少存储传输过程中占用的空间。

项目为 UTF-8 环境，char c = '中'，是否合法

可以，因为 Unicode 编码采用 2 个字节的编码，UTF-8 是 Unicode 的一种实现，它使用可变长度的字符集进行编码，char c = '中' 是两个字节，所以能够存储。合法。

Arrays.asList 获得的 List 应该注意什么

`Arrays.asList` 是 Array 中的一个静态方法，它能够实现把数组转换成为 List 序列，需要注意下面几点

- Arrays.asList 转换完成后的 List 不能再进行结构化的修改，什么是结构化的修改？就是不能再进行任何 List 元素的增加或者减少的操作。

```
1  public static void main(String[] args) {
2      Integer[] integer = new Integer[] { 1, 2, 3, 4 };
3      List integetList = Arrays.asList(integer);
4      integetList.add(5);
5  }
```

结果会直接抛出

```
1  Exception in thread "main" java.lang.UnsupportedOperationException
```

我们看一下源码就能发现问题


```
1 // 这是 java.util.Arrays 的内部类, 而不是 java.util.ArrayList
2 private static class ArrayList<E> extends AbstractList<E>
3     implements RandomAccess, java.io.Serializable
```

继承 AbstractList 中对 add、remove、set 方法是直接抛异常的, 也就是说如果继承的子类没有去重写这些方法, 那么子类的实例去调用这些方法是会直接抛异常的。

下面是AbstractList中方法的定义, 我们可以看到具体抛出的异常:

```
1 public void add(int index, E element) {
2     throw new UnsupportedOperationException();
3 }
4 public E remove(int index) {
5     throw new UnsupportedOperationException();
6 }
7 public E set(int index, E element) {
8     throw new UnsupportedOperationException();
9 }
```

虽然 set 方法也抛出了一场, 但是由于 内部类 ArrayList 重写了 set 方法, 所以支持其可以对元素进行修改。

```
    @Override
    public E set(int index, E element) {
        E oldValue = a[index];
        a[index] = element;
        return oldValue;
    }
```

- Arrays.asList 不支持基础类型的转换

Java 中的基础数据类型 (byte,short,int,long,float,double,boolean) 是不支持使用 Arrays.asList 方法去转换的

Collection 和 Collections 的区别

Collection 和 Collections 都是位于 `java.util` 包下的类

Collection 是集合类的父类, 它是一个顶级接口, 大部分抽象类比如说

`AbstractList`、`AbstractSet` 都继承了 Collection 类, Collection 类只定义一节标准方法比如说 add、remove、set、equals 等, 具体的方法由抽象类或者实现类去实现。

Collections 是集合类的工具类, Collections 提供了一些工具类的基本使用

- sort 方法, 对当前集合进行排序, 实现 Comparable 接口的类, 只能使用一种排序方案, 这种方案叫做自然比较
- 比如实现线程安全的容器 `Collections.synchronizedList`、`Collections.synchronizedMap` 等
- reverse 反转, 使用 reverse 方法可以根据元素的自然顺序 对指定列表按降序进行排序。
- fill, 使用指定元素替换指定列表中的所有元素。

有很多用法，读者可以翻阅 Collections 的源码查看，Collections 不能进行实例化，所以 Collections 中的方法都是由 `Collections.方法` 直接调用。

你知道 fail-fast 和 fail-safe 吗

`fail-fast` 是 Java 中的一种 `快速失败` 机制，`java.util` 包下所有的集合都是快速失败的，快速失败会抛出 `ConcurrentModificationException` 异常，fail-fast 你可以把它理解为一种快速检测机制，它只能用来检测错误，不会对错误进行恢复，fail-fast 不一定只在 `多线程` 环境下存在，`ArrayList` 也会抛出这个异常，主要原因是由于 `modCount` 不等于 `expectedModCount`。

`fail-safe` 是 Java 中的一种 `安全失败` 机制，它表示的是在遍历时不是直接在原集合上进行访问，而是先复制原有集合内容，在拷贝的集合上进行遍历。由于迭代时是对原集合的拷贝进行遍历，所以在遍历过程中对原集合所作的修改并不能被迭代器检测到，所以不会触发 `ConcurrentModificationException`。`java.util.concurrent` 包下的容器都是安全失败的，可以在多线程条件下使用，并发修改。

ArrayList、LinkedList 和 Vector 的区别

这也是一道老生常谈的问题了

`ArrayList`、`LinkedList`、`Vector` 都是位于 `java.util` 包下的工具类，它们都实现了 `List` 接口。

- `ArrayList` 的底层是动态数组，它是基于数组的特性而演变出来的，所以 `ArrayList` 遍历访问非常快，但是增删比较慢，因为会涉及到数组的拷贝。`ArrayList` 是一个非线程安全的容器，在并发场景下会造成问题，如果想使用线程安全的容器的话，推荐使用 `Collections.synchronizedList`；`ArrayList` 在扩容时会增加 50% 的容量。
- `LinkedList` 的底层是双向链表，所以 `LinkedList` 的增加和删除非常快，只需把元素删除，把各自的指针指向新的元素即可。但是 `LinkedList` 遍历比较慢，因为只有每次访问一个元素才能知道下一个元素的值。`LinkedList` 也是一个非线程安全的容器，推荐使用 `Collections.synchronizedList`
- `Vector` 向量是最早出现的集合容器，`Vector` 是一个线程安全的容器，它的每个方法都粗暴的加上了 `synchronized` 锁，所以它的增删、遍历效率都很低。`Vector` 在扩容时，它的容量会增加一倍。

Exception 和 Error 有什么区别

Exception 泛指的是 `异常`，Exception 主要分为两种异常，一种是编译期出现的异常，称为 `checkedException`，一种是程序运行期间出现的异常，称为 `uncheckedException`，常见的 `checkedException` 有 `IOException`，`uncheckedException` 统称为 `RuntimeException`，常见的 `RuntimeException` 主要有 `NullPointerException`、`IllegalArgumentException`、`ArrayIndexOutOfBoundsException` 等，Exception 可以被捕获。

Error 是指程序运行过程中出现的错误，通常情况下会造成程序的崩溃，Error 通常是不可恢复的，Error 不能被捕获。

详细可以参考这篇文章 [看完这篇 Exception 和 Error，和面试官扯皮就没问题了](#)

String、StringBuilder 和 StringBuffer 有什么区别

String 特指的是 Java 中的字符串，String 类位于 `java.lang` 包下，String 类是由 final 修饰的，String 字符串一旦创建就不能被修改，任何对 String 进行修改的操作都相当于重新创建了一个字符串。String 字符串的底层使用 StringBuilder 来实现的

StringBuilder 位于 `java.util` 包下，StringBuilder 是一非线程安全的容器，StringBuilder 的 append 方法常用于字符串拼接，它的拼接效率要比 String 中 `+` 号的拼接效率高。StringBuilder 一般不用于并发环境

StringBuffer 位于 `java.util` 包下，StringBuffer 是一个线程安全的容器，多线程场景下一般使用 StringBuffer 用作字符串的拼接

StringBuilder 和 StringBuffer 都是继承于 **AbstractStringBuilder** 类，AbstractStringBuilder 类实现了 StringBuffer 和 StringBuilder 的常规操作。

动态代理是基于什么原理

代理一般分为 **静态代理** 和 **动态代理**，它们都是代理模式的一种应用，静态代理指的是在程序运行前已经编译好，程序知道由谁来执行代理方法。

而动态代理只有在程序运行期间才能确定，相比于静态代理，动态代理的优势在于可以很方便的对代理类的函数进行统一的处理，而不用修改每个代理类中的方法。可以说动态代理是基于 **反射** 实现的。通过反射我们可以直接操作类或者对象，比如获取类的定义，获取声明的属性和方法，调用方法，在运行时可以修改类的定义。

动态代理是一种在运行时构建代理、动态处理方法调用的机制。动态代理的实现方式有很多，Java 提供的代理被称为 **JDK 动态代理**，JDK 动态代理是基于类的继承。

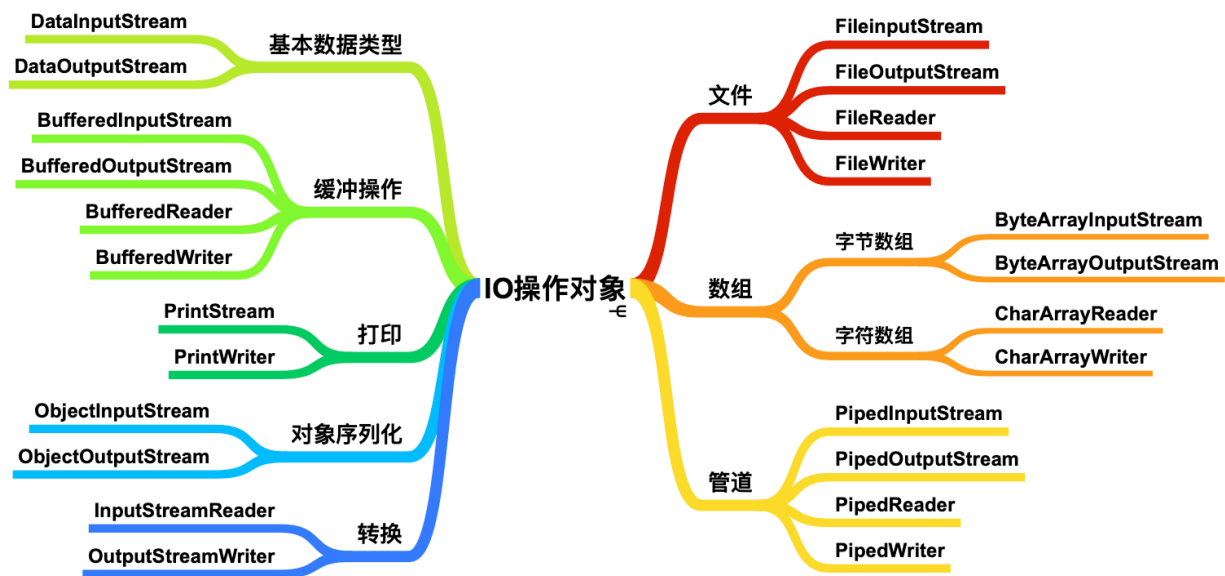
int 和 Integer 的区别

int 和 Integer 区别可就太多了

- int 是 Java 中的基本数据类型，int 代表的是 **整型**，一个 int 占 4 字节，也就是 32 位，int 的初始值是默认值是 0，int 在 Java 内存模型中被分配在栈中，int 没有方法。
- Integer 是 Java 中的基本数据类型的包装类，Integer 是一个对象，Integer 可以进行方法调用，Integer 的默认值是 null，Integer 在 Java 内存模型中被分配在堆中。int 和 Integer 在计算时可以进行相互转换，int -> Integer 的过程称为 **装箱**，Integer -> int 的过程称为 **拆箱**，Integer 还有 IntegerCache，会自动缓存 -128 - 127 中的值

Java 提供了哪些 I/O 方式

Java I/O 方式有很多种，传统的 I/O 也称为 **BIO**，主要流有如下几种



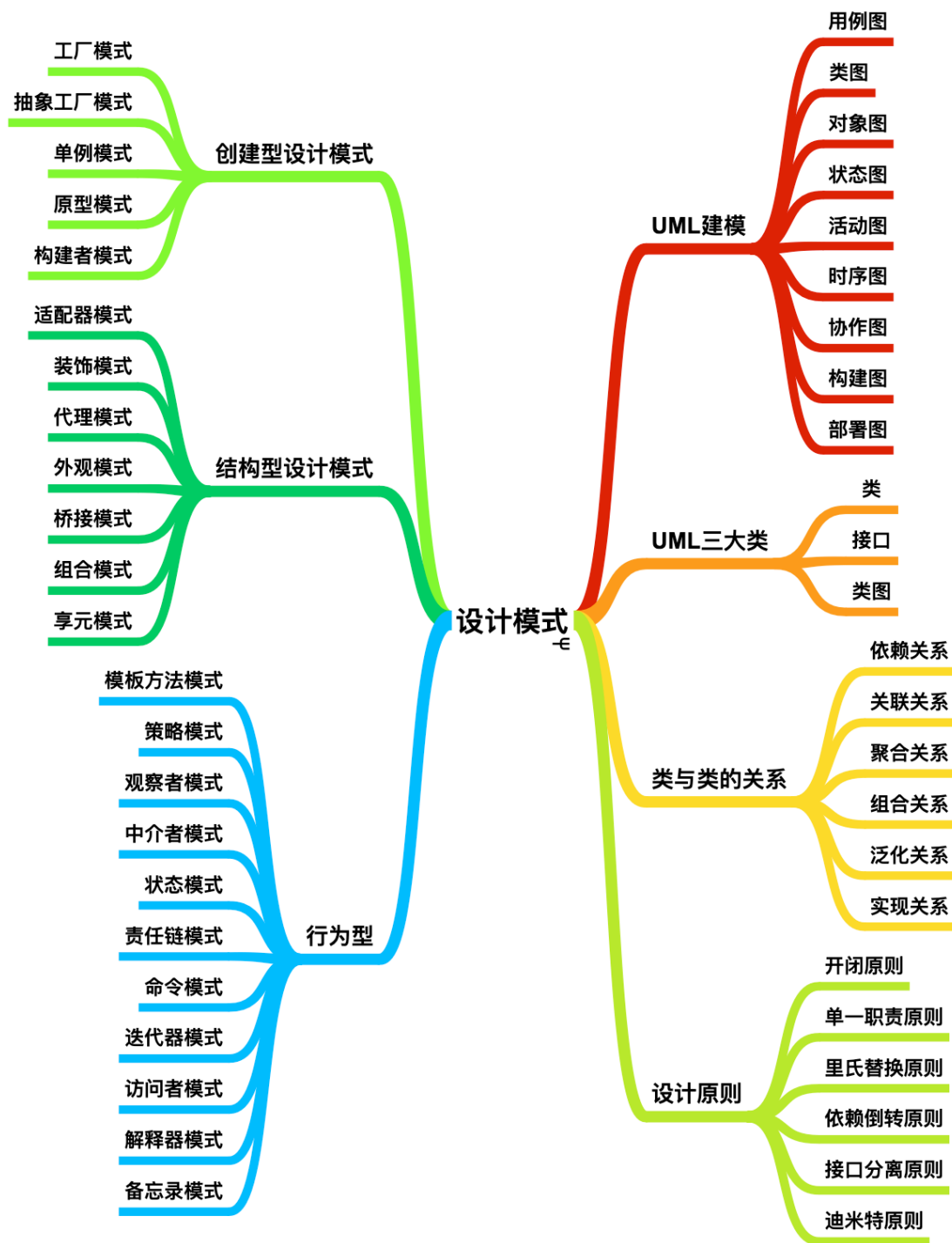
Java I/O 包的实现比较简单，但是容易出现性能瓶颈，传统的 I/O 是基于同步阻塞的。

JDK 1.4 之后提供了 **NIO**，也就是位于 `java.nio` 包下，提供了基于 **channel**、**Selector**、**Buffer** 的抽象，可以构建多路复用、同步非阻塞 I/O 程序。

JDK 1.7 之后对 NIO 进行了进一步改进，引入了 **异步非阻塞** 的方式，也被称为 **AIIO(Asynchronous IO)**。可以用生活中的例子来说明：项目经理交给手下员工去改一个 bug，那么项目经理不会一直等待员工解决 bug，他肯定在员工解决 bug 的期间给其他手下分配 bug 或者做其他事情，员工解决完 bug 之后再告诉项目经理 bug 解决完了。

谈谈你知道的设计模式

一张思维导图镇场



比如全局唯一性可以用 **单例模式**。

可以使用 **策略模式** 优化过多的 if...else...

制定标准用 **模版模式**

接手其他人的锅，但不想改原来的类用 **适配器模式**

使用 **组合** 而不是继承

使用 **装饰器** 可以制作加糖、加奶酪的咖啡

代理 可以用于任何中间商.....

Comparator 和 **Comparable** 有什么不同

- Comparable 更像是自然排序
- Comparator 更像是定制排序

同时存在时采用 **Comparator**（定制排序）的规则进行比较。

对于一些普通的数据类型（比如 String, Integer, Double...），它们默认实现了 Comparable 接口，实现了 compareTo 方法，我们可以直接使用。

而对于一些自定义类，它们可能在不同情况下需要实现不同的比较策略，我们可以新创建 Comparator 接口，然后使用特定的 Comparator 实现进行比较。

Object 类中一般都有哪些方法

Object 类是所有对象的父类，它里面包含一些所有对象都能够使用的方法

- hashCode(): 用于计算对象的哈希码
- equals(): 用于对象之间比较值是否相等
- toString(): 用于把对象转换成为字符串
- clone(): 用于对象之间的拷贝
- wait(): 用于实现对象之间的等待
- notify(): 用于通知对象释放资源
- notifyAll(): 用于通知所有对象释放资源
- finalize(): 用于告知垃圾回收器进行垃圾回收
- getClass(): 用于获得对象类

Java 泛型和类型擦除

关于 **Java 泛型和擦除**看一篇就够了。

反射的基本原理，反射创建类实例的三种方式是什么

反射机制就是使 Java 程序在运行时具有 **自省(introspect)** 的能力，通过反射我们可以直接操作类和对象，比如获取某个类的定义，获取类的属性和方法，构造方法等。

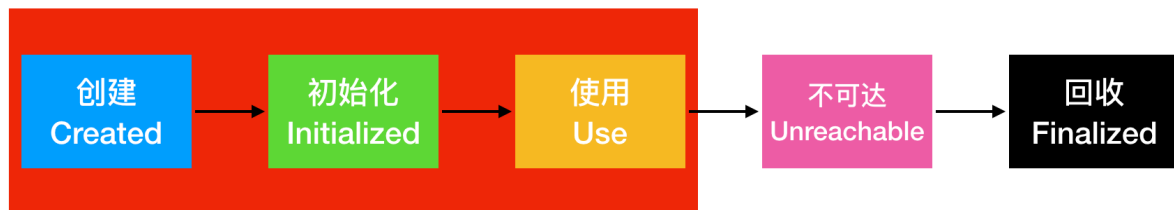
创建类实例的三种方式是

- 对象实例.getClass();
- 通过 Class.forName() 创建
- 对象实例.newInstance() 方法创建

强引用、弱引用、虚引用和幻象引用的区别

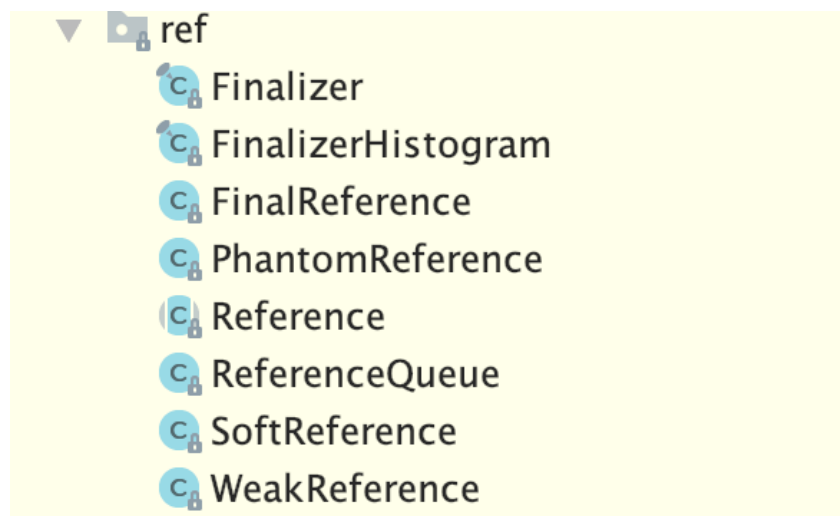
我们说的不同的引用类型其实都是逻辑上的，而对于虚拟机来说，主要体现的是对象的不同的 **可达性(reachable)** 状态和对 **垃圾收集(garbage collector)** 的影响。

可以通过下面的流程来对对象的生命周期做一个总结



对象被创建并初始化，对象在运行时被使用，然后离开对象的作用域，对象会变成不可达并会被垃圾收集器回收。图中用红色标明的区域表示对象处于强可达阶段。

JDK1.2 介绍了 `java.lang.ref` 包，对象的生命周期有四个阶段：强可达(Strongly Reachable)、软可达(Soft Reachable)、弱可达(Weak Reachable)、幻象可达(Phantom Reachable)。



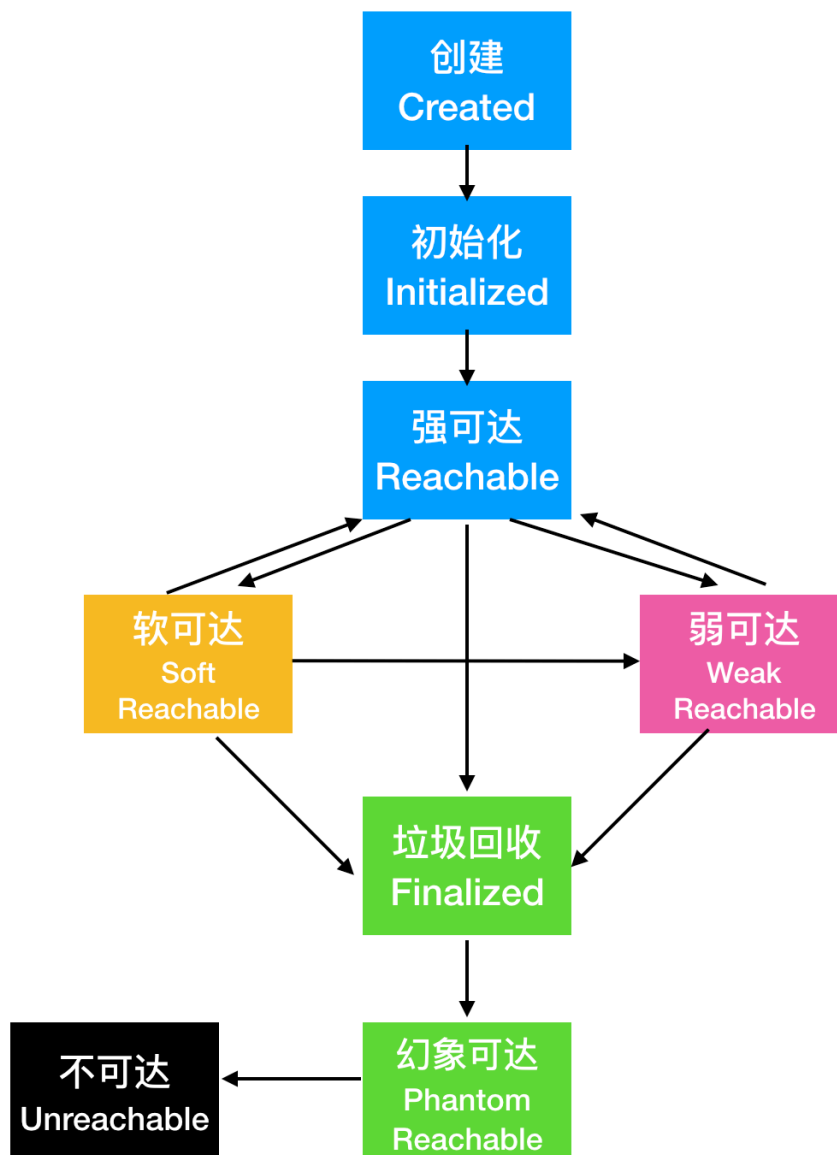
如果只讨论符合垃圾回收条件的对象，那么只有三种：软可达、弱可达和幻象可达。

- 软可达：软可达就是我们只能通过软引用才能访问的状态，软可达的对象是由 `SoftReference` 引用的对象，并且没有强引用的对象。软引用是用来描述一些还有用但是非必须的对象。垃圾收集器会尽可能长时间的保留软引用的对象，但是会在发生 `OutOfMemoryError` 之前，回收软引用的对象。如果回收完软引用的对象，内存还是不够分配的话，就会直接抛出 `OutOfMemoryError`。
- 弱可达：弱可达的对象是 `WeakReference` 引用的对象。垃圾收集器可以随时收集弱引用的对象，不会尝试保留软引用的对象。
- 幻象可达：幻象可达是由 `PhantomReference` 引用的对象，幻象可达就是没有强、软、弱引用进行关联，并且已经被 `finalize` 过了，只有幻象引用指向这个对象的时候。

除此之外，还有强可达和不可达的两种可达性判断条件

- 强可达：就是一个对象刚被创建、初始化、使用中的对象都是处于强可达的状态
- 不可达(unreachable)：处于不可达的对象就意味着对象可以被清除了。

下面是一个不同可达性状态的转换图



判断可达性条件，也是 JVM 垃圾收集器决定如何处理对象的一部分考虑因素。

所有的对象可达性引用都是 `java.lang.ref.Reference` 的子类，它里面有一个 `get()` 方法，返回引用对象。如果已通过程序或垃圾收集器清除了此引用对象，则此方法返回 `null`。也就是说，除了幻象引用外，软引用和弱引用都是可以得到对象的。而且这些对象可以人为 **拯救**，变为强引用，例如把 `this` 关键字赋值给对象，只要重新和引用链上的任意一个对象建立关联即可。

final、finally 和 finalize() 的区别

这三者可以说是没有任何关联之处，我们上面谈到了，`final` 可以用来修饰类、变量和方法，可以参考上面 `final` 的那道面试题。

`finally` 是一个关键字，它经常和 `try` 块一起使用，用于异常处理。使用 `try...finally` 的代码块种，`finally` 部分的代码一定会被执行，所以我们经常在 `finally` 方法中用于资源的关闭操作。

JDK1.7 中，推荐使用 `try-with-resources` 优雅的关闭资源，它直接使用 `try{}{}` 进行资源的关闭即可，就不用写 `finally` 关键字了。

`finalize` 是 `Object` 对象中的一个方法，用于对象的回收方法，这个方法我们一般不推荐使用，`finalize` 是和垃圾回收关联在一起的，在 Java 9 中，将 `finalize` 标记为了 `deprecated`，如果没有特别原因，不要实现 `finalize` 方法，也不要指望他来进行垃圾回收。

内部类有哪些分类，分别解释一下

在 Java 中，可以将一个类的定义放在另外一个类的定义内部，这就是**内部类**。内部类本身就是类的一个属性，与其他属性定义方式一致。

内部类的分类一般主要有四种

- 成员内部类
- 局部内部类
- 匿名内部类
- 静态内部类

静态内部类 就是定义在类内部的静态类，静态内部类可以访问外部类所有的静态变量，而不可访问外部类的非静态变量；

成员内部类 就是定义在类内部，成员位置上的非静态类，就是成员内部类。成员内部类可以访问外部类所有的变量和方法，包括静态和非静态，私有和公有。

定义在方法中的内部类，就是 **局部内部类**。定义在实例方法中的局部类可以访问外部类的所有变量和方法，定义在静态方法中的局部类只能访问外部类的静态变量和方法。

匿名内部类 就是没有名字的内部类，除了没有名字，匿名内部类还有以下特点：

- 匿名内部类必须继承一个抽象类或者实现一个接口
- 匿名内部类不能定义任何静态成员和静态方法。
- 当所在的方法的形参需要被匿名内部类使用时，必须声明为 final。
- 匿名内部类不能是抽象的，它必须要实现继承的类或者实现的接口的所有抽象方法。

说出几种常用的异常

- NullPointerException: 空指针异常
- NoSuchMethodException: 找不到方法
- IllegalArgumentException: 不合法的参数异常
- IndexOutOfBoundsException: 数组下标越界异常
- IOException: 由于文件未找到、未打开或者I/O操作不能进行而引起异常
- ClassNotFoundException: 找不到文件所抛出的异常
- NumberFormatException: 字符的UTF代码数据格式有错引起异常；
- InterruptedException: 线程中断抛出的异常

静态绑定和动态绑定的区别

一个Java 程序要经过编写、编译、运行三个步骤，其中编写代码不在我们讨论的范围之内，那么我们的重点自然就放在了 **编译** 和 **运行** 这两个阶段，由于编译和运行阶段过程相当繁琐，下面就我的理解来进行解释：

Java 程序从源文件创建到程序运行要经过两大步骤：

- 1、编译时期是由编译器将源文件编译成字节码的过程
- 2、字节码文件由Java虚拟机解释执行

绑定

绑定就是一个方法的调用与调用这个方法的类连接在一起的过程被称为绑定。

绑定主要分为两种：

静态绑定 和 动态绑定

绑定的其他叫法

静态绑定 == 前期绑定 == 编译时绑定

动态绑定 == 后期绑定 == 运行时绑定

为了方便区分：下面统一称呼为静态绑定和动态绑定

静态绑定

在程序运行前，也就是编译时期 **JVM** 就能够确定方法由谁调用，这种机制称为静态绑定

识别静态绑定的三个关键字以及各自的理解

如果一个方法由 `private`、`static`、`final` 任意一个关键字所修饰，那么这个方法为前期绑定的

构造方法也是前期绑定

`private`： `private` 关键字是私有的意思，如果被 `private` 修饰的方法是无法由本类之外的其他类所调用的，也就是本类所特有的方法，所以也就由编译器识别此方法是属于哪个类的

```
1  public class Person {
2
3      private String talk;
4
5      private String canTalk(){
6          return talk;
7      }
8  }
9
10 class Animal{
11
12     public static void main(String[] args) {
13         Person p = new Person();
14         // private 修饰的方法是Person类独有的，所以Animal类无法访问(动物本来就不能说话)
15         //      p.canTalk();
16     }
17 }
```

`final`： `final` 修饰的方法不能被重写，但是可以由子类进行调用，如果将方法声明为 `final` 可以有效的关闭动态绑定

```
1  public class Fruit {
2
3      private String fruitName;
4
5      final String eatingFruit(String name){
6          System.out.println("eating " + name);
7          return fruitName;
8      }
9  }
```

```

8     }
9 }
10
11 class Apple extends Fruit{
12
13     // 不能重写final方法, eatingFruit方法只属于Fruit类, Apple类无法调用
14     // String eatingFruit(String name){
15     //     super.eatingFruit(name);
16     // }
17
18     String eatingApple(String name){
19         return super.eatingFruit(name);
20     }
21 }

```

static: static 修饰的方法比较特殊, 不用通过 new 出某个类来调用, 由 类名.变量名 直接调用该方法, 这个就很关键了, new 很关键, 也可以认为是开启多态的导火索, 而由类名.变量名直接调用的话, 此时的类名是确定的, 并不会产生多态, 如下代码:

```

1  public class SuperClass {
2
3      public static void sayHello(){
4
5          System.out.println("由 superClass 说你好");
6      }
7  }
8
9  public class SubClass extends SuperClass{
10
11      public static void sayHello(){
12          System.out.println("由 SubClass 说你好");
13      }
14
15      public static void main(String[] args) {
16          SuperClass.sayHello();
17          SubClass.sayHello();
18      }
19  }

```

Generate

Constructor

SubClass 继承 SuperClass 后, 在

toString()

是无法重写

Override Methods...

Copyright

sayHello 方法的, 也就是说 sayHello() 方法是对子类隐藏的, 但是你可以编写自己的 sayHello() 方法, 也就是子类 SubClass 的sayHello() 方法, 由此可见, 方法由 static 关键词所修饰, 也是编译时绑定

动态绑定

在运行时根据具体对象的类型进行绑定

除了由 **private**、**final**、**static** 所修饰的方法和构造方法外，**JVM** 在运行期间决定方法由哪个对象调用的过程称为动态绑定

如果把编译、运行看成一条时间线的话，在运行前必须要进行程序的编译过程，那么在编译期进行的绑定是前期绑定，在程序运行了，发生的绑定就是后期绑定

```
1  public class Father {
2
3      void drinkMilk(){
4          System.out.println("父亲喜欢喝牛奶");
5      }
6  }
7
8  public class Son extends Father{
9
10     @Override
11     void drinkMilk() {
12         System.out.println("儿子喜欢喝牛奶");
13     }
14
15     public static void main(String[] args) {
16         Father son = new Son();
17         son.drinkMilk();
18     }
19 }
```

Son 类继承 Father 类，并重写了父类的 drinkMilk() 方法，在输出结果得出的是儿子喜欢喝牛奶。那么上面的绑定方式是什么呢？

上面的绑定方式称之为 **动态绑定**，因为在你编写 Father son = new Son() 的时候，编译器并不知道 son 对象真正引用的是谁，在程序运行时期才知道，这个 son 是一个 Father 类的对象，但是却指向了 Son 的引用，这种概念称之为多态，那么我们就能整理出来多态的三个原则：

- 继承
- 重写
- 父类引用指向子类对象

也就是说，在 Father son = new Son()，触发了动态绑定机制。

动态绑定的过程

1. 虚拟机提取对象的实际类型的方法表；
2. 虚拟机搜索方法签名；
3. 调用方法。

动态绑定和静态绑定的特点

静态绑定

静态绑定在编译时期触发，那么它的主要特点是

1、编译期触发，能够提早知道代码错误

2、提高程序运行效率

动态绑定

1、使用动态绑定的前提条件能够提高代码的可用性，使代码更加灵活。

2、多态是设计模式的基础，能够降低耦合性。

HTTP 篇

HTTP 和 HTTPS 的区别

HTTP 是一种 **超文本传输协议(Hypertext Transfer Protocol)**，**HTTP** 是一个在计算机世界里专门在两点之间传输文字、图片、音频、视频等超文本数据的约定和规范

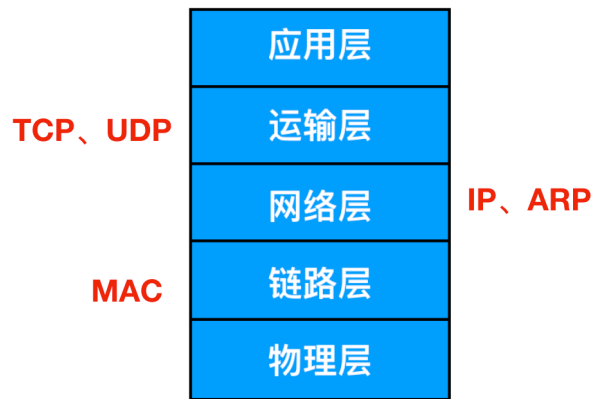


HTTP 主要内容分为三部分，**超文本 (Hypertext)**、**传输 (Transfer)**、**协议 (Protocol)**。

- 超文本就是不单单只是本文，它还可以传输图片、音频、视频，甚至点击文字或图片能够进行 **超链接** 的跳转。
- 上面这些概念可以统称为数据，传输就是数据需要经过一系列的物理介质从一个端系统传送到另外一个端系统的过程。通常我们把传输数据包的一方称为 **请求方**，把接到二进制数据包的一方称为 **应答方**。
- 而协议指的就是是网络中(包括互联网)传递、管理信息的一些规范。如同人与人之间相互交流是需要遵循一定的规矩一样，计算机之间的相互通信需要共同遵守一定的规则，这些规则就称为协议，只不过是网络协议。

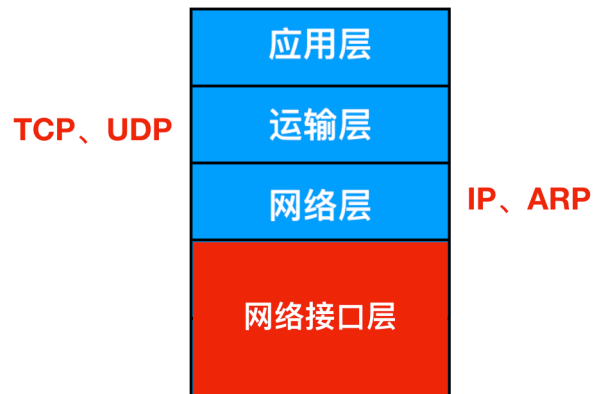
说到 HTTP，不得不提的就是 TCP/IP 网络模型，一般是五层模型。如下图所示

HTTP 、SMTP、FTP、Telnet、DNS



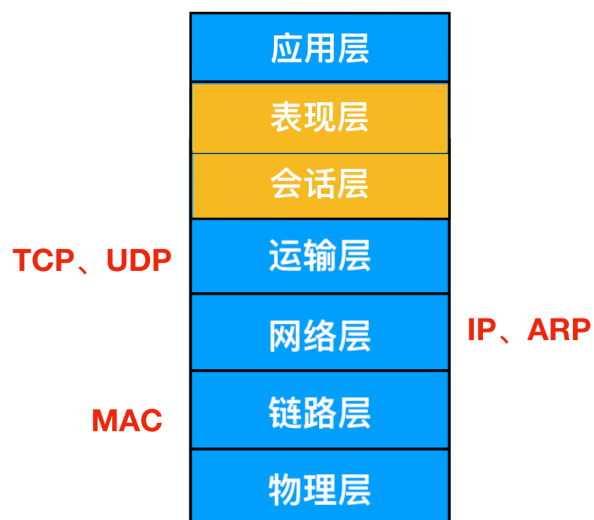
但是也可以分为四层，就是把链路层和物理层都表示为网络接口层

HTTP 、SMTP、FTP、Telnet、DNS



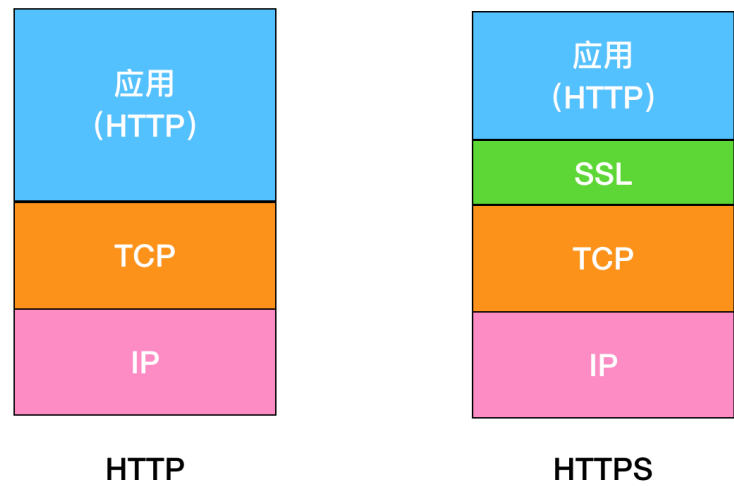
还有一种就是 OSI 七层网络模型，它就是在五层协议之上加了表示层和会话层

HTTP 、SMTP、FTP、Telnet、DNS



而 HTTPS 的全称是 **Hypertext Transfer Protocol Secure**，从名称我们可以看出 HTTPS 要比 HTTP 多了 secure 安全性这个概念，实际上，HTTPS 并不是一个新的应用层协议，它其实就是 HTTP + TLS/SSL 协议组合而成，而安全性的保证正是 TLS/SSL 所做的工作。

也就是说，**HTTPS 就是身披了一层 SSL 的 HTTP。**



那么，HTTP 和 HTTPS 的主要区别是什么呢？

- 最简单的，HTTP 在地址栏上的协议是以 `http://` 开头，而 HTTPS 在地址栏上的协议是以 `https://` 开头
- ```
1 http://www.cxuanblog.com/
2 https://www.cxuanblog.com/
```
- HTTP 是未经安全加密的协议，它的传输过程容易被攻击者监听、数据容易被窃取、发送方和接收方容易被伪造；而 HTTPS 是安全的协议，它通过 **密钥交换算法 - 签名算法 - 对称加密算法 - 摘要算法** 能够解决上面这些问题。



- HTTP 的默认端口是 80，而 HTTPS 的默认端口是 443。

## HTTP Get 和 Post 区别

HTTP 中包括许多方法，**Get 和 Post 是 HTTP 中最常用的两个方法**，基本上使用 HTTP 方法中有 99% 都是在使用 Get 方法和 Post 方法，所以有必要我们对这两个方法有更加深刻的认识。

- get 方法一般用于请求，比如你在浏览器地址栏输入 `www.cxuanblog.com` 其实就是发送了一个 get 请求，它的主要特征是请求服务器返回资源，而 post 方法一般用于 `<form>` 表单 的提交，相当于是把信息提交给服务器，等待服务器作出响应，get 相当于一个是 pull/拉的操作，而 post 相当于是一个 push/推的操作。
- get 方法是不安全的，因为你在发送请求的过程中，你的请求参数会拼在 URL 后面，从而导致容易被攻击者窃取，对你的信息造成破坏和伪造；

```
1 /test/demo_form.asp?name1=value1&name2=value2
```

而 post 方法是把参数放在请求体 body 中的，这对用户来说不可见。

```
1 POST /test/demo_form.asp HTTP/1.1
2 Host: w3schools.com
3 name1=value1&name2=value2
```

- get 请求的 URL 有长度限制，而 post 请求会把参数和值放在消息体中，对数据长度没有要求。
- get 请求会被浏览器主动 cache，而 post 不会，除非手动设置。
- get 请求在浏览器反复的 回退/前进 操作是无害的，而 post 操作会再次提交表单请求。
- get 请求在发送过程中会产生一个 TCP 数据包；post 在发送过程中会产生两个 TCP 数据包。对于 get 方式的请求，浏览器会把 http header 和 data 一并发送出去，服务器响应 200（返回数据）；而对于 post，浏览器先发送 header，服务器响应 100 continue，浏览器再发送 data，服务器响应 200 ok（返回数据）。

## 什么是无状态协议，HTTP 是无状态协议吗，怎么解决

**无状态协议(Stateless Protocol)** 就是指浏览器对于事务的处理没有记忆能力。举个例子来说就是比如客户请求获得网页之后关闭浏览器，然后再次启动浏览器，登录该网站，但是服务器并不知道客户关闭了一次浏览器。

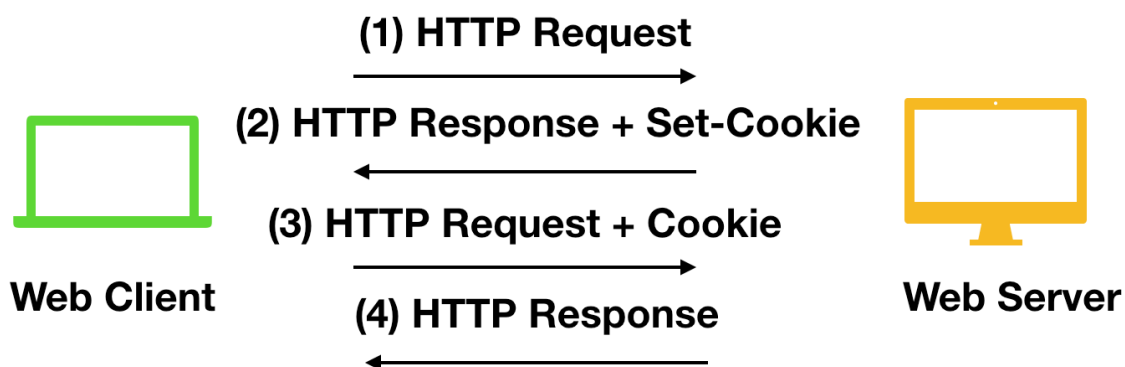
HTTP 就是一种无状态的协议，他对用户的操作没有记忆能力。可能大多数用户不相信，他可能觉得每次输入用户名和密码登陆一个网站后，下次登陆就不再重新输入用户名和密码了。这其实不是 HTTP 做的事情，起作用的是一个叫做 **小甜饼(Cookie)** 的机制。它能够让浏览器具有 **记忆** 能力。

如果你的浏览器允许 cookie 的话，查看方式 `chrome://settings/content/cookies`

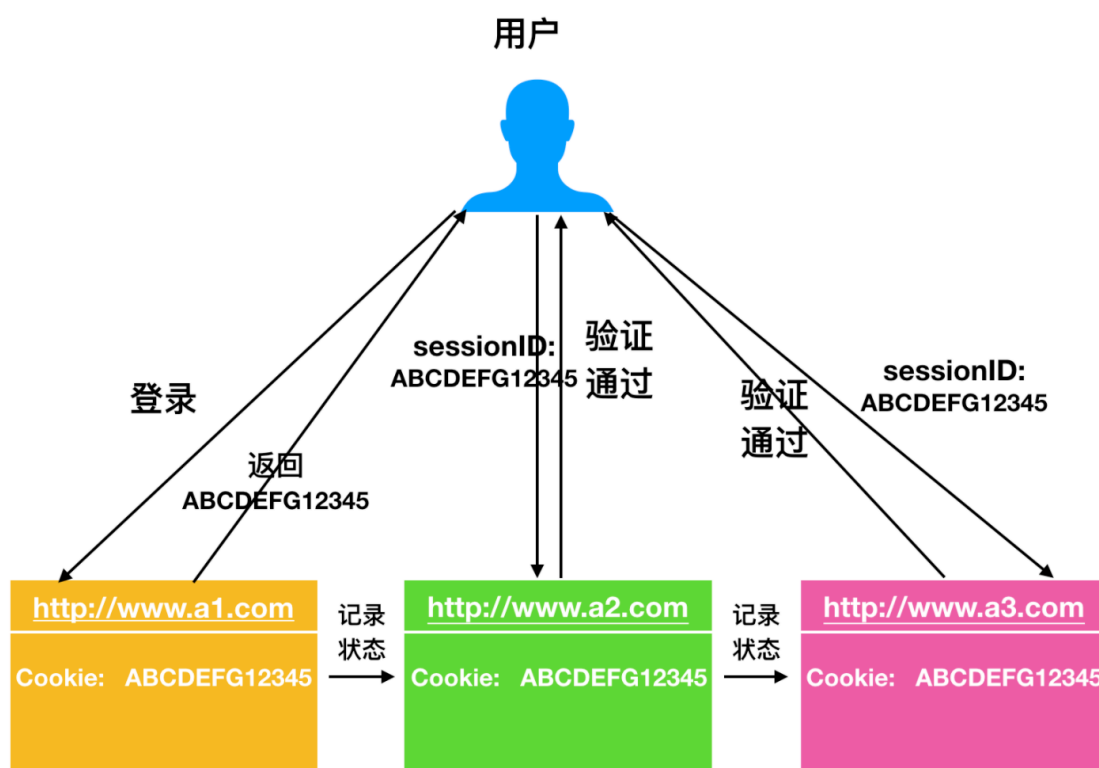


也就说明你的记忆芯片通电了..... 当你想服务端发送请求时，服务端会给你发送一个认证信息，服务器第一次接收到请求时，开辟了一块 Session 空间（创建了Session对象），同时生成一个 sessionId，并通过响应头的 **Set-Cookie: JSESSIONID=XXXXXXX** 命令，向客户端发送要求设置 Cookie 的响应；客户端收到响应后，在本机客户端设置了一个 **JSESSIONID=XXXXXXX** 的 Cookie 信息，该 Cookie 的过期时间为浏览器会话结束；





接下来客户端每次向同一个网站发送请求时，请求头都会带上该 Cookie 信息（包含 sessionId），然后，服务器通过读取请求头中的 Cookie 信息，获取名称为 JSESSIONID 的值，得到此次请求的 sessionId。这样，你的浏览器才具有了记忆能力。



还有一种方式是使用 JWT 机制，它也是能够让你的浏览器具有记忆能力的一种机制。与 Cookie 不同，JWT 是保存在客户端的信息，它广泛的应用于单点登录的情况。JWT 具有两个特点

- JWT 的 Cookie 信息存储在 **客户端**，而不是服务端内存中。也就是说，JWT 直接本地进行验证就可以，验证完毕后，这个 Token 就会在 Session 中随请求一起发送到服务器，通过这种方式，可以节省服务器资源，并且 token 可以进行多次验证。
- JWT 支持跨域认证，Cookies 只能用在 **单个节点的域** 或者它的 **子域** 中有效。如果它们尝试通过第三个节点访问，就会被禁止。使用 JWT 可以解决这个问题，使用 JWT 能够通过 **多个节点** 进行用户认证，也就是我们常说的 **跨域认证**。

## UDP 和 TCP 的区别

TCP 和 UDP 都位于计算机网络模型中的运输层，它们负责传输应用层产生的数据。下面我们就来聊一聊 TCP 和 UDP 分别的特征和他们的区别

## UDP 是什么

UDP 的全称是 **User Datagram Protocol**，用户数据报协议。它不需要所谓的 **握手** 操作，从而加快了通信速度，允许网络上的其他主机在接收方同意通信之前进行数据传输。

数据报是与分组交换网络关联的传输单元。

UDP 的特点主要有

- UDP 能够支持容忍数据包丢失的带宽密集型应用程序
- UDP 具有低延迟的特点
- UDP 能够发送大量的数据包
- UDP 能够允许 DNS 查找，DNS 是建立在 UDP 之上的应用层协议。

## TCP 是什么

TCP 的全称是 **Transmission Control Protocol**，传输控制协议。它能够帮助你确定计算机连接到 Internet 以及它们之间的数据传输。通过三次握手来建立 TCP 连接，三次握手就是用来启动和确认 TCP 连接的过程。一旦连接建立后，就可以发送数据了，当数据传输完成后，会通过关闭虚拟电路来断开连接。

TCP 的主要特点有

- TCP 能够确保连接的建立和数据包的发送
- TCP 支持错误重传机制
- TCP 支持拥塞控制，能够在网络拥堵的情况下延迟发送
- TCP 能够提供错误校验和，甄别有害的数据包。

## TCP 和 UDP 的不同



下面为你罗列了一些 TCP 和 UDP 的不同点，方便理解，方便记忆。

| TCP                                  | UDP                      |
|--------------------------------------|--------------------------|
| TCP 是面向连接的协议                         | UDP 是无连接的协议              |
| TCP 在发送数据前先需要建立连接，然后再发送数据            | UDP 无需建立连接就可以直接发送大量数据    |
| TCP 会按照特定顺序重新排列数据包                   | UDP 数据包没有固定顺序，所有数据包都相互独立 |
| TCP 传输的速度比较慢                         | UDP 的传输会更快               |
| TCP 的头部字节有 20 字节                     | UDP 的头部字节只需要 8 个字节       |
| TCP 是重量级的，在发送任何用户数据之前，TCP需要三次握手建立连接。 | UDP 是轻量级的。没有跟踪连接，消息排序等。  |
| TCP 会进行错误校验，并能够进行错误恢复                | UDP 也会错误检查，但会丢弃错误的数据包。   |
| TCP 有发送确认                            | UDP 没有发送确认               |
| TCP 会使用握手协议，例如 SYN，SYN-ACK，ACK       | 无握手协议                    |
| TCP 是可靠的，因为它可以确保将数据传送到路由器。           | 在 UDP 中不能保证将数据传送到目标。     |

## TCP 三次握手和四次挥手

TCP 三次握手和四次挥手也是面试题的热门考点，它们分别对应 TCP 的连接和释放过程。下面就来简单认识一下这两个过程

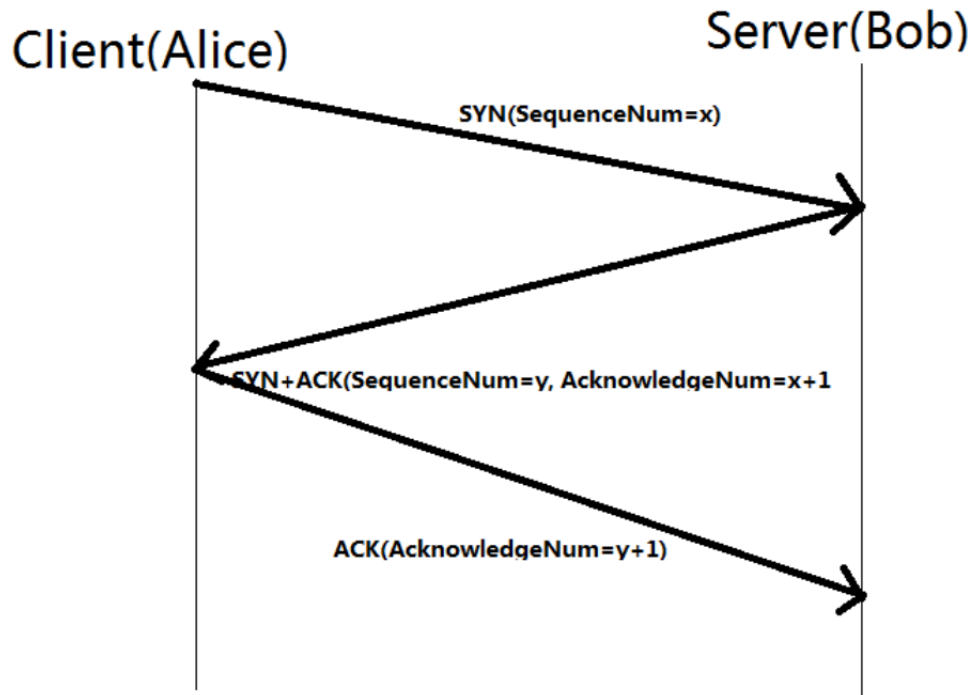
### TCP 三次握手

在了解具体的流程前，我们需要先认识几个概念

| 消息类型    | 描述                     |
|---------|------------------------|
| SYN     | 这个消息是用来初始化和建立连接的。      |
| ACK     | 帮助对方确认收到的 SYN 消息       |
| SYN-ACK | 本地的 SYN 消息和较早的 ACK 数据包 |
| FIN     | 用来断开连接                 |

- SYN：它的全称是 **Synchronize Sequence Numbers**，同步序列编号。是 TCP/IP 建立连接时使用的握手信号。在客户机和服务器之间建立 TCP 连接时，首先会发送的一个信号。客户端在接收到 SYN 消息时，就会在自己的段内生成一个随机值 X。
- SYN-ACK：服务器收到 SYN 后，打开客户端连接，发送一个 SYN-ACK 作为答复。确认号设置为比接收到的序列号多一个，即  $X + 1$ ，服务器为数据包选择的序列号是另一个随机数 Y。
- ACK：**Acknowledge character**，确认字符，表示发来的数据已确认接收无误。最后，客户端

将 ACK 发送给服务器。序列号被设置为所接收的确认值即  $Y + 1$ 。



如果用现实生活来举例的话就是

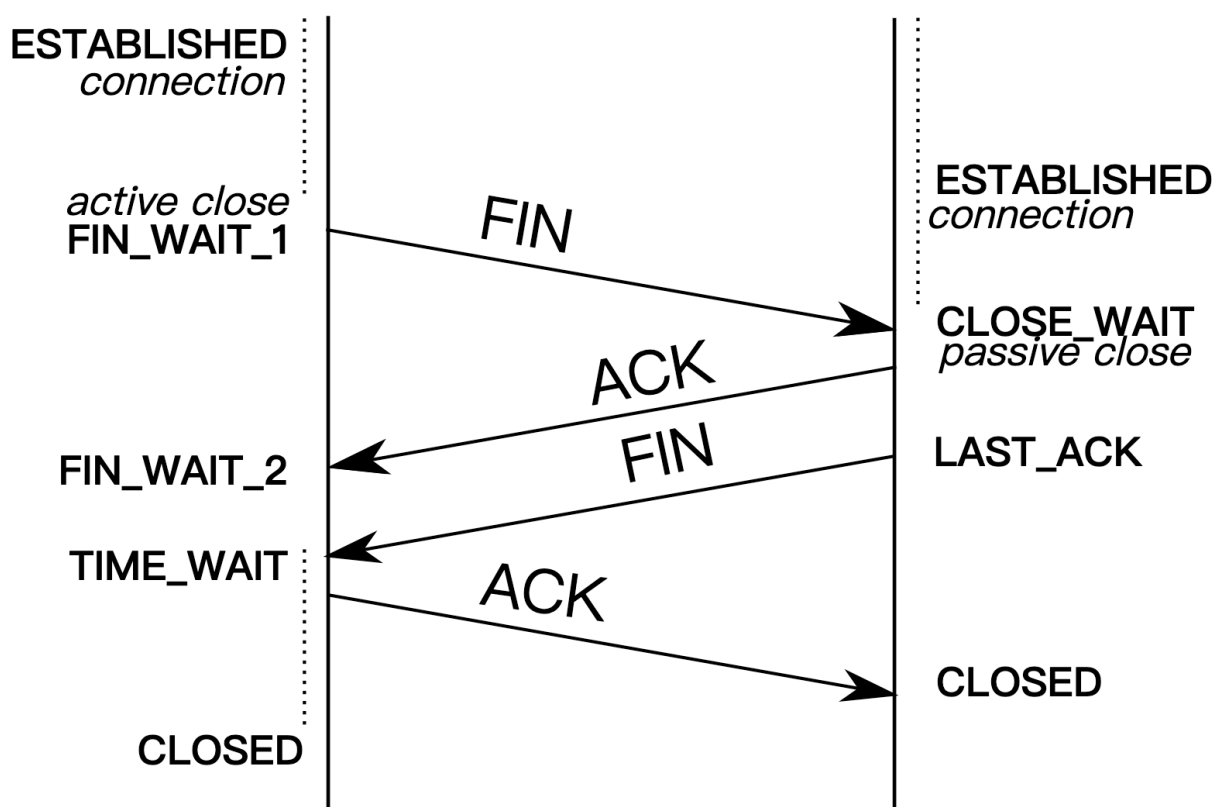
小明 - 客户端 小红 - 服务端

- 小明给小红打电话，接通了后，小明说喂，能听到吗，这就相当于是连接建立。
- 小红给小明回应，能听到，你能听到我说的话吗，这就相当于是请求响应。
- 小明听到小红的回应后，好的，这相当于是连接确认。在这之后小明和小红就可以通话/交换信息了。

## TCP 四次挥手

在连接终止阶段使用四次挥手，连接的每一端都会独立的终止。下面我们来描述一下这个过程。

# Initiator                  Receiver



- 首先，客户端应用程序决定要终止连接(这里服务端也可以选择断开连接)。这会使客户端将 FIN 发送到服务器，并进入 **FIN\_WAIT\_1** 状态。当客户端处于 FIN\_WAIT\_1 状态时，它会等待来自服务器的 ACK 响应。
- 然后第二步，当服务器收到 FIN 消息时，服务器会立刻向客户端发送 ACK 确认消息。
- 当客户端收到服务器发送的 ACK 响应后，客户端就进入 **FIN\_WAIT\_2** 状态，然后等待来自服务器的 **FIN** 消息
- 服务器发送 ACK 确认消息后，一段时间（可以进行关闭后）会发送 FIN 消息给客户端，告知客户端可以进行关闭。
- 当客户端收到从服务端发送的 FIN 消息时，客户端就会由 FIN\_WAIT\_2 状态变为 **TIME\_WAIT** 状态。处于 TIME\_WAIT 状态的客户端允许重新发送 ACK 到服务器为了防止信息丢失。客户端在 TIME\_WAIT 状态下花费的时间取决于它的实现，在等待一段时间后，连接关闭，客户端上所有的资源（包括端口号和缓冲区数据）都被释放。

还是可以用上面那个通话的例子来进行描述

- 小明对小红说，我所有的东西都说完了，我要挂电话了。
- 小红说，收到，我这边还有一些东西没说。
- 经过若干秒后，小红也说完了，小红说，我说完了，现在可以挂断了
- 小明收到消息后，又等了若干时间后，挂断了电话。

## 简述 HTTP1.0/1.1/2.0 的区别

### HTTP 1.0

HTTP 1.0 是在 1996 年引入的，从那时开始，它的普及率就达到了惊人的效果。

- HTTP 1.0 仅仅提供了最基本的认证，这时候用户名和密码还未经加密，因此很容易收到窥探。
- HTTP 1.0 被设计用来使用短链接，即每次发送数据都会经过 TCP 的三次握手和四次挥手，效率比较低。
- HTTP 1.0 只使用 header 中的 If-Modified-Since 和 Expires 作为缓存失效的标准。
- HTTP 1.0 不支持断点续传，也就是说，每次都会传送全部的页面和数据。
- HTTP 1.0 认为每台计算机只能绑定一个 IP，所以请求消息中的 URL 并没有传递主机名 (hostname) 。

## HTTP 1.1

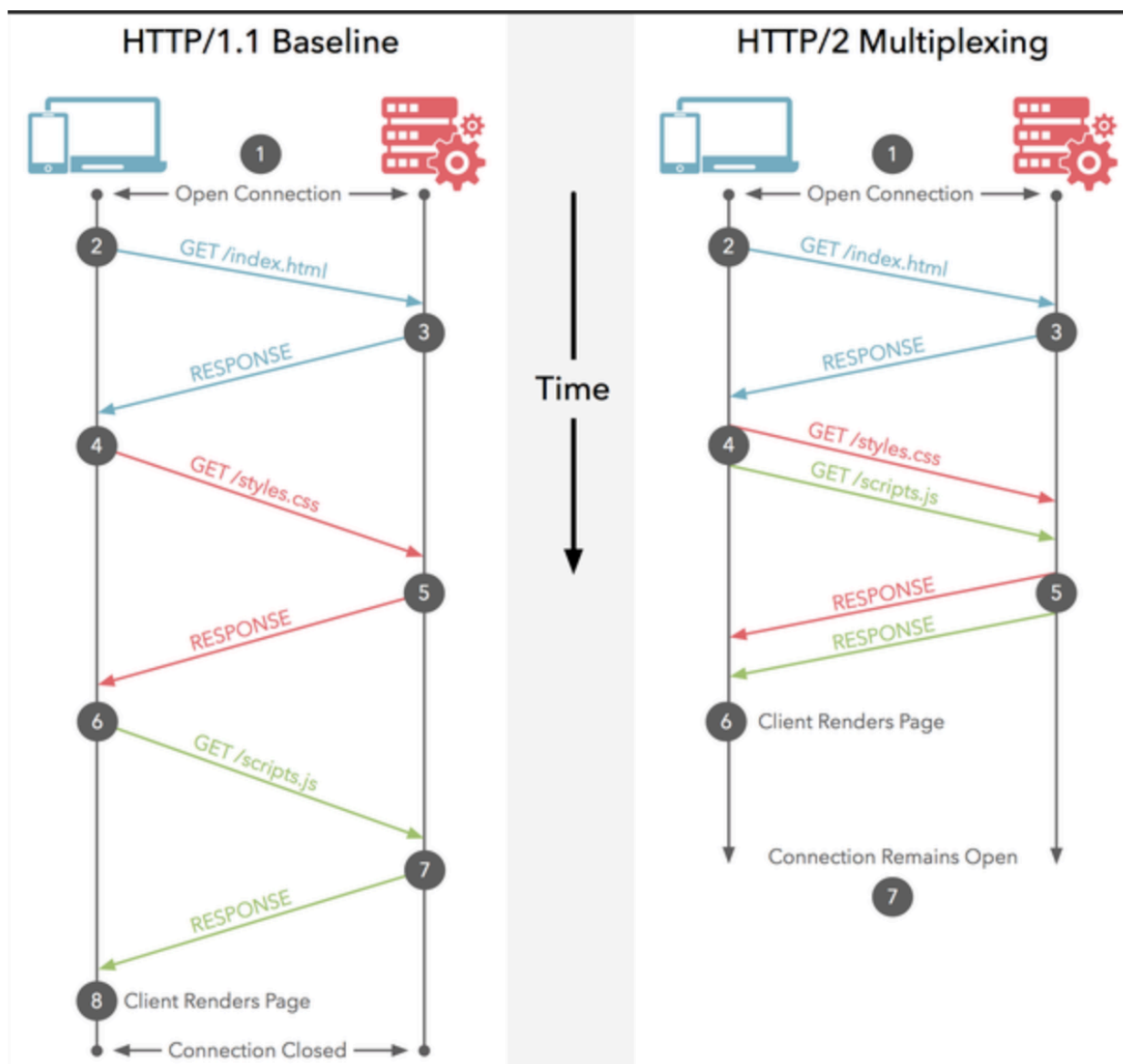
HTTP 1.1 是 HTTP 1.0 开发三年后出现的，也就是 1999 年，它做出了以下方面的变化

- HTTP 1.1 使用了摘要算法来进行身份验证
- HTTP 1.1 默认使用长连接，长连接就是只需一次建立就可以传输多次数据，传输完成后，只需要一次切断连接即可。长连接的连接时长可以通过请求头中的 `keep-alive` 来设置
- HTTP 1.1 中新增加了 E-tag, If-Unmodified-Since, If-Match, If-None-Match 等缓存控制标头来控制缓存失效。
- HTTP 1.1 支持断点续传，通过使用请求头中的 `Range` 来实现。
- HTTP 1.1 使用了虚拟网络，在一台物理服务器上可以存在多个虚拟主机 (Multi-homed Web Servers)，并且它们共享一个 IP 地址。

## HTTP 2.0

HTTP 2.0 是 2015 年开发出来的标准，它主要做的改变如下

- **头部压缩**，由于 HTTP 1.1 经常会出现 `User-Agent`、`Cookie`、`Accept`、`Server`、`Range` 等字段可能会占用几百甚至几千字节，而 Body 却经常只有几十字节，所以导致头部偏重。HTTP 2.0 使用 `HPACK` 算法进行压缩。
- **二进制格式**，HTTP 2.0 使用了更加靠近 TCP/IP 的二进制格式，而抛弃了 ASCII 码，提升了解析效率
- **强化安全**，由于安全已经成为重中之重，所以 HTTP2.0 一般都跑在 HTTPS 上。
- **多路复用**，即每一个请求都是是用作连接共享。一个请求对应一个 id，这样一个连接上可以有多个请求。



## 请你说一下 HTTP 常见的请求头

这个问题比较开放，因为 HTTP 请求头有很多，这里只简单举出几个例子，具体的可以参考我的另一篇文章

<https://mp.weixin.qq.com/s/XZZR0945lcI6X4S0g5fZXg>

HTTP 标头会分为四种，分别是 通用标头、实体标头、请求标头、响应标头。分别介绍一下

### 通用标头

通用标头主要有三个，分别是 Date、Cache-Control 和 Connection

#### Date

Date 是一个通用标头，它可以出现在请求标头和响应标头中，它的基本表示如下

```
1 Date: Wed, 21 Oct 2015 07:28:00 GMT
```

表示的是格林威治标准时间，这个时间要比北京时间慢八个小时

北京时间 → 格林威治标准时间

北京时间: 20:00 (8:00 PM)

格林威治标准时间: 12:00 (12:00 PM)

格林威治标准时间 → 北京时间

格林威治标准时间: 12:00 (12:00 PM))

北京时间: 20:00 (8:00 PM)

换级: 00:00

## Cache-Control

Cache-Control 是一个通用标头，他可以出现在 请求标头 和 响应标头 中，Cache-Control 的种类比较多，虽然说这是一个通用标头，但是又一些特性是请求标头具有的，有一些是响应标头才有的。主要大类有 可缓存性 、 阈值性 、 重新验证并重新加载 和 其他特性

## Connection

Connection 决定当前事务（一次三次握手和四次挥手）完成后，是否会关闭网络连接。Connection 有两种，一种是 持久性连接 ，即一次事务完成后不关闭网络连接

```
1 Connection: keep-alive
```

另一种是 非持久性连接 ，即一次事务完成后关闭网络连接

```
1 Connection: close
```

HTTP1.1 其他通用标头如下

| 首部字段名             | 说明            |
|-------------------|---------------|
| Cache-Control     | 控制缓存的行为       |
| Connection        | 逐跳首部、连接的管理    |
| Date              | 创建报文的日期时间     |
| Pragma            | 报文指令          |
| Trailer           | 报文末端的首部一览     |
| Transfer-Encoding | 指定报文主体的传输编码方式 |
| Upgrade           | 升级为其他协议       |
| Via               | 代理服务器的相关信息    |
| Warning           | 错误通知          |

## 实体标头



实体标头是描述消息正文内容的 HTTP 标头。实体标头用于 HTTP 请求和响应中。头部 `Content-Length`、`Content-Language`、`Content-Encoding` 是实体头。

- `Content-Length` 实体报头指示实体主体的大小，以字节为单位，发送到接收方。
  - `Content-Language` 实体报头描述了客户端或者服务端能够接受的语言。
  - `Content-Encoding` 这又是一个比较麻烦的属性，这个实体报头用来压缩媒体类型。`Content-Encoding` 指示对实体应用了何种编码。
- 常见的内容编码有这几种：`gzip`、`compress`、`deflate`、`identity`，这个属性可以应用在请求报文和响应报文中

```
1 Accept-Encoding: gzip, deflate //请求头
2 Content-Encoding: gzip //响应头
```

下面是一些实体标头字段

| 首部字段名            | 说明             |
|------------------|----------------|
| Allow            | 资源可支持的HTTP方法   |
| Content-Encoding | 实体主体适用的编码方式    |
| Content-Language | 实体主体的自然语言      |
| Content-Length   | 实体主体的大小（单位：字节） |
| Content-Location | 替代对应资源的URI     |
| Content-MD5      | 实体主体的报文摘要      |
| Content-Range    | 实体主体的位置范围      |
| Content-Type     | 实体主体的媒体类型      |
| Expires          | 实体主体过期的日期时间    |
| Last-Modified    | 资源的最后修改日期时间    |

## 请求标头

### Host

`Host` 请求头指明了服务器的域名（对于虚拟主机来说），以及（可选的）服务器监听的 TCP 端口号。如果没有给定端口号，会自动使用被请求服务的默认端口（比如请求一个 HTTP 的 URL 会自动使用 80 作为端口）。

```
1 Host: developer.mozilla.org
```

上面的 `Accpet`、`Accept-Language`、`Accept-Encoding` 都是属于内容协商的请求标头。

### Referer

`HTTP Referer` 属性是请求标头的一部分，当浏览器向 web 服务器发送请求的时候，一般会带上 `Referer`，告诉服务器该网页是从哪个页面链接过来的，服务器因此可以获得一些信息用于处理。

```
1 Referer: https://developer.mozilla.org/testpage.html
```

### If-Modified-Since

If-Modified-Since 通常会与 If-None-Match 搭配使用，If-Modified-Since 用于确认代理或客户端拥有的本地资源的有效性。获取资源的更新日期时间，可通过确认首部字段 `Last-Modified` 来确定。

大白话说就是如果在 `Last-Modified` 之后更新了服务器资源，那么服务器会响应 200，如果在 `Last-Modified` 之后没有更新过资源，则返回 304。

```
1 If-Modified-Since: Mon, 18 Jul 2016 02:36:04 GMT
```

## If-None-Match

If-None-Match HTTP 请求标头使请求成为条件请求。对于 GET 和 HEAD 方法，仅当服务器没有与给定资源匹配的 `ETag` 时，服务器才会以 200 状态发送回请求的资源。对于其他方法，仅当最终现有资源的 `ETag` 与列出的任何值都不匹配时，才会处理请求。

```
1 If-None-Match: "c561c68d0ba92bbeb8b0fff2a9199f722e3a621a"
```

## Accept

接受请求 HTTP 标头会通告客户端其能够理解的 MIME 类型

## Accept-Charset

accept-charset 属性规定服务器处理表单数据所接受的字符集。

常用的字符集有：UTF-8 - Unicode 字符编码；ISO-8859-1 - 拉丁字母表的字符编码

## Accept-Language

首部字段 Accept-Language 用来告知服务器用户代理能够处理的自然语言集（指中文或英文等），以及自然语言集的相对优先级。可一次指定多种自然语言集。

请求标头我们大概就介绍这几种，后面会有一篇文章详细深挖所有的响应头的，下面是一个响应头的汇总，基于 HTTP 1.1

| 首部字段名               | 说明                              |
|---------------------|---------------------------------|
| Accept              | 用户代理可处理的媒体类型                    |
| Accept-Charset      | 优先的字符集                          |
| Accept-Encoding     | 优先的内容编码                         |
| Accept-Language     | 优先的语言（自然语言）                     |
| Authorization       | Web认证信息                         |
| Expect              | 期待服务器的特定行为                      |
| From                | 用户的电子邮箱地址                       |
| Host                | 请求资源所在服务器                       |
| If-Match            | 比较实体标记（ETag）                    |
| If-Modified-Since   | 比较资源的更新时间                       |
| If-None-Match       | 比较实体标记（与 If-Match 相反）           |
| If-Range            | 资源未更新时发送实体 Byte 的范围请求           |
| If-Unmodified-Since | 比较资源的更新时间（与If-Modified-Since相反） |
| Max-Forwards        | 最大传输逐跳数                         |
| Proxy-Authorization | 代理服务器要求客户端的认证信息                 |
| Range               | 实体的字节范围请求                       |
| Referer             | 对请求中 URI 的原始获取方                 |
| TE                  | 传输编码的优先级                        |
| User-Agent          | HTTP 客户端程序的信息                   |

## 响应标头

### Access-Control-Allow-Origin

一个返回的 HTTP 标头可能会具有 Access-Control-Allow-Origin , `Access-Control-Allow-Origin` 指定一个来源，它告诉浏览器允许该来源进行资源访问。

### Keep-Alive

Keep-Alive 表示的是 Connection 非持续连接的存活时间，可以进行指定。

### Server

服务器标头包含有关原始服务器用来处理请求的软件的信息。

应该避免使用过于冗长和详细的 Server 值，因为它们可能会泄露内部实施细节，这可能会使攻击者容易地发现并利用已知的安全漏洞。例如下面这种写法

```
1 Server: Apache/2.4.1 (Unix)
```

### Set-Cookie

Set-Cookie 用于服务器向客户端发送 sessionID。

### Transfer-Encoding

首部字段 Transfer-Encoding 规定了传输报文主体时采用的编码方式。

HTTP /1.1 的传输编码方式仅对分块传输编码有效。

## X-Frame-Options

HTTP 首部字段是可以自行扩展的。所以在 Web 服务器和浏览器的应用上，会出现各种非标准的首部字段。

首部字段 `X-Frame-Options` 属于 HTTP 响应首部，用于控制网站内容在其他 Web 网站的 Frame 标签内的显示问题。其主要目的是为了防止点击劫持（clickjacking）攻击。

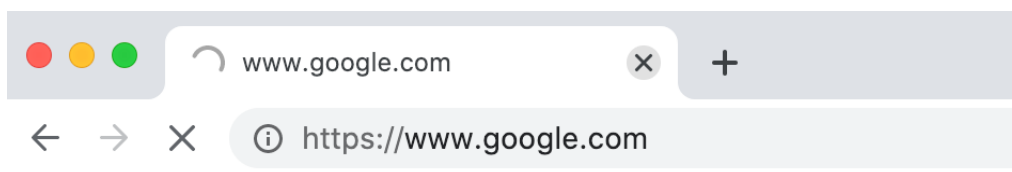
下面是一个响应头的汇总，基于 HTTP 1.1

| 首部字段名              | 说明             |
|--------------------|----------------|
| Accept-Ranges      | 是否接受字节范围请求     |
| Age                | 推算资源创建经过时间     |
| ETag               | 资源的匹配信息        |
| Location           | 令客户端重定向至指定URI  |
| Proxy-Authenticate | 代理服务器对客户端的认证信息 |
| Retry-After        | 对再次发起请求的时机要求   |
| Server             | HTTP服务器的安装信息   |
| Vary               | 代理服务器缓存的管理信息   |
| WWW-Authenticate   | 服务器对客户端的认证信息   |

## 地址栏输入 URL 发生了什么

这道题也是一道经常会考的面试题。那么下面我们就来探讨一下从你输入 URL 后到响应，都经历了哪些过程。

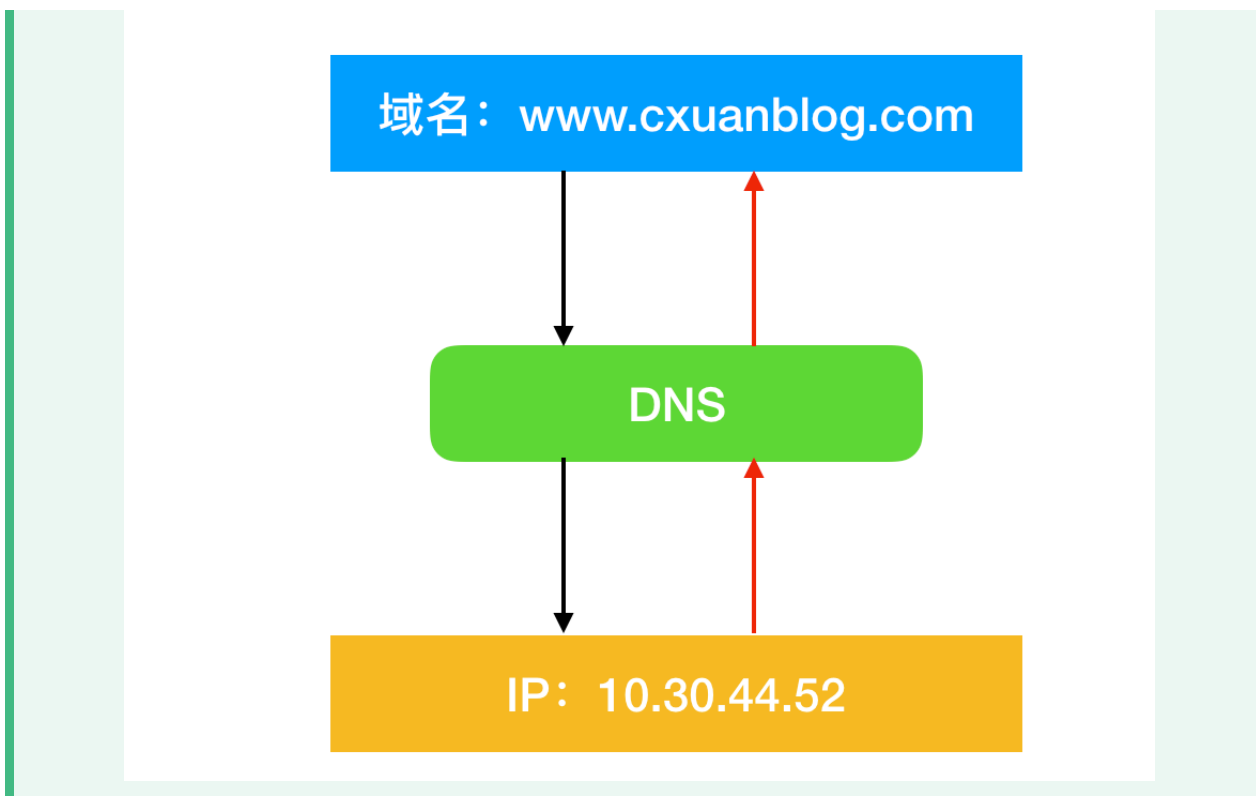
- 首先，你需要在浏览器中的 URL 地址上，输入你想访问的地址，如下



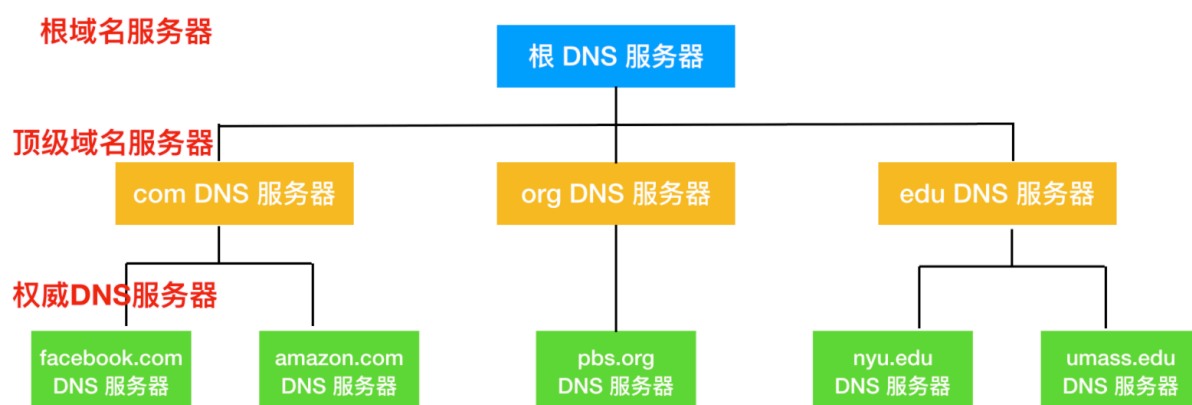
你应该访问不到的，对不对~

- 然后，浏览器会根据你输入的 URL 地址，去查找域名是否被本地 DNS 缓存，不同浏览器对 DNS 的设置不同，如果浏览器缓存了你访问的 URL 地址，那就直接返回 ip。如果没有缓存你的 URL 地址，浏览器就会发起系统调用来查询本机 `hosts` 文件是否有配置 ip 地址，如果找到，直接返回。如果找不到，就向网络中发起一个 DNS 查询。

首先来看一下 DNS 是啥，互联网中识别主机的方式有两种，通过 `主机名` 和 `IP 地址`。我们人喜欢用名字的方式进行记忆，但是通信链路中的路由却喜欢定长、有层次结构的 IP 地址。所以需要一种能够把主机名到 IP 地址的转换服务，这种服务就是由 DNS 提供的。DNS 的全称是 `Domain Name System` 域名系统。DNS 是一种由分层的 DNS 服务器实现的分布式数据库。DNS 运行在 UDP 上，使用 53 端口。



DNS 是一种分层数据库，它的主要层次结构如下



一般域名服务器的层次结构主要是以上三种，除此之外，还有另一类重要的 DNS 服务器，它是 **本地 DNS 服务器(local DNS server)**。严格来说，本地 DNS 服务器并不属于上述层次结构，但是本地 DNS 服务器又是至关重要的。每个 **ISP(Internet Service Provider)** 比如居民区的 ISP 或者一个机构的 ISP 都有一台本地 DNS 服务器。当主机和 ISP 进行连接时，该 ISP 会提供一台主机的 IP 地址，该主机会具有一台或多台其本地 DNS 服务器的 IP 地址。通过访问网络连接，用户能够容易的确定 DNS 服务器的 IP 地址。当主机发出 DNS 请求后，该请求被发往本地 DNS 服务器，它起着代理的作用，并将该请求转发到 DNS 服务器层次系统中。

首先，查询请求会先找到本地 DNS 服务器来查询是否包含 IP 地址，如果本地 DNS 无法查询到目标 IP 地址，就会向根域名服务器发起一个 DNS 查询。

注意：DNS 涉及两种查询方式：一种是 **递归查询(Recursive query)**，一种是 **迭代查询(Iteration query)**。《计算机网络：自顶向方法》竟然没有给出递归查询和迭代查询的区别，找了一下网上的资料大概明白了下。

如果根域名服务器无法告知本地 DNS 服务器下一步需要访问哪个顶级域名服务器，就会使用递归查询；

如果根域名服务器能够告知 DNS 服务器下一步需要访问的顶级域名服务器，就会使用迭代查询。

在由根域名服务器 -> 顶级域名服务器 -> 权威 DNS 服务器后，由权威服务器告诉本地服务器目标 IP 地址，再有本地 DNS 服务器告诉用户需要访问的 IP 地址。

- 第三步，浏览器需要和目标服务器建立 TCP 连接，需要经过三次握手的过程，具体的握手过程请参考上面的回答。
- 在建立连接后，浏览器会向目标服务器发起 **HTTP-GET** 请求，包括其中的 URL，HTTP 1.1 后默认使用长连接，只需要一次握手即可多次传输数据。
- 如果目标服务器只是一个简单的页面，就会直接返回。但是对于某些大型网站的站点，往往不会直接返回主机名所在的页面，而会直接重定向。返回的状态码就不是 200，而是 301,302 以 3 开头的重定向码，浏览器在获取了重定向响应后，在响应报文中 Location 项找到重定向地址，浏览器重新第一步访问即可。
- 然后浏览器重新发送请求，携带新的 URL，返回状态码 200 OK，表示服务器可以响应请求，返回报文。

## HTTPS 的工作原理

我们上面描述了一下 HTTP 的工作原理，下面来讲述一下 HTTPS 的工作原理。因为我们知道 HTTPS 不是一种新出现的协议，而是

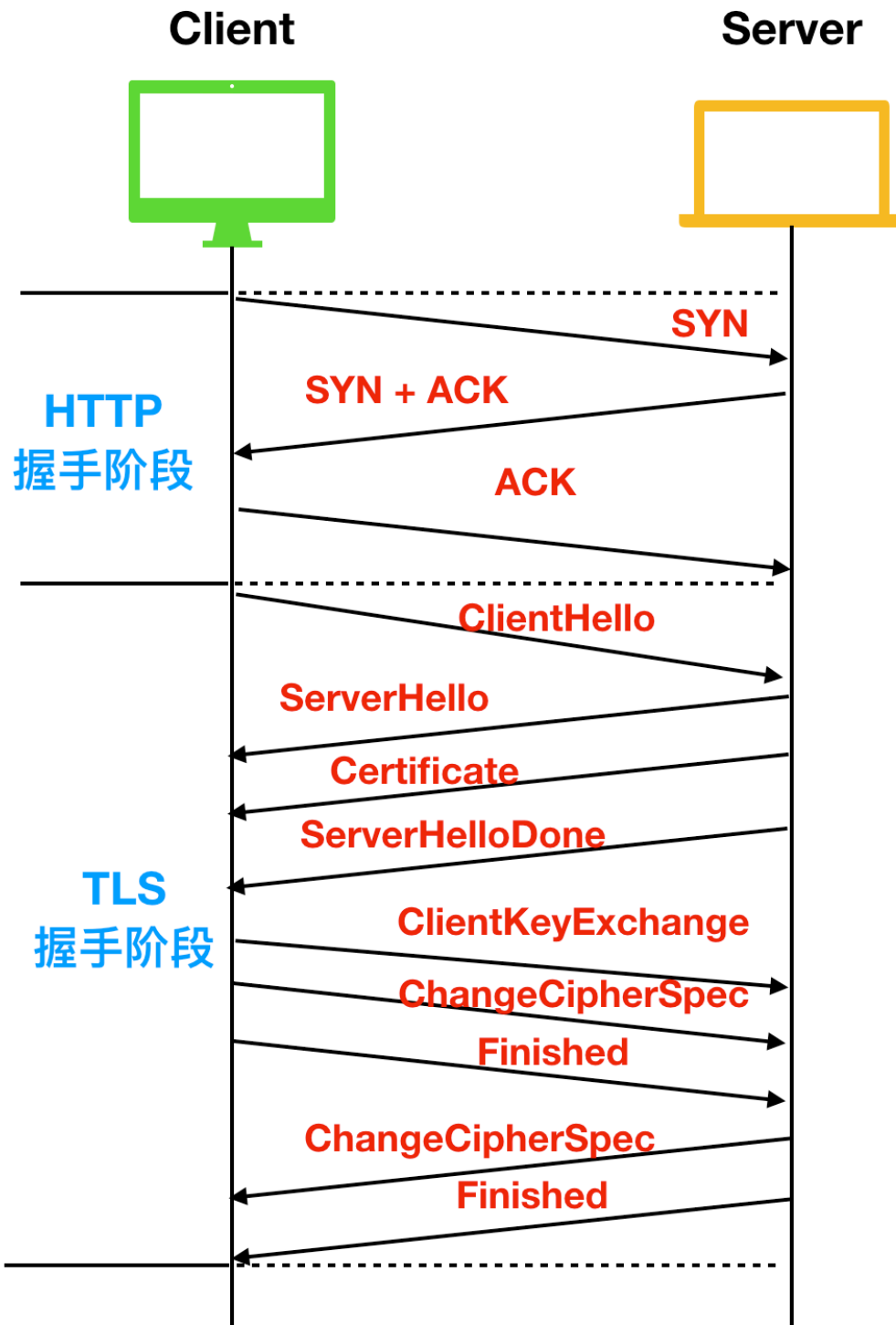


所以，我们探讨 HTTPS 的握手过程，其实就是 SSL/TLS 的握手过程。

TLS 旨在为 Internet 提供通信安全的加密协议。TLS 握手是启动和使用 TLS 加密的通信会话的过程。在 TLS 握手期间，Internet 中的通信双方会彼此交换信息，验证密码套件，交换会话密钥。

每当用户通过 HTTPS 导航到具体的网站并发送请求时，就会进行 TLS 握手。除此之外，每当其他任何通信使用 HTTPS（包括 API 调用和在 HTTPS 上查询 DNS）时，也会发生 TLS 握手。

TLS 具体的握手过程会根据所使用的 **密钥交换算法的类型** 和双方支持的 **密码套件** 而不同。我们以 **RSA 非对称加密** 来讨论这个过程。整个 TLS 通信流程图如下



- 在进行通信前，首先会进行 HTTP 的三次握手，握手完成后，再进行 TLS 的握手过程
- ClientHello：客户端通过向服务器发送 `hello` 消息来发起握手过程。这个消息中会夹带着客户端支持的 `TLS 版本号(TLS1.0 、 TLS1.2、 TLS1.3)`、客户端支持的密码套件、以及一串 `客户端随机数`。
- ServerHello：在客户端发送 `hello` 消息后，服务器会发送一条消息，这条消息包含了服务器的 SSL 证书、服务器选择的密码套件和服务器生成的随机数。
- 认证(Authentication)：客户端的证书颁发机构会认证 SSL 证书，然后发送 `Certificate` 报文，报文中包含公开密钥证书。最后服务器发送 `ServerHelloDone` 作为 `hello` 请求的响应。第一部分握手阶段结束。
- 加密阶段：在第一个阶段握手完成后，客户端会发送 `ClientKeyExchange` 作为响应，这个响应中包含了一种称为 `The premaster secret` 的密钥字符串，这个字符串就是使用上面公开密



钥证书进行加密的字符串。随后客户端会发送 `ChangeCipherSpec`，告诉服务端使用私钥解密这个 `premaster secret` 的字符串，然后客户端发送 `Finished` 告诉服务端自己发送完成了。

Session key 其实就是用公钥证书加密的公钥。

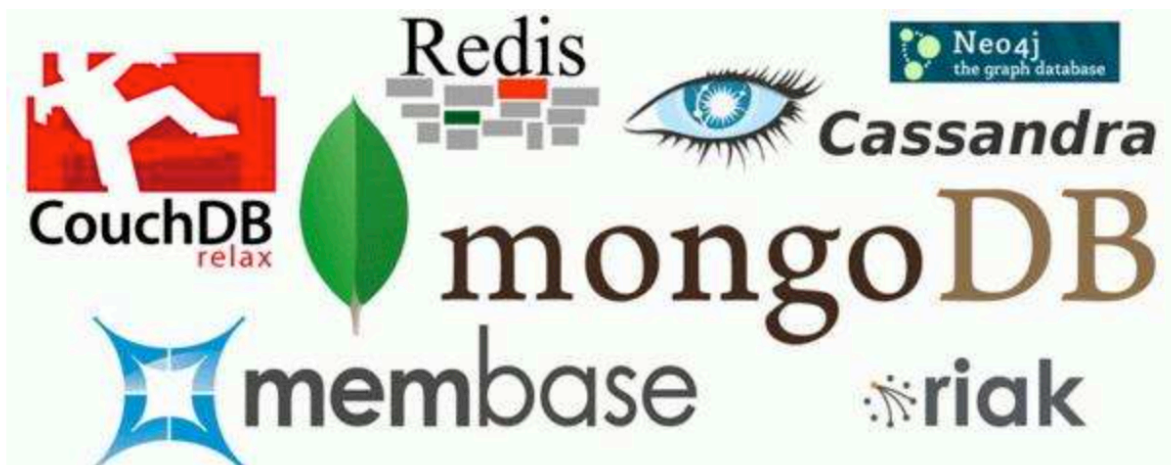
- **实现了安全的非对称加密**：然后，服务器再发送 `ChangeCipherSpec` 和 `Finished` 告诉客户端解密完成，至此实现了 RSA 的非对称加密。

## MySQL 篇

MySQL 一直是本人很薄的部分，后面会多输出 MySQL 的文章贡献给大家，毕竟 MySQL 涉及到数据存储、锁、磁盘寻道、分页等操作系统概念，而且互联网对 MySQL 的注重程度是不言而喻的，后面要加紧对 MySQL 的研究。写的如果不好，还请大家见谅。



## 非关系型数据库和关系型数据库区别，优势比较



非关系型数据库（感觉翻译不是很准确）称为 **NoSQL**，也就是 Not Only SQL，不仅仅是 SQL。非关系型数据库不需要写一些复杂的 SQL 语句，其内部存储方式是以 **key-value** 的形式存在可以把它想象成电话本的形式，每个人名（key）对应电话（value）。常见的非关系型数据库主要有 **Hbase**、**Redis**、**MongoDB** 等。非关系型数据库不需要经过 SQL 的重重解析，所以性能很高；非关系型数据库的可扩展性比较强，数据之间没有耦合性，遇见需要新加字段的需求，就直接增加一个 key-value 键值对即可。





关系型数据库以 **表格** 的形式存在，以 **行和列** 的形式存取数据，关系型数据库这一系列的行和列被称为表，无数张表组成了 **数据库**，常见的关系型数据库有 **Oracle**、**DB2**、**Microsoft SQL Server**、**MySQL** 等。关系型数据库能够支持复杂的 SQL 查询，能够体现出数据之间、表之间的关联关系；关系型数据库也支持事务，便于提交或者回滚。

它们之间的劣势都是基于对方的优势来满足的。

## MySQL 事务四大特性

一说到 MySQL 事务，你肯定能想起来四大特性：**原子性**、**一致性**、**隔离性**、**持久性**，下面再对这事务的四大特性做一个描述

- **原子性(Atomicity)**：原子性指的就是 MySQL 中的包含事务的操作要么 **全部成功**、要么全部 **失败回滚**，因此事务的操作如果成功就必须全部应用到数据库，如果操作失败则不能对数据库有任何影响。

这里涉及到一个概念，什么是 MySQL 中的事务？

事务是一组操作，组成这组操作的各个单元，要不全都成功要不全都失败，这个特性就是事务。

在 MySQL 中，事务是在引擎层实现的，只有使用 **innodb** 引擎的数据库或表才支持事务。

- **一致性(Consistency)**：一致性指的是一个事务在执行前后其状态一致。比如 A 和 B 加起来的钱一共是 1000 元，那么不管 A 和 B 之间如何转账，转多少次，事务结束后两个用户的钱加起来还得是 1000，这就是事务的一致性。
- **持久性(Durability)**：持久性指的是一旦事务提交，那么发生的改变就是永久性的，即使数据库遇到特殊情况比如故障的时候也不会产生干扰。
- **隔离性(Isolation)**：隔离性需要重点说一下，当多个事务同时进行，就有可能出现 **脏读** (dirty read)、**不可重复读(non-repeatable read)**、**幻读(phantom read)** 的情况，为了解决这些并发问题，提出了隔离性的概念。

脏读：事务 A 读取了事务 B 更新后的数据，但是事务 B 没有提交，然后事务 B 执行回滚操作，那么事务 A 读到的数据就是脏数据

不可重复读：事务 A 进行多次读取操作，事务 B 在事务 A 多次读取的过程中执行更新操作并提交，提交后事务 A 读到的数据不一致。

幻读：事务 A 将数据库中所有学生的成绩由 A -> B，此时事务 B 手动插入了一条成绩为 A 的记录，在事务 A 更改完毕后，发现还有一条记录没有修改，那么这种情况就叫做出现了幻读。

SQL的隔离级别有四种，它们分别是 读未提交(read uncommitted)、 读已提交(read committed)、 可重复读(repetable read) 和 串行化(serializable)。下面分别来解释一下。

- 读未提交：读未提交指的是一个事务在提交之前，它所做的修改就能够被其他事务所看到。
- 读已提交：读已提交指的是一个事务在提交之后，它所做的变更才能够让其他事务看到。
- 可重复读：可重复读指的是一个事务在执行的过程中，看到的数据是和启动时看到的数据是一致的。未提交的变更对其他事务不可见。
- 串行化：顾名思义是对于同一行记录，写 会加 写锁，读 会加 读锁。当出现读写锁冲突的时候，后访问的事务必须等前一个事务执行完成，才能继续执行。

这四个隔离级别可以解决脏读、不可重复读、幻象读这三类问题。总结如下

| 事务隔离级别 | 脏读  | 不可重复读 | 幻读  |
|--------|-----|-------|-----|
| 读未提交   | 允许  | 允许    | 允许  |
| 读已提交   | 不允许 | 允许    | 允许  |
| 可重复读   | 不允许 | 不允许   | 允许  |
| 串行化    | 不允许 | 不允许   | 不允许 |

其中隔离级别由低到高是：读未提交 < 读已提交 < 可重复读 < 串行化

隔离级别越高，越能够保证数据的完整性和一致性，但是对并发的性能影响越大。大多数数据库的默认级别是 读已提交(Read committed)，比如 Sql Server、Oracle，但是 MySQL 的默认隔离级别是 可重复读(repeatable-read)。

## MySQL 常见存储引擎的区别

MySQL 常见的存储引擎，可以使用

```
1 SHOW ENGINES
```

命令，来列出所有的存储引擎

| Engine             | Support | Comment                                                        | Transactions | XA     | Savepoints |
|--------------------|---------|----------------------------------------------------------------|--------------|--------|------------|
| InnoDB             | DEFAULT | Supports transactions, row-level locking, and foreign keys     | YES          | YES    | YES        |
| MRG_MYISAM         | YES     | Collection of identical MyISAM tables                          | NO           | NO     | NO         |
| MEMORY             | YES     | Hash based, stored in memory, useful for temporary tables      | NO           | NO     | NO         |
| BLACKHOLE          | YES     | /dev/null storage engine (anything you write to it disappears) | NO           | NO     | NO         |
| MyISAM             | YES     | MyISAM storage engine                                          | NO           | NO     | NO         |
| CSV                | YES     | CSV storage engine                                             | NO           | NO     | NO         |
| ARCHIVE            | YES     | Archive storage engine                                         | NO           | NO     | NO         |
| PERFORMANCE_SCHEMA | YES     | Performance Schema                                             | NO           | NO     | NO         |
| FEDERATED          | NO      | Federated MySQL storage engine                                 | (NULL)       | (NULL) | (NULL)     |

可以看到，InnoDB 是 MySQL 默认支持的存储引擎，支持事务、行级锁定和外键。

## MyISAM 存储引擎的特点

在 5.1 版本之前，MyISAM 是 MySQL 的默认存储引擎，MyISAM 并发性比较差，使用的场景比较少，主要特点是

- 不支持 **事务** 操作，ACID 的特性也就不存在了，这一设计是为了性能和效率考虑的。
- 不支持 **外键** 操作，如果强行增加外键，MySQL 不会报错，只不过外键不起作用。
- MyISAM 默认的锁粒度是 **表级锁**，所以并发性能比较差，加锁比较快，锁冲突比较少，不太容易发生死锁的情况。
- MyISAM 会在磁盘上存储三个文件，文件名和表名相同，扩展名分别是 **.frm(存储表定义)**、**.MYD(MYData,存储数据)**、**MYI(MyIndex,存储索引)**。这里需要特别注意的是 MyISAM 只缓存 **索引文件**，并不缓存数据文件。
- MyISAM 支持的索引类型有 **全局索引(Full-Text)**、**B-Tree 索引**、**R-Tree 索引**  
Full-Text 索引：它的出现是为了解决针对文本的模糊查询效率较低的问题。  
B-Tree 索引：所有的索引节点都按照平衡树的数据结构来存储，所有的索引数据节点都在叶节点  
R-Tree索引：它的存储方式和 B-Tree 索引有一些区别，主要设计用于存储空间和多维数据的字段做索引,目前的 MySQL 版本仅支持 geometry 类型的字段作索引，相对于 BTREE，RTREE 的优势在于范围查找。
- 数据库所在主机如果宕机，MyISAM 的数据文件容易损坏，而且难以恢复。
- 增删改查性能方面：SELECT 性能较高，适用于查询较多的情况

## InnoDB 存储引擎的特点

自从 MySQL 5.1 之后，默认的存储引擎变成了 InnoDB 存储引擎，相对于 MyISAM，InnoDB 存储引擎有了较大的改变，它的主要特点是

- 支持事务操作，具有事务 ACID 隔离特性，默认的隔离级别是 **可重复读(repetable-read)**、通过 MVCC（并发版本控制）来实现的。能够解决 **脏读** 和 **不可重复读** 的问题。
- InnoDB 支持外键操作。
- InnoDB 默认的锁粒度 **行级锁**，并发性能比较好，会发生死锁的情况。
- 和 MyISAM 一样的是，InnoDB 存储引擎也有 **.frm文件存储表结构** 定义，但是不同的是，InnoDB 的表数据与索引数据是存储在一起的，都位于 B+ 数的叶子节点上，而 MyISAM 的表数据和索引数据是分开的。
- InnoDB 有安全的日志文件，这个日志文件用于恢复因数据库崩溃或其他情况导致的数据丢失问题，保证数据的一致性。
- InnoDB 和 MyISAM 支持的索引类型相同，但具体实现因为文件结构的不同有很大差异。
- 增删改查性能方面，果执行大量的增删改操作，推荐使用 InnoDB 存储引擎，它在删除操作时是对行删除，不会重建表。

## MyISAM 和 InnoDB 存储引擎的对比

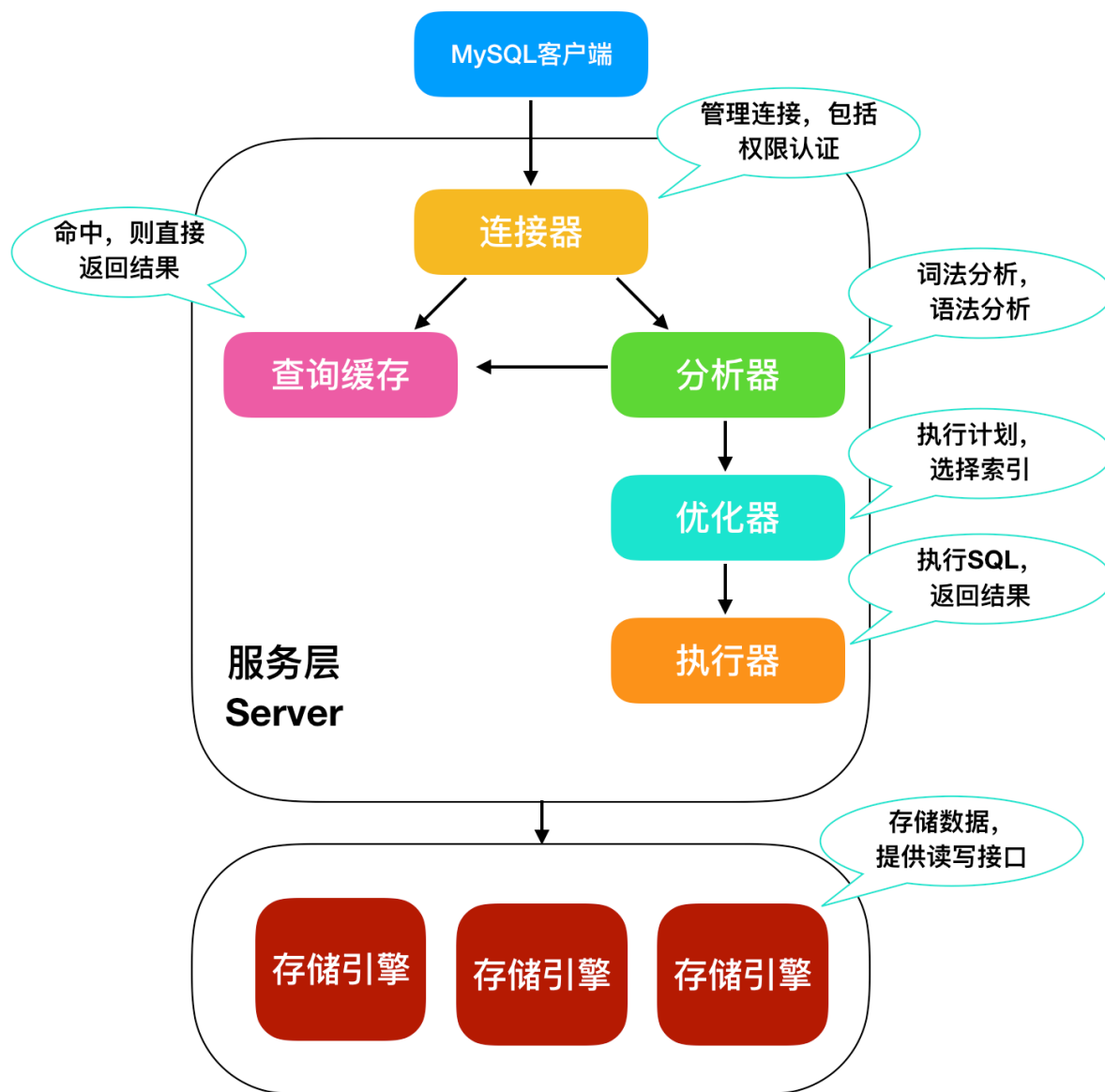
- **锁粒度方面**：由于锁粒度不同，InnoDB 比 MyISAM 支持更高的并发；InnoDB 的锁粒度为行锁、MyISAM 的锁粒度为表锁、行锁需要对每一行进行加锁，所以锁的开销更大，但是能解决脏读和不可重复读的问题，相对来说也更容易发生死锁
- **可恢复性上**：由于 InnoDB 是有事务日志的，所以在产生由于数据库崩溃等条件后，可以根据日志文件进行恢复。而 MyISAM 则没有事务日志。
- **查询性能上**：MyISAM 要优于 InnoDB，因为 InnoDB 在查询过程中，是需要维护数据缓存，而且查询过程是先定位到行所在的数据块，然后在从数据块中定位到要查找的行；而 MyISAM 可以

直接定位到数据所在的内存地址，可以直接找到数据。

- **表结构文件上**：MyISAM 的表结构文件包括：.frm(表结构定义),.MYI(索引),.MYD(数据)；而 InnoDB 的表数据文件为:.ibd和.frm(表结构定义)；

## MySQL 基础架构

这道题应该从 MySQL 架构来理解，我们可以把 MySQL 拆解成几个零件，如下图所示



### MySQL 逻辑架构

大致上来说，MySQL 可以分为 **Server** 层和 **存储引擎** 层。

Server 层包括连接器、查询缓存、分析器、优化器、执行器，包括大多数 MySQL 中的核心功能，所有跨存储引擎的功能也在这一层实现，包括 **存储过程**、**触发器**、**视图**等。

存储引擎层包括 MySQL 常见的存储引擎，包括 **MyISAM**、**InnoDB** 和 **Memory** 等，最常用的是 InnoDB，也是现在 MySQL 的默认存储引擎。存储引擎也可以在创建表的时候手动指定，比如下面

```
1 CREATE TABLE t (i INT) ENGINE = <Storage Engine>;
```

然后我们就可以探讨 MySQL 的执行过程了

## 连接器

首先需要在 MySQL 客户端登陆才能使用，所以需要有一个 **连接器** 来连接用户和 MySQL 数据库，我们一般是使用

```
1 mysql -u 用户名 -p 密码
```

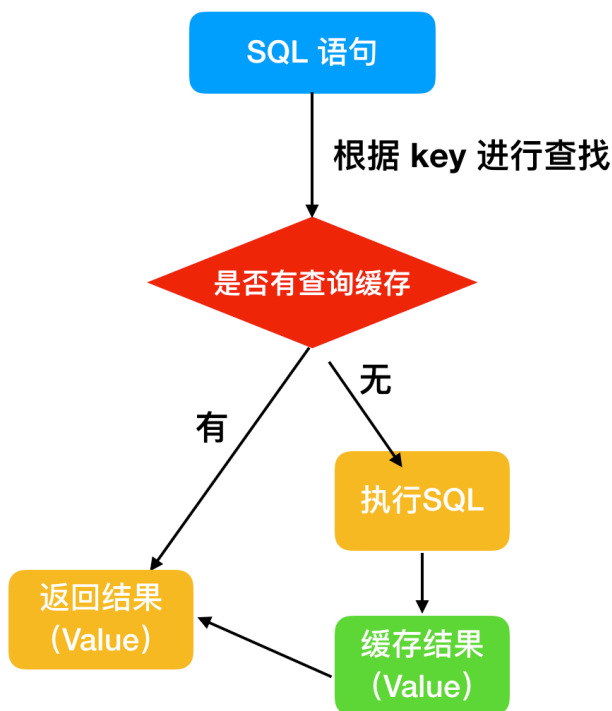
来进行 MySQL 登陆，和服务端建立连接。在完成 **TCP 握手** 后，连接器会根据你输入的用户名和密码验证你的登录身份。如果用户名或者密码错误，MySQL 就会提示 **Access denied for user**，来结束执行。如果登录成功后，MySQL 会根据权限表中的记录来判定你的权限。

## 查询缓存

连接完成后，你就可以执行 SQL 语句了，这行逻辑就会来到第二步：查询缓存。

MySQL 在得到一个执行请求后，会首先去 **查询缓存** 中查找，是否执行过这条 SQL 语句，之前执行过的语句以及结果会以 **key-value** 对的形式，被直接放在内存中。key 是查询语句，value 是查询的结果。如果通过 key 能够查找到这条 SQL 语句，就直接返回 SQL 的执行结果。

如果语句不在查询缓存中，就会继续后面的执行阶段。执行完成后，执行结果就会被放入查询缓存中。可以看到，如果查询命中缓存，MySQL 不需要执行后面的复杂操作，就可以直接返回结果，效率会很高。



### 但是查询缓存不建议使用

为什么呢？因为只要在 MySQL 中对某一张表执行了更新操作，那么所有的查询缓存就会失效，对于更新频繁的数据库来说，查询缓存的命中率很低。

## 分析器

如果没有命中查询，就开始执行真正的 SQL 语句。

- 首先，MySQL 会根据你写的 SQL 语句进行解析，分析器会先做 **词法分析**，你写的 SQL 就是由

多个字符串和空格组成的一条 SQL 语句，MySQL 需要识别出里面的字符串是什么，代表什么。

- 然后进行 **语法分析**，根据词法分析的结果，语法分析器会根据语法规则，判断你输入的这个 SQL 语句是否满足 MySQL 语法。如果 SQL 语句不正确，就会提示 **You have an error in your SQL syntax**

## 优化器

经过分析器的词法分析和语法分析后，你这条 SQL 就 **合法** 了，MySQL 就知道你要做什么了。但是在执行前，还需要进行优化器的处理，优化器会判断你使用了哪种索引，使用了何种连接，优化器的作用就是确定效率最高的执行方案。

## 执行器

MySQL 通过分析器知道了你的 SQL 语句是否合法，你想要做什么操作，通过优化器知道了该怎么做效率最高，然后就进入了执行阶段，开始执行这条 SQL 语句

在执行阶段，MySQL 首先会判断你有没有执行这条语句的权限，没有权限的话，就会返回没有权限的错误。如果有权限，就打开表继续执行。打开表的时候，执行器就会根据表的引擎定义，去使用这个引擎提供的接口。对于有索引的表，执行的逻辑也差不多。

至此，MySQL 对于一条语句的执行过程也就完成了。

## SQL 的执行顺序

我们在编写一个查询语句的时候

```
1 SELECT DISTINCT
2 < select_list >
3 FROM
4 < left_table > < join_type >
5 JOIN < right_table > ON < join_condition >
6 WHERE
7 < where_condition >
8 GROUP BY
9 < group_by_list >
10 HAVING
11 < having_condition >
12 ORDER BY
13 < order_by_condition >
14 LIMIT < limit_number >
```

它的执行顺序你知道吗？这道题就给你一个回答。

## FROM 连接

首先，对 SELECT 语句执行查询时，对 **FROM** 关键字两边的表执行连接，会形成 **笛卡尔积**，这时候会产生一个 **虚表VT1(virtual table)**

首先先来解释一下什么是 **笛卡尔积**

现在我们有二个集合  $A = \{0,1\}$ ， $B = \{2,3,4\}$

那么，集合  $A * B$  得到的结果就是



$A * B = \{(0,2)、(1,2)、(0,3)、(1,3)、(0,4)、(1,4)\};$

$B * A = \{(2,0)、(2,1)、(3,0)、(3,1)、(4,0)、(4,1)\};$

上面  $A * B$  和  $B * A$  的结果就可以称为两个集合相乘的 **笛卡尔积**

我们可以得出结论，A 集合和 B 集合相乘，包含了集合 A 中的元素和集合 B 中元素之和，也就是 A 元素的个数 \* B 元素的个数

再来解释一下什么是虚表

在 MySQL 中，有三种类型的表

一种是 **永久表**，永久表就是创建以后用来长期保存数据的表

一种是 **临时表**，临时表也有两类，一种是和永久表一样，只保存临时数据，但是能够长久存在的；还有一种是临时创建的，SQL 语句执行完成就会删除。

一种是 **虚表**，虚表其实就是 **视图**，数据可能会来自多张表的执行结果。

## ON 过滤

然后对 FROM 连接的结果进行 ON 筛选，创建 VT2，把符合记录的条件存在 VT2 中。

## JOIN 连接

第三步，如果是 **OUTER JOIN(left join、right join)**，那么这一步就将添加外部行，如果是 left join 就把 ON 过滤条件的左表添加进来，如果是 right join，就把右表添加进来，从而生成新的虚拟表 VT3。

## WHERE 过滤

第四步，是执行 WHERE 过滤器，对上一步生产的虚拟表引用 WHERE 筛选，生成虚拟表 VT4。

WHERE 和 ON 的区别

- 如果有外部列，ON 针对过滤的是关联表，主表(保留表)会返回所有的列；
- 如果没有添加外部列，两者的效果是一样的；

应用

- 对主表的过滤应该使用 WHERE；
- 对于关联表，先条件查询后连接则用 ON，先连接后条件查询则用 WHERE；

## GROUP BY

根据 group by 字句中的列，会对 VT4 中的记录进行分组操作，产生虚拟机表 VT5。果应用了 group by，那么后面的所有步骤都只能得到的 VT5 的列或者是聚合函数（count、sum、avg等）。

## HAVING

紧跟着 GROUP BY 字句后面的是 HAVING，使用 HAVING 过滤，会把符合条件的放在 VT6

## SELECT

第七步才会执行 SELECT 语句，将 VT6 中的结果按照 SELECT 进行刷选，生成 VT7

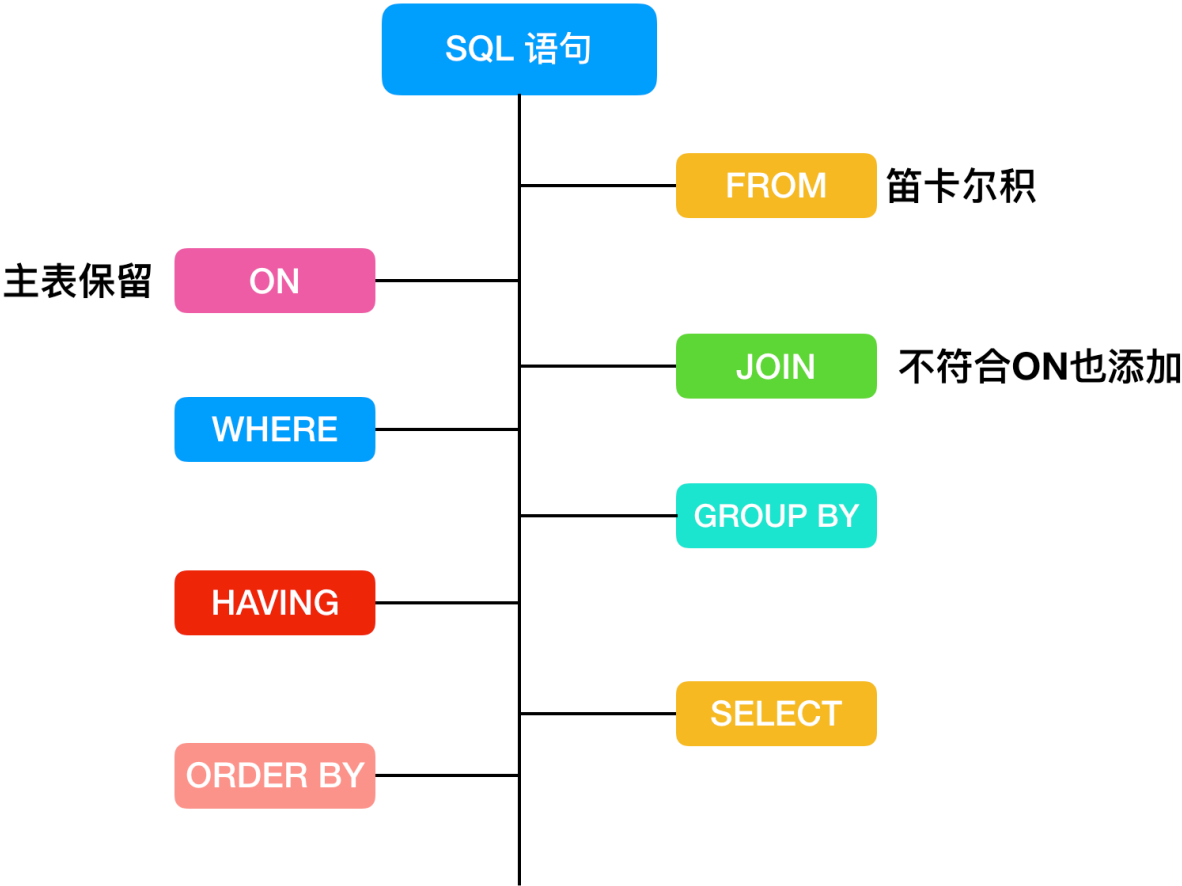
### DISTINCT

在第八步中，会对 TV7 生成的记录进行去重操作，生成 VT8。事实上如果应用了 group by 子句那么 distinct 是多余的，原因同样在于，分组的时候是将列中唯一的值分成一组，同时只为每一组返回一行记录，那么所以的记录都将是不相同的。

### ORDER BY

应用 order by 子句。按照 order\_by\_condition 排序 VT8，此时返回的一个游标，而不是虚拟表。sql 是基于集合的理论的，集合不会预先对他的行排序，它只是成员的逻辑集合，成员的顺序是无关紧要的。

SQL 语句执行的过程如下



### 什么是临时表，何时删除临时表

什么是临时表？MySQL 在执行 SQL 语句的过程中，通常会临时创建一些 **存储中间结果集** 的表，临时表只对当前连接可见，在连接关闭时，临时表会被删除并释放所有表空间。

临时表分为两种：一种是 **内存临时表**，一种是 **磁盘临时表**，什么区别呢？内存临时表使用的是 MEMORY 存储引擎，而临时表采用的是 MyISAM 存储引擎。



MEMORY 存储引擎：`memory` 是 MySQL 中一类特殊的存储引擎，它使用存储在内存中的内容来创建表，而且数据全部放在内存中。每个基于 MEMORY 存储引擎的表实际对应一个磁盘文件。该文件的文件名与表名相同，类型为 `frm` 类型。而其数据文件，都是存储在内存中，这样有利于数据的快速处理，提高整个表的效率。MEMORY 用到的很少，因为它是把数据存到内存中，如果内存出现异常就会影响数据。如果重启或者关机，所有数据都会消失。因此，基于 MEMORY 的表的生命周期很短，一般是一次性的。

MySQL 会在下面几种情况产生临时表

- 使用 UNION 查询：UNION 有两种，一种是 `UNION`，一种是 `UNION ALL`，它们都用于联合查询；区别是 使用 UNION 会去掉两个表中的重复数据，相当于对结果集做了一下 `去重 (distinct)`。使用 UNION ALL，则不会排重，返回所有的行。使用 UNION 查询会产生临时表。
- 使用 `TEMPTABLE` 算法 或者是 UNION 查询中的视图。TEMPTABLE 算法是一种创建临时表的算法，它是将结果放置到临时表中，意味这要 MySQL 要先创建好一个临时表，然后将结果放到临时表中去，然后再使用这个临时表进行相应的查询。
- ORDER BY 和 GROUP BY 的子句不一样时也会产生临时表。
- DISTINCT 查询并且加上 ORDER BY 时；
- SQL中用到 `SQL_SMALL_RESULT` 选项时；如果查询结果比较小的时候，可以加上 `SQL_SMALL_RESULT` 来优化，产生临时表
- FROM 中的子查询；
- EXPLAIN 查看执行计划结果的 Extra 列中，如果使用 `Using Temporary` 就表示会用到临时表。

## MySQL 常见索引类型

索引是存储在一张表中特定列上的 `数据结构`，索引是在列上创建的。并且，索引是一种数据结构。

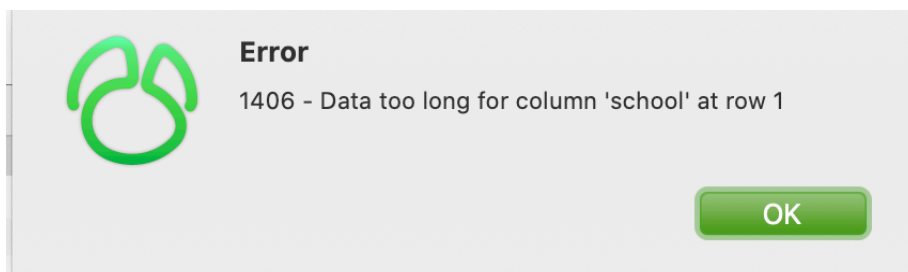
在 MySQL 中，主要有下面几种索引

- `全局索引(FULLTEXT)`：全局索引，目前只有 MyISAM 引擎支持全局索引，它的出现是为了解决针对文本的模糊查询效率较低的问题。
- `哈希索引(HASH)`：哈希索引是 MySQL 中用到的唯一 key-value 键值对的数据结构，很适合作为索引。HASH 索引具有一次定位的好处，不需要像树那样逐个节点查找，但是这种查找适合应用于查找单个键的情况，对于范围查找，HASH 索引的性能就会很低。
- `B-Tree 索引`：B 就是 Balance 的意思，BTree 是一种平衡树，它有很多变种，最常见的就是 B+ Tree，它被 MySQL 广泛使用。
- `R-Tree 索引`：R-Tree 在 MySQL 很少使用，仅支持 geometry 数据类型，支持该类型的存储引擎只有 MyISAM、BDb、InnoDB、NDb、Archive 几种，相对于 B-Tree 来说，R-Tree 的优势在于范围查找。

## varchar 和 char 的区别和使用场景

MySQL 中没有 nvarchar 数据类型，所以直接比较的是 varchar 和 char 的区别

`char`：表示的是 `定长` 的字符串，当你输入小于指定的数目，比如你指定的数目是 `char(6)`，当你输入小于 6 个字符的时候，char 会在你最后一个字符后面补空值。当你输入超过指定允许最大长度后，MySQL 会报错



**varchar**： varchar 指的是长度为 n 个字节的可变长度，并且是 **非Unicode** 的字符数据。n 的值是介于 1 - 8000 之间的数值。存储大小为实际大小。

Unicode 是一种字符编码方案，它为每种语言中的每个字符都设定了统一唯一的二进制编码，以实现跨语言、跨平台进行文本转换、处理的要求

使用 char 存储定长的数据非常方便、char 检索效率高，无论你存储的数据是否到了 10 个字节，都要去占用 10 字节的空间

使用 varchar 可以存储变长的数据，但存储效率没有 char 高。

## 什么是 内连接、外连接、交叉连接、笛卡尔积

连接的方式主要有三种：外连接、内连接、交叉连接

- 外连接(OUTER JOIN)**：外连接分为三种，分别是 **左外连接(LEFT OUTER JOIN 或 LEFT JOIN)**、**右外连接(RIGHT OUTER JOIN 或 RIGHT JOIN)**、**全外连接(FULL OUTER JOIN 或 FULL JOIN)**

左外连接：又称为左连接，这种连接方式会显示左表不符合条件的数据行，右边不符合条件的数据行直接显示 NULL

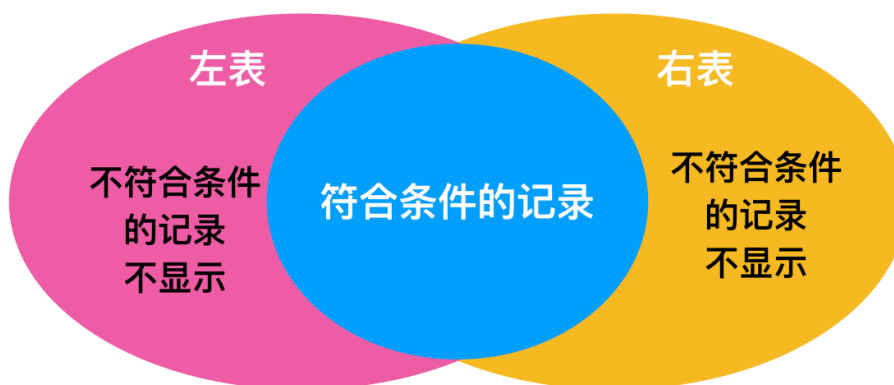


右外连接：也被称为右连接，他与左连接相对，这种连接方式会显示右表不符合条件的数据行，左表不符合条件的数据行直接显示 NULL



### MySQL 暂不支持全外连接

- **内连接(INNER JOIN)**：结合两个表中相同的字段，返回关联字段相符的记录。



- **笛卡尔积(Cartesian product)**：我在上面提到了笛卡尔积，为了方便，下面再列出来一下。

现在我们有二个集合  $A = \{0,1\}$  ,  $B = \{2,3,4\}$

那么，集合  $A * B$  得到的结果就是

$A * B = \{(0,2)、(1,2)、(0,3)、(1,3)、(0,4)、(1,4)\};$

$B * A = \{(2,0)、(2,1)、(3,0)、(3,1)、(4,0)、(4,1)\};$

上面  $A * B$  和  $B * A$  的结果就可以称为两个集合相乘的 **笛卡尔积**

我们可以得出结论，A 集合和 B 集合相乘，包含了集合 A 中的元素和集合 B 中元素之和，也就是 A 元素的个数 \* B 元素的个数

- 交叉连接的原文是 **Cross join**，就是笛卡尔积在 SQL 中的实现，SQL 中使用关键字 **CROSS JOIN** 来表示交叉连接，在交叉连接中，随便增加一个表的字段，都会对结果造成很大的影响。

```
1 SELECT * FROM t_Class a CROSS JOIN t_Student b WHERE a.classid=b.classid
```

或者不用 CROSS JOIN，直接用 FROM 也能表示交叉连接的效果

```
1 SELECT * FROM t_Class a ,t_Student b WHERE a.classid=b.classid
```

如果表中字段比较多，不适宜用交叉连接，交叉连接的效率比较差。

- 全连接：全连接也就是 `full join`，MySQL 中不支持全连接，但是可以使用其他连接查询来模拟全连接，可以使用 `UNION` 和 `UNION ALL` 进行模拟。例如

```
1 (select column1,column2...columnN from tableA) union (select
 column1,column2...columnN from tableB)
2
3
4 或
5 (select column1,column2...columnN from tableA) union all (select
 column1,column2...columnN from tableB);
```

使用 UNION 和 UNION ALL 的注意事项

通过 union 连接的 SQL 分别单独取出的列数必须相同

使用 union 时，多个相等的行将会被合并，由于合并比较耗时，一般不直接使用 union 进行合并，而是通常采用 union all 进行合并

## 谈谈 SQL 优化的经验

- 查询语句无论是使用哪种判断条件 等于、小于、大于，`WHERE` 左侧的条件查询字段不要使用函数或者表达式
- 使用 `EXPLAIN` 命令优化你的 SELECT 查询，对于复杂、效率低的 sql 语句，我们通常是使用 `explain sql` 来分析这条 sql 语句，这样方便我们分析，进行优化。
- 当你的 SELECT 查询语句只需要使用一条记录时，要使用 `LIMIT 1`
- 不要直接使用 `SELECT *`，而应该使用具体需要查询的表字段，因为使用 `EXPLAIN` 进行分析时，`SELECT *` 使用的是全表扫描，也就是 `type = all`。
- 为每一张表设置一个 ID 属性
- 避免在 `WHERE` 字句中对字段进行 `NULL` 判断
- 避免在 `WHERE` 中使用 `!=` 或 `<>` 操作符
- 使用 `BETWEEN AND` 替代 `IN`
- 为搜索字段创建索引
- 选择正确的存储引擎，InnoDB、MyISAM、MEMORY 等
- 使用 `LIKE %abc%` 不会走索引，而使用 `LIKE abc%` 会走索引
- 对于枚举类型的字段(即有固定罗列值的字段)，建议使用 `ENUM` 而不是 `VARCHAR`，如性别、星期、类型、类别等
- 拆分大的 DELETE 或 INSERT 语句
- 选择合适的字段类型，选择标准是 尽可能小、尽可能定长、尽可能使用整数。
- 字段设计尽可能使用 `NOT NULL`
- 进行水平切割或者垂直分割

水平分割：通过建立结构相同的几张表分别存储数据

垂直分割：将经常一起使用的字段放在一个单独的表中，分割后的表记录之间是一一对应关系。

## 操作系统篇

## 解释一下什么是操作系统

操作系统是运行在计算机上最重要的一种 **软件**，它管理计算机的资源 and 进程以及所有的硬件和软件。它为计算机硬件和软件提供了一种中间层



操作系统所处的位置

通常情况下，计算机上会运行着许多应用程序，它们都需要对内存和 CPU 进行交互，操作系统的目的就是为了保证这些访问和交互能够准确无误的进行。

## 解释一下操作系统的主要目的是什么

操作系统是一种软件，它的主要目的有三种

- 管理计算机资源，这些资源包括 CPU、内存、磁盘驱动器、打印机等。
- 提供一种图形界面，就像我们前面描述的那样，它提供了用户和计算机之间的桥梁。
- 为其他软件提供服务，操作系统与软件进行交互，以便为其分配运行所需的任何必要资源。

## 操作系统的种类有哪些

操作系统通常预装在你购买计算机之前。大部分用户都会使用默认的操作系统，但是你也可以升级甚至更改操作系统。但是一般常见的操作系统只有三种：**Windows**、**macOS** 和 **Linux**。

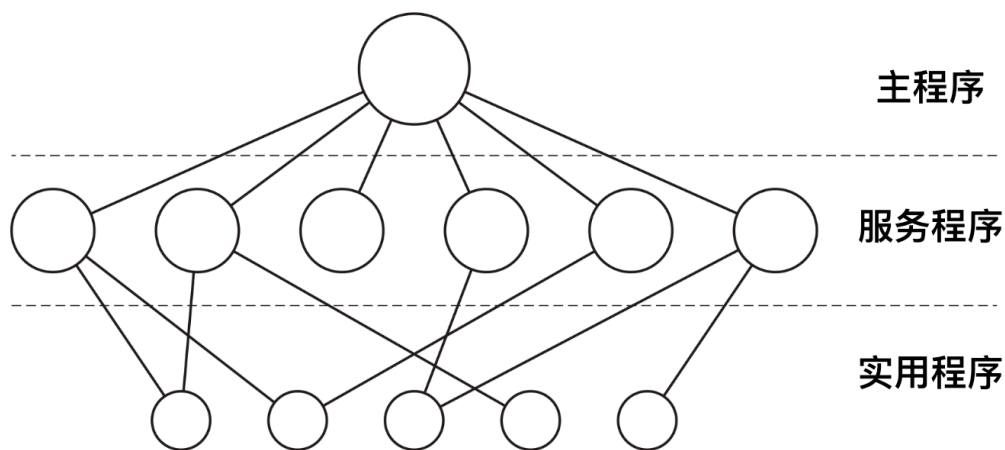
## 操作系统结构

### 单体系统

在大多数系统中，整个系统在内核态以单一程序的方式运行。整个操作系统是以程序集合来编写的，链接在一块形成一个大的二进制可执行程序，这种系统称为单体系统。

在单体系统中构造实际目标程序时，会首先编译所有单个过程（或包含这些过程的文件），然后使用系统链接器将它们全部绑定到一个可执行文件中

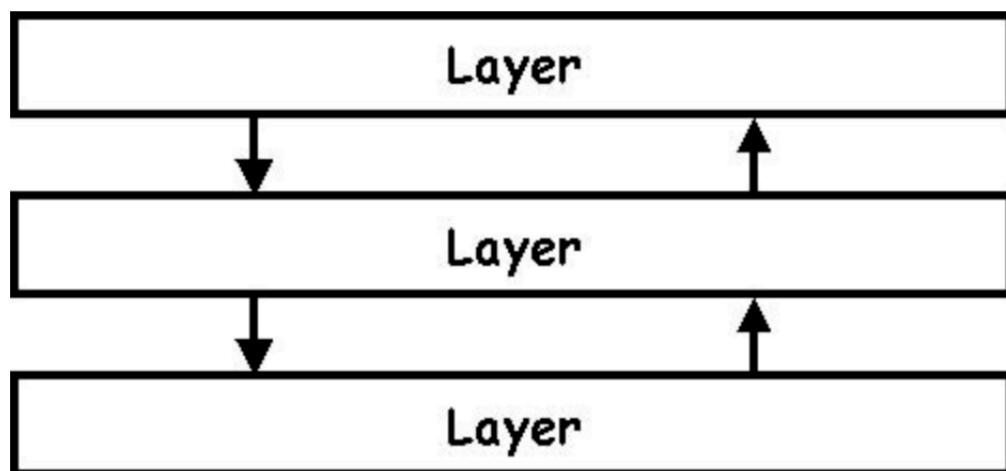
在单体系统中，对于每个系统调用都会有一个服务程序来保障和运行。需要一组实用程序来弥补服务程序需要的功能，例如从用户程序中获取数据。可将各种过程划分为一个三层模型



除了在计算机初启动时所装载的核心操作系统外，许多操作系统还支持额外的扩展。比如 I/O 设备驱动和文件系统。这些部件可以按需装载。在 UNIX 中把它们叫做 **共享库(shared library)**，在 Windows 中则被称为 **动态链接库(Dynamic Link Library,DLL)**。他们的扩展名为 **.dll**，在 **C:\Windows\system32** 目录下存在 1000 多个 DLL 文件，所以不要轻易删除 C 盘文件，否则可能就炸了哦。

## 分层系统

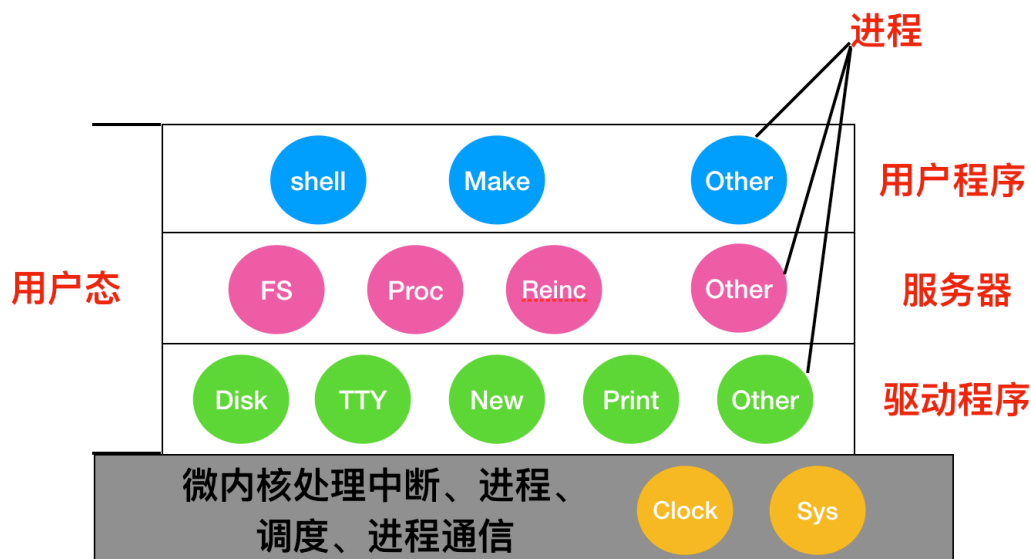
分层系统使用层来分隔不同的功能单元。每一层只与该层的上层和下层通信。每一层都使用下面的层来执行其功能。层之间的通信通过预定义的固定接口通信。



## 微内核

为了实现高可靠性，将操作系统划分成小的、层级之间能够更好定义的模块是很有必要的，只有一个模块 --- 微内核 --- 运行在内核态，其余模块可以作为普通用户进程运行。由于把每个设备驱动和文件系统分别作为普通用户进程，这些模块中的错误虽然会使这些模块崩溃，但是不会使整个系统死机。

**MINIX 3** 是微内核的代表作，它的具体结构如下



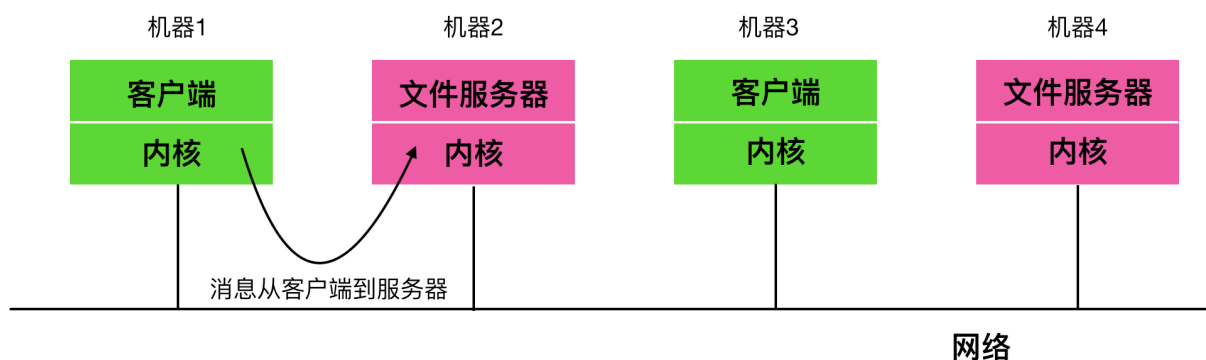
### MINI 3 系统结构

在内核的外部，系统的构造有三层，它们都在用户态下运行，最底层是设备驱动器。由于它们都在用户态下运行，所以不能物理的访问 I/O 端口空间，也不能直接发出 I/O 命令。相反，为了能够对 I/O 设备编程，驱动器构建一个结构，指明哪个参数值写到哪个 I/O 端口，并声称一个内核调用，这样就完成了一次调用过程。

### 客户-服务器模式

微内核思想的策略是把进程划分为两类：**服务器**，每个服务器用来提供服务；**客户端**，使用这些服务。这个模式就是所谓的 **客户-服务器** 模式。

客户-服务器模式会有两种载体，一种情况是一台计算机既是客户又是服务器，在这种方式下，操作系统会有某种优化；但是普遍情况下是客户端和服务器在不同的机器上，它们通过局域网或广域网连接。



客户通过发送消息与服务器通信，客户端并不需要知道这些消息是在本地机器上处理，还是通过网络被送到远程机器上处理。对于客户端而言，这两种情形是一样的：都是发送请求并得到回应。

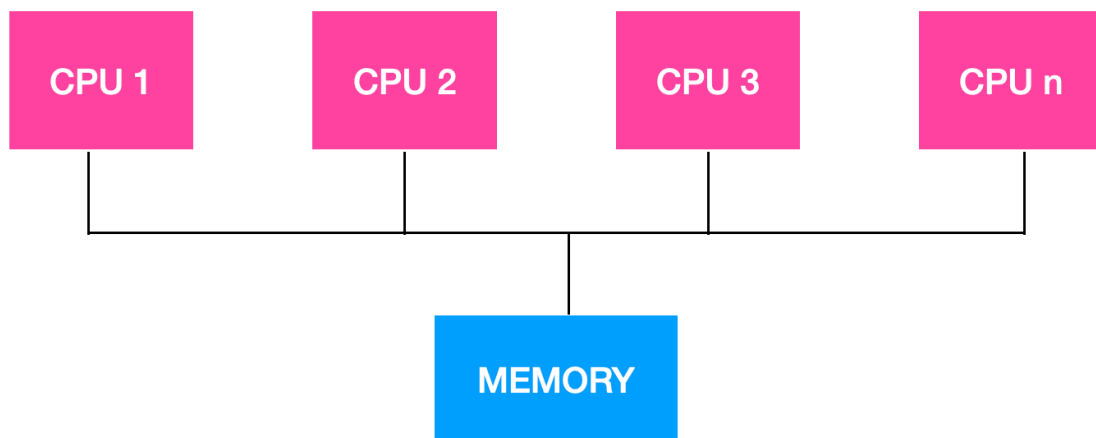
### 什么是按需分页

在操作系统中，进程是以页为单位加载到内存中的，按需分页是一种 **虚拟内存** 的管理方式。在使用请求分页的系统中，只有在尝试访问页面所在的磁盘并且该页面尚未在内存中时，也就发生了 **缺页异常**，操作系统才会将磁盘页面复制到内存中。



## 多处理系统的优势

随着处理器的不断增加，我们的计算机系统由单机系统变为了多处理系统，多处理系统的吞吐量比较高，多处理系统拥有多个并行的处理器，这些处理器共享时钟、内存、总线、外围设备等。



多处理系统由于可以共享资源，因此可以开源节流，省钱。整个系统的可靠性也随之提高。

## 什么是内核

在计算机中，内核是一个计算机程序，它是操作系统的核心，可以控制操作系统中所有的内容。内核通常是在 boot loader 装载程序之前加载的第一个程序。

这里还需要了解一下什么是 **boot loader**。

boot loader 又被称为引导加载程序，它是一个程序，能够将计算机的操作系统放入内存中。在电源通电或者计算机重启时，BIOS 会执行一些初始测试，然后将控制权转移到引导加载程序所在的主引导记录(MBR)。

## 什么是实时系统

实时操作系统对时间做出了严格的要求，实时操作系统分为两种：**硬实时**和**软实时**

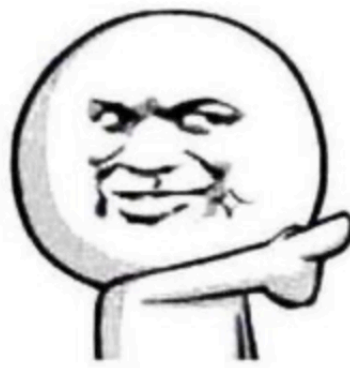
**硬实时操作系统** 规定某个动作必须在规定的时刻内完成或发生，比如汽车生产车间，焊接机器必须在某一时刻内完成焊接，焊接的太早或者太晚都会对汽车造成永久性伤害。

**软实时操作系统** 虽然不希望偶尔违反最终的时限要求，但是仍然可以接受。并且不会引起任何永久性伤害。比如数字音频、多媒体、手机都是属于软实时操作系统。

你可以简单理解硬实时和软实时的两个指标：**是否在时刻内必须完成以及是否造成严重损害**。

## 什么是虚拟内存

**虚拟内存** 是一种内存分配方案，是一项可以用来辅助内存分配的机制。我们知道，应用程序是按页装载进内存中的。但并不是所有的页都会装载到内存中，计算机中的硬件和软件会将数据从 RAM 临时传输到磁盘中来弥补内存的不足。如果没有虚拟内存的话，一旦你将计算机内存填满后，计算机会对你说



后退，我要开始装逼了

呃，不，对不起，您无法再加载任何应用程序，请关闭另一个应用程序以加载新的应用程序。对于虚拟内存，计算机可以执行操作是查看内存中最近未使用过的区域，然后将其复制到硬盘上。虚拟内存通过复制技术实现了 妹子，你快来看哥哥能装这么多程序 的资本。复制是自动进行的，你无法感知到它的存在。

## 什么是进程和进程表

**进程** 就是正在执行程序的实例，比如说 Web 程序就是一个进程，shell 也是一个进程，文章编辑器 typora 也是一个进程。

操作系统负责管理所有正在运行的进程，操作系统会为每个进程分配特定的时间来占用 CPU，操作系统还会为每个进程分配特定的资源。

操作系统为了跟踪每个进程的活动状态，维护了一个 **进程表**。在进程表的内部，列出了每个进程的状态以及每个进程使用的资源等。

<http://courses.cs.vt.edu/csonline/OS/Lessons/Processes/index.html> 这个网站上面有一个关于进程状态轮转的动画，做的真是太好了。

## 什么是线程，线程和进程的区别

这又是一道老生常谈的问题了，从操作系统的角度来回答一下吧。

我们上面说到进程是正在运行的程序的实例，而线程其实就是进程中的单条流向，因为线程具有进程中的某些属性，所以线程又被称为轻量级的进程。浏览器如果是一个进程的话，那么浏览器下面的每个 tab 页可以看作是一个个的线程。

下面是线程和进程持有资源的区别

| 每个进程中的内容  | 每个线程中的内容 |
|-----------|----------|
| 地址空间      | 程序计数器    |
| 全局变量      | 寄存器      |
| 打开文件      | 堆栈       |
| 子进程       | 状态       |
| 即将发生的定时器  |          |
| 信号与信号处理程序 |          |
| 账户信息      |          |

线程不像进程那样具有很强的独立性，线程之间会共享数据

创建线程的开销要比进程小很多，因为创建线程仅仅需要 **堆栈指针** 和 **程序计数器** 就可以了，而创建进程需要操作系统分配新的地址空间，数据资源等，这个开销比较大。

## 使用多线程的好处是什么

多线程是程序员不得不知的基本素养之一，所以，下面我们给出一些多线程编程的好处

- 能够提高对用户的响应顺序
- 在流程中的资源共享
- 比较经济适用
- 能够对多线程架构有深入的理解

## 什么是 RR 调度算法

**RR(round-robin)** 调度算法主要针对分时系统，RR 的调度算法会把时间片以相同的部分并循环的分配给每个进程，RR 调度算法没有优先级的概念。这种算法的实现比较简单，而且每个线程都会占有时间片，并不存在线程饥饿的问题。

## 导致系统出现死锁的情况

死锁的出现需要同时满足下面四个条件

- **互斥(Mutual Exclusion)**：一次只能有一个进程使用资源。如果另一个进程请求该资源，则必须延迟请求进程，直到释放该资源为止。
- **保持并等待(Hold and Wait)**：必须存在一个进程，该进程至少持有一个资源，并且正在等待获取其他进程当前所持有的资源。
- **无抢占(No Preemption)**：资源不能被抢占，也就是说，在进程完成其任务之后，只能由拥有它的进程自动释放资源。
- **循环等待(Circular Wait)**：必须存在一组 {p0, p1, ..... pn} 的等待进程，使 p0 等待 p1 持有的资源，p1 等待由 p2 持有的资源，pn-1 正在等待由 pn 持有的资源，而 pn 正在等待由 p0 持有的资源。

## RAID 的不同级别

RAID 称为 **磁盘冗余阵列**，简称 **磁盘阵列**。利用虚拟化技术把多个硬盘结合在一起，成为一个或多个磁盘阵列组，目的是提升性能或数据冗余。

RAID 有不同的级别

- RAID 0 - 无容错的条带化磁盘阵列
- RAID 1 - 镜像和双工
- RAID 2 - 内存式纠错码
- RAID 3 - 比特交错奇偶校验
- RAID 4 - 块交错奇偶校验
- RAID 5 - 块交错分布式奇偶校验
- RAID 6 - P + Q冗余

## 什么是 DMA

DMA 的中文名称是 **直接内存访问**，它意味着 CPU 授予 I/O 模块权限在不涉及 CPU 的情况下读取或写入内存。也就是 DMA 可以不需要 CPU 的参与。这个过程由称为 DMA 控制器（DMAC）的芯片管理。由于 DMA 设备可以直接在内存之间传输数据，而不是使用 CPU 作为中介，因此可以缓解总线上的拥塞。DMA 通过允许 CPU 执行任务，同时 DMA 系统通过系统和内存总线传输数据来提高系统并行性。

## 多线程编程的好处是什么

对不起，我忍不住想偷笑



说直白点，为什么单线程能够处理的却要用多线程来处理？当然是为了提高程序的装逼并行能力了。多线程 **在某些情况下** 能够使你程序运行的更快，这也是为什么多核 CPU 会出现，但是多核 CPU 的出现会导致数据的一致性问题，不过这些问题程序员就能解决。另一个角度来说，多线程编程能够提高程序员的编程能力和编程思维。同时也能提高程序员的管理能力，你如果把每条线程流当作罗老师时间管理的女主一样，能够及时协调好所有P友的关系，那你也是超神程序员了，所以，是谁说程序员不会做管理的？Doug Lea 大佬牛逼！！

ps：Doug Lea 大佬开发的 JUC 工具包，此处不加狗头。

## 什么是设备驱动程序

在计算机中，设备驱动程序是一种计算机程序，它能够控制或者操作连接到计算机的特定设备。驱动程序提供了与硬件进行交互的软件接口，使操作系统和其他计算机程序能够访问特定设备，不用需要了解其硬件的具体构造。

## 进程间的通信方式

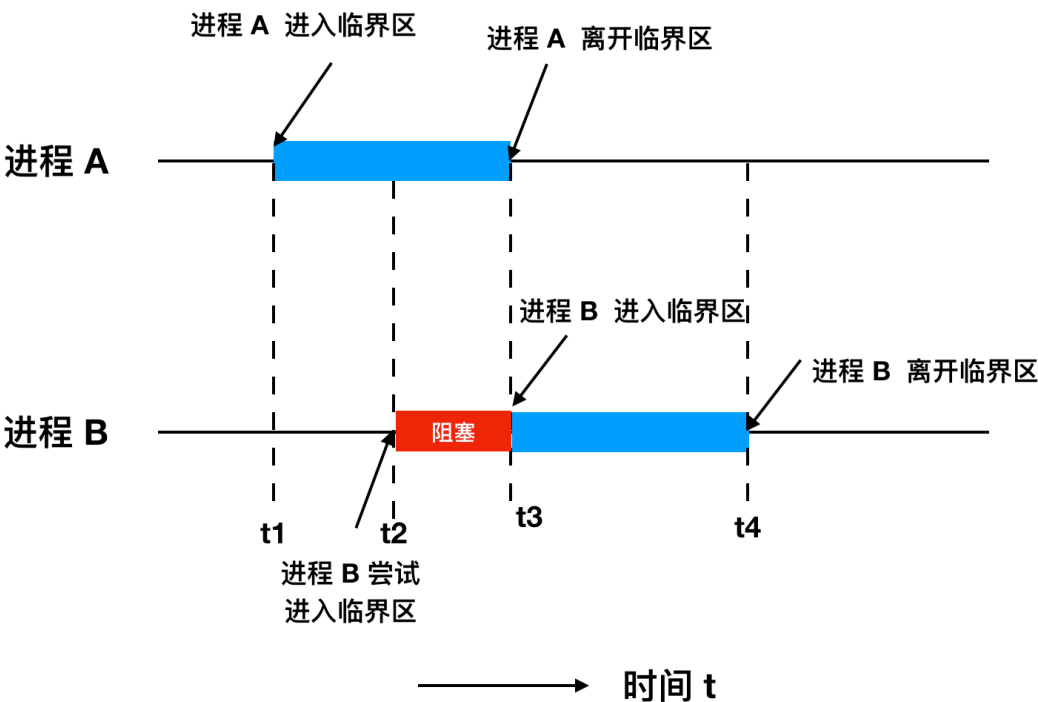
### 通信概念

进程间的通信方式比较多，首先你需要理解下面这几个概念

- 竞态条件：即两个或多个线程同时对一共享数据进行修改，从而影响程序运行的正确性时，这种就被称为 **竞态条件(race condition)**。
- 临界区：不仅 **共享资源** 会造成竞态条件，事实上共享文件、共享内存也会造成竞态条件、那么该如何避免呢？或许一句话可以概括说明：**禁止一个或多个进程在同一时刻对共享资源（包括共享内存、共享文件等）进行读写**。换句话说，我们需要一种 **互斥(mutual exclusion)** 条件，这也就是说，如果一个进程在某种方式下使用共享变量和文件的话，除该进程之外的其他进程就禁止做这种事（访问统一资源）。

一个好的解决方案，应该包含下面四种条件

1. 任何时候两个进程不能同时处于临界区
2. 不应对 CPU 的速度和数量做任何假设
3. 位于临界区外的进程不得阻塞其他进程
4. 不能使任何进程无限等待进入临界区



使用临界区的互斥

- 忙等互斥：当一个进程在对资源进行修改时，其他进程必须进行等待，进程之间要具有互斥性，我们讨论的解决方案其实都是基于忙等互斥提出的。

### 解决方案

进程间的通信用专业一点的术语来表示就是 `Inter Process Communication, IPC`，它主要有下面几种通信方式

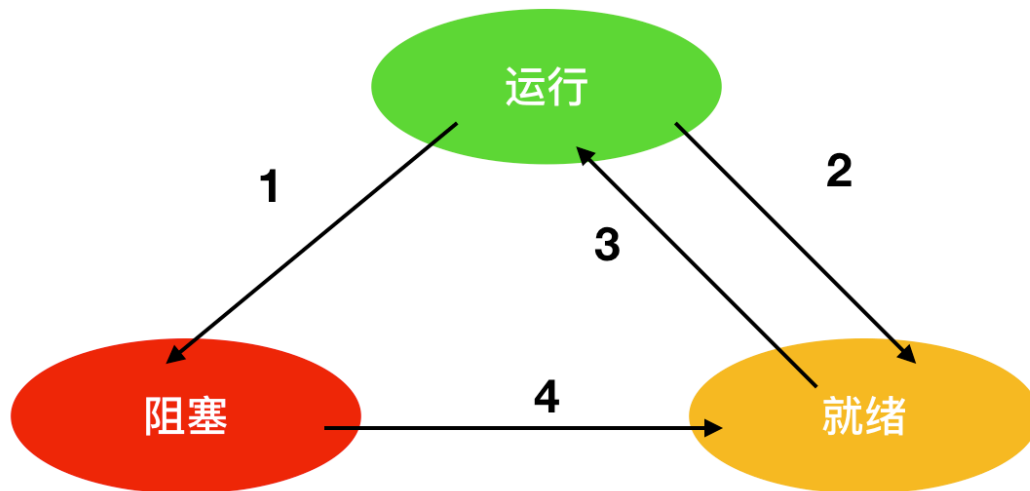


- **消息传递**：消息传递是进程间实现通信和同步等待的机制，使用消息传递，进程间的交流不需要共享变量，直接就可以进行通信；消息传递分为发送方和接收方
- **先进先出队列**：先进先出队列指的是两个不相关联进程间的通信，两个进程之间可以彼此相互进程通信，这是一种全双工通信方式
- **管道**：管道用于两个相关进程之间的通信，这是一种半双工的通信方式，如果需要全双工，需要另外一个管道。
- **直接通信**：在这种进程通信的方式中，进程与进程之间只存在一条链接，进程间要明确通信双方的命名。
- **间接通信**：间接通信是通信双方不会直接建立连接，而是找到一个中介者，这个中介者可能是个对象等等，进程可以在其中放置消息，并且可以从中删除消息，以此达到进程间通信的目的。
- **消息队列**：消息队列是内核中存储消息的链表，它由消息队列标识符进行标识，这种方式能够在不同的进程之间提供全双工的通信连接。
- **共享内存**：共享内存是使用所有进程之间的内存来建立连接，这种类型需要同步进程访问来相互保护。

## 进程间状态模型

第一个进程是 `cat`，将三个文件级联并输出。第二个进程是 `grep`，它从输入中选择具有包含关键字 `tree` 的内容，根据这两个进程的相对速度（这取决于两个程序的相对复杂度和各自所分配到的 CPU 时间片），可能会发生下面这种情况，`grep` 准备就绪开始运行，但是输入进程还没有完成，于是必须阻塞 `grep` 进程，直到输入完毕。

当一个进程开始运行时，它可能会经历下面这几种状态



- 第一种是进程因为等待输入而阻塞
- 第二种是调度程序选择另一个进程
- 第三种是调度程序选择一个进程开始运行
- 第四种是出现有效的输入

进程间的状态切换图

图中会涉及三种状态

1. **运行态**，运行态指的就是进程实际占用 CPU 时间片运行时
2. **就绪态**，就绪态指的是可运行，但因为其他进程正在运行而处于就绪状态
3. **阻塞态**，除非某种外部事件发生，否则进程不能运行

逻辑上来说，运行态和就绪态是很相似的。这两种情况下都表示进程 **可运行**，但是第二种情况没有获得 CPU 时间分片。第三种状态与前两种状态不同的原因是这个进程不能运行，CPU 空闲时也不能运行。

三种状态会涉及四种状态间的切换，在操作系统发现进程不能继续执行时会发生 **状态1** 的轮转，在某些系统中进程执行系统调用，例如 `pause`，来获取一个阻塞的状态。在其他系统中包括 UNIX，当进程从管道或特殊文件（例如终端）中读取没有可用的输入时，该进程会被自动终止。

转换 2 和转换 3 都是由进程调度程序（操作系统的一部分）引起的，进程本身不知道调度程序的存在。转换 2 的出现说明进程调度器认定当前进程已经运行了足够长的时间，是时候让其他进程运行 CPU 时间片了。当所有其他进程都运行过后，这时候该是让第一个进程重新获得 CPU 时间片的时候了，就会发生转换 3。

程序调度指的是，决定哪个进程优先被运行和运行多久，这是很重要的一点。已经设计出许多算法来尝试平衡系统整体效率与各个流程之间的竞争需求。



当进程等待的一个外部事件发生时（如从外部输入一些数据后），则发生转换 4。如果此时没有其他进程在运行，则立刻触发转换 3，该进程便开始运行，否则该进程会处于就绪阶段，等待 CPU 空闲后再轮到它运行。

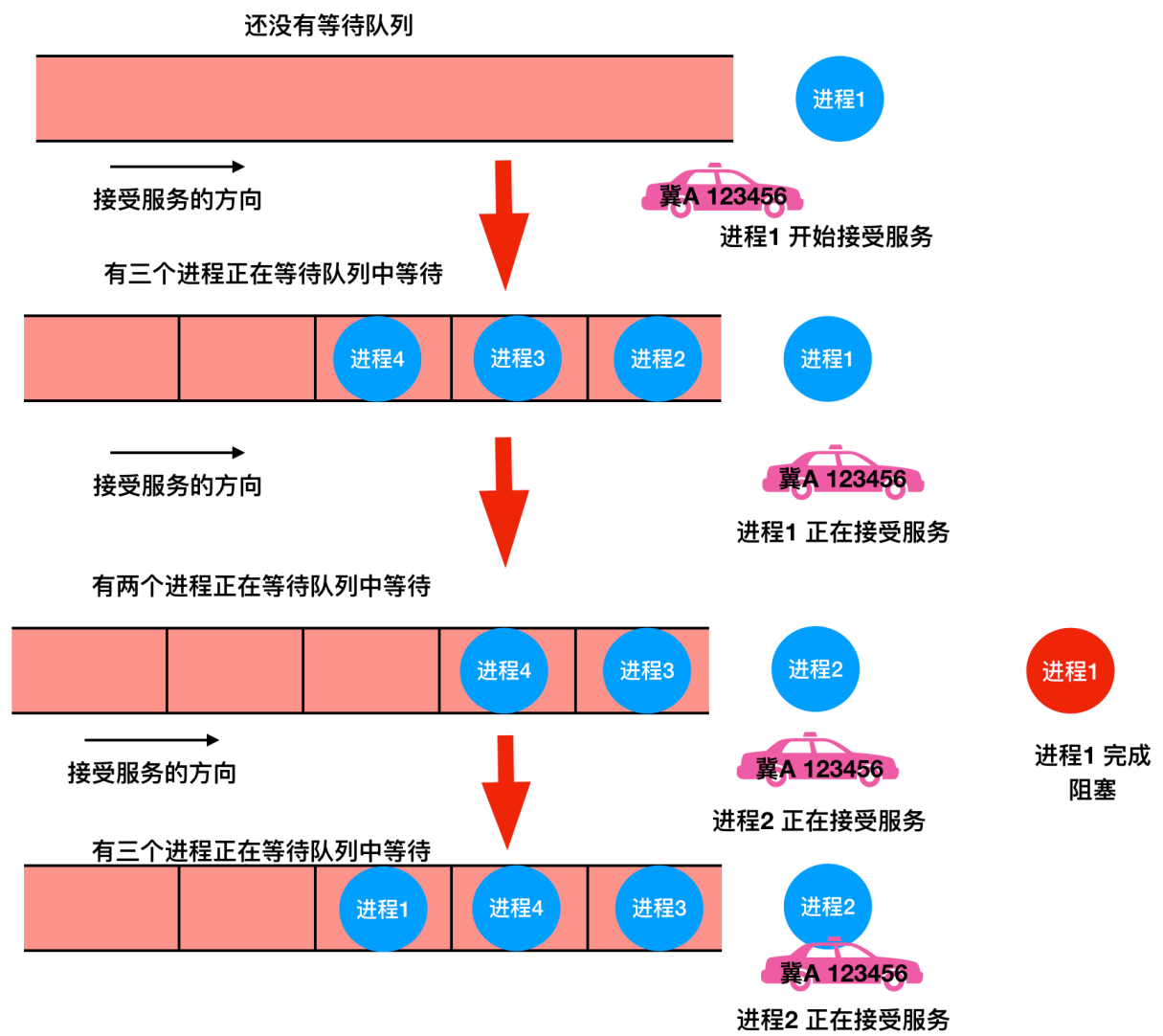
## 调度算法都有哪些

调度算法分为三大类：批处理中的调度、交互系统中的调度、实时系统中的调度

### 批处理中的调度

#### 先来先服务

很像是先到先得。。。可能最简单的非抢占式调度算法的设计就是 **先来先服务(first-come,first-serverd)**。使用此算法，将按照请求顺序为进程分配 CPU。最基本的，会有一个就绪进程的等待队列。当第一个任务从外部进入系统时，将会立即启动并允许运行任意长的时间。它不会因为运行时间太长而中断。当其他作业进入时，它们排到就绪队列尾部。当正在运行的进程阻塞，处于等待队列的第一个进程就开始运行。当一个阻塞的进程重新处于就绪态时，它会像一个新到达的任务，会排在队列的末尾，即排在所有进程最后。

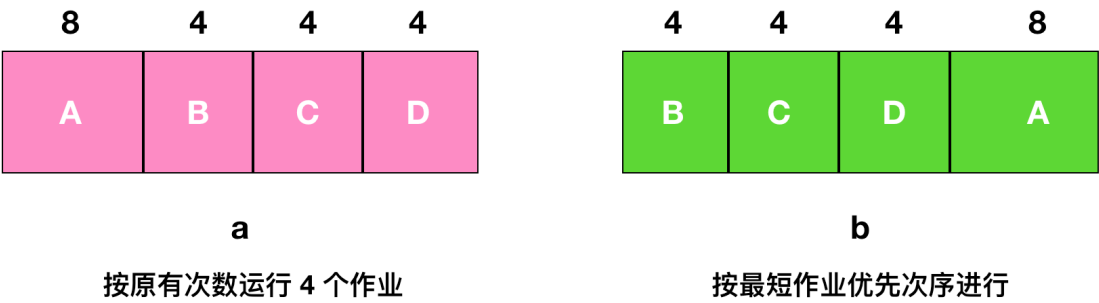


这个算法的强大之处在于易于理解和编程，在这个算法中，一个单链表记录了所有就绪进程。要选取一个进程运行，只要从该队列的头部移走一个进程即可；要添加一个新的作业或者阻塞一个进程，只要把这个作业或进程附加在队列的末尾即可。这是很简单的一种实现。

不过，先来先服务也是有缺点的，那就是没有优先级的关系，试想一下，如果有 100 个 I/O 进程正在排队，第 101 个是一个 CPU 密集型进程，那岂不是需要等 100 个 I/O 进程运行完毕才会等到一个 CPU 密集型进程运行，这在实际情况根本不可能，所以需要优先级或者抢占式进程的出现来优先选择重要的进程运行。

### 最短作业优先

批处理中，第二种调度算法是 **最短作业优先(Shortest Job First)**，我们假设运行时间已知。例如，一家保险公司，因为每天要做类似的工作，所以人们可以相当精确地预测处理 1000 个索赔的一批作业需要多长时间。当输入队列中有若干个同等重要的作业被启动时，调度程序应使用最短优先作业算法



如上图 a 所示，这里有 4 个作业 A、B、C、D，运行时间分别为 8、4、4、4 分钟。若按图中的次序运行，则 A 的周转时间为 8 分钟，B 为 12 分钟，C 为 16 分钟，D 为 20 分钟，平均时间内为 14 分钟。

现在考虑使用最短作业优先算法运行 4 个作业，如上图 b 所示，目前的周转时间分别为 4、8、12、20，平均为 11 分钟，可以证明最短作业优先是最优的。考虑有 4 个作业的情况，其运行时间分别为 a、b、c、d。第一个作业在时间 a 结束，第二个在时间 a + b 结束，以此类推。平均周转时间为  $(4a + 3b + 2c + d) / 4$ 。显然 a 对平均值的影响最大，所以 a 应该是最短优先作业，其次是 b，然后是 c，最后是 d 它就只能影响自己的周转时间了。

需要注意的是，在所有的进程都可以运行的情况下，最短作业优先的算法才是最优的。

### 最短剩余时间优先

最短作业优先的抢占式版本被称之为 **最短剩余时间优先(Shortest Remaining Time Next)** 算法。使用这个算法，调度程序总是选择剩余运行时间最短的那个进程运行。当一个新作业到达时，其整个时间同当前进程的剩余时间做比较。如果新的进程比当前运行进程需要更少的时间，当前进程就被挂起，而运行新的进程。这种方式能够使短期作业获得良好的服务。

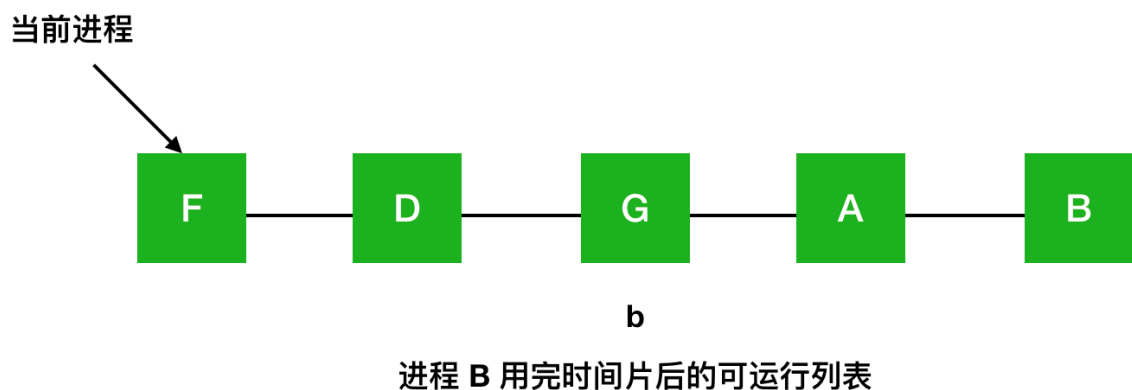
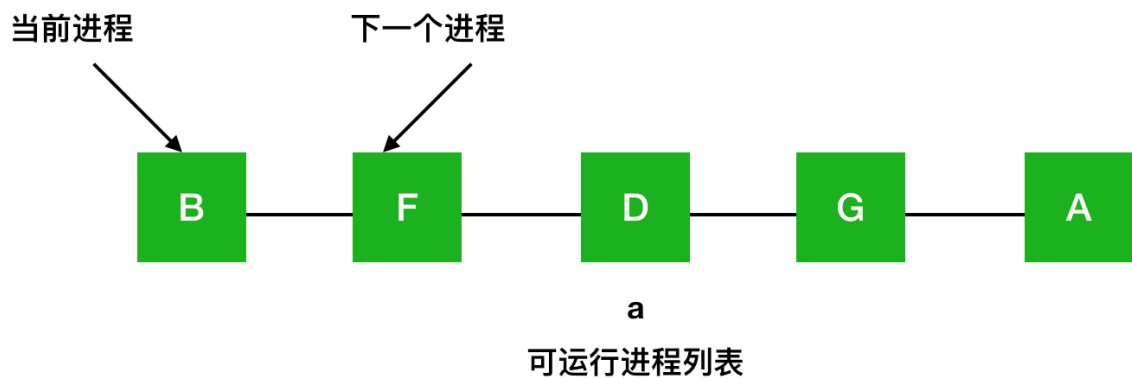
### 交互式系统中的调度

交互式系统中在个人计算机、服务器和其他系统中都是很常用的，所以有必要来探讨一下交互式调度

#### 轮询调度

一种最古老、最简单、最公平并且最广泛使用的算法就是 **轮询算法(round-robin)**。每个进程都会被分配一个时间段，称为 **时间片(quantum)**，在这个时间片内允许进程运行。如果时间片结束时进程还在运行的话，则抢占一个 CPU 并将其分配给另一个进程。如果进程在时间片结束前阻塞或结束，则 CPU 立即进行切换。轮询算法比较容易实现。调度程序所做的就是维护一个可运行进程的列表，就像

下图中的 a，当一个进程用完时间片后就被移到队列的末尾，就像下图的 b。



## 优先级调度

事实情况是不是所有的进程都是优先级相等的。例如，在一所大学中的等级制度，首先是院长，然后是教授、秘书、后勤人员，最后是学生。这种将外部情况考虑在内就实现了 **优先级调度(priority scheduling)**

某学院等级制度



院长 > 教授 > 秘书 > 后勤人员 > 学生

它的基本思想很明确，每个进程都被赋予一个优先级，优先级高的进程优先运行。

但是也不意味着高优先级的进程能够永远一直运行下去，调度程序会在每个时钟中断期间降低当前运行进程的优先级。如果此操作导致其优先级降低到下一个最高进程的优先级以下，则会发生进程切换。或者，可以为每个进程分配允许运行的最大时间间隔。当时间间隔用完后，下一个高优先级的进程会得到运行的机会。

## 最短进程优先

对于批处理系统而言，由于最短作业优先常常伴随着最短响应时间，一种方式是根据进程过去的行为进行推测，并执行估计运行时间最短的那一个。假设每个终端上每条命令的预估运行时间为  $T_0$ ，现在假设测量到其下一次运行时间为  $T_1$ ，可以用两个值的加权来改进估计时间，即  $aT_0 + (1 - a)T_1$ 。通过选择  $a$  的值，可以决定是尽快忘掉老的运行时间，还是在一段长时间内始终记住它们。当  $a = 1/2$

时，可以得到下面这个序列

$$T_0, \quad \frac{T_0}{2} + \frac{T_1}{2}, \quad \frac{T_0}{4} + \frac{T_1}{4} + \frac{T_2}{2}, \quad \frac{T_0}{8} + \frac{T_1}{8} + \frac{T_2}{4} + \frac{T_3}{2}$$

可以看到，在三轮过后， $T_0$  在新的估计值中所占比重下降至  $1/8$ 。

有时把这种通过当前测量值和先前估计值进行加权平均从而得到下一个估计值的技术称作 **老化 (aging)**。这种方法会使用很多预测值基于当前值的情况。

## 彩票调度

有一种既可以给出预测结果而又有一种比较简单的实现方式的算法，就是 **彩票调度 (lottery scheduling)** 算法。他的基本思想为进程提供各种系统资源的 **彩票**。当做出一个调度决策的时候，就随机抽出一张彩票，拥有彩票的进程将获得资源。比如在 CPU 进行调度时，系统可以每秒持有 50 次抽奖，每个中奖进程会获得额外运行时间的奖励。

可以把彩票理解为 buff，这个 buff 有 15% 的几率能让你产生 **速度之靴** 的效果。

## 公平分享调度

如果用户 1 启动了 9 个进程，而用户 2 启动了一个进程，使用轮转或相同优先级调度算法，那么用户 1 将得到 90 % 的 CPU 时间，而用户 2 将之得到 10 % 的 CPU 时间。

为了阻止这种情况的出现，一些系统在调度前会把进程的拥有者考虑在内。在这种模型下，每个用户都会分配一些 CPU 时间，而调度程序会选择进程并强制执行。因此如果两个用户每个都会有 50% 的 CPU 时间片保证，那么无论一个用户有多少个进程，都将获得相同的 CPU 份额。

## 页面置换算法都有哪些

| 算法            | 注释            |
|---------------|---------------|
| 最优算法          | 不可实现，但可以用作基准  |
| NRU(最近未使用) 算法 | 和 LRU 算法很相似   |
| FIFO(先进先出) 算法 | 有可能会抛弃重要的页面   |
| 第二次机会算法       | 比 FIFO 有较大的改善 |
| 时钟算法          | 实际使用          |
| LRU(最近最少)算法   | 比较优秀，但是很难实现   |
| NFU(最不经常食用)算法 | 和 LRU 很类似     |
| 老化算法          | 近似 LRU 的高效算法  |
| 工作集算法         | 实施起来开销很大      |
| 工作集时钟算法       | 比较有效的算法       |

- **最优算法** 在当前页面中置换最后要访问的页面。不幸的是，没有办法来判定哪个页面是最后一个要访问的，因此实际上该算法不能使用。然而，它可以作为衡量其他算法的标准。
- **NRU** 算法根据 R 位和 M 位的状态将页面氛围四类。从编号最小的类别中随机选择一个页面。NRU 算法易于实现，但是性能不是很好。存在更好的算法。
- **FIFO** 会跟踪页面加载进入内存中的顺序，并把页面放入一个链表中。有可能删除存在时间最长但是还在使用的页面，因此这个算法也不是一个很好的选择。
- **第二次机会** 算法是对 FIFO 的一个修改，它会在删除页面之前检查这个页面是否仍在使用。如果页面正在使用，就会进行保留。这个改进大大提高了性能。
- **时钟** 算法是第二次机会算法的另外一种实现形式，时钟算法和第二次算法的性能差不多，但是会花费更少的时间来执行算法。
- **LRU** 算法是一个非常优秀的算法，但是没有 **特殊的硬件(TLB)** 很难实现。如果没有硬件，就不能使用 LRU 算法。
- **NFU** 算法是一种近似于 LRU 的算法，它的性能不是非常好。
- **老化** 算法是一种更接近 LRU 算法的实现，并且可以更好的实现，因此是一个很好的选择
- 最后两种算法都使用了工作集算法。工作集算法提供了合理的性能开销，但是它的实现比较复杂。**WSClock** 是另外一种变体，它不仅能够提供良好的性能，而且可以高效地实现。

**最好的算法是老化算法和WSClock算法。**他们分别是基于 LRU 和工作集算法。他们都具有良好的性能并且能够被有效的实现。还存在其他一些好的算法，但实际上这两个可能是最重要的。

## 影响调度程序的指标是什么

会有下面几个因素决定调度程序的好坏

- CPU 使用率：

CPU 正在执行任务（即不处于空闲状态）的时间百分比。

- 等待时间

这是进程轮流执行的时间，也就是进程切换的时间

- 吞吐量

单位时间内完成进程的数量

- 响应时间

这是从提交流程到获得有用输出所经过的时间。

- 周转时间

从提交流程到完成流程所经过的时间。

## 什么是僵尸进程

僵尸进程是已完成且处于终止状态，但在进程表中却仍然存在的进程。僵尸进程通常发生在父子关系的进程中，由于父进程仍需要读取其子进程的退出状态所造成的。